

Towards Formal Idempotence Guarantees in Ansible Playbooks

Contents

1	Abstract	4
2	Introduction	5
2.1	Infrastructure as Code (IaC)	5
2.2	Examples of existing tools	5
2.3	Why Idempotence?	6
2.4	Goal	8
3	A Brief Look at Ansible Playbook	9
3.1	Ansible's Module System	9
3.2	Ansible's DSL	9
3.2.1	Tasks and Modules	10
3.2.2	Loops	10
3.2.3	Blocks	11
3.2.4	Plays and Playbooks	11
3.2.5	Handlers	11
3.2.6	Roles	11
3.2.7	Variables	11
3.2.8	Registers and when	11
3.3	When will we see these?	11
4	Existing Work	13
4.1	Code Smells	13
4.2	Property Checking	13
5	Approach	14
5.1	Key Properties of Ansible	14
5.2	Approach	14
6	BBFS	15
6.1	Filesystems and Paths in BBFS	15
6.2	Filesystem Transformations in BBFS	17
6.3	Executing our First BBFS Expression	18
6.4	Control Flow in BBFS	19
6.5	The Symbolic Analysis	20
6.6	Symbolic Semantics	24
6.7	BBFS Summary	26
6.7.1	Syntax	26
6.7.2	Concrete Semantics	27
6.7.3	Symbolic Semantics	27
6.7.4	Utility Functions	27
6.8	Proof of Correctness	28
6.8.1	Idempotence Proof	28
6.8.2	Soundness Proof	30
7	Formalizing and Desugaring Ansible	44
7.1	Extent of Formalization	44
7.2	Simplification	44
7.3	Syntax of $\mathcal{A}^H$	45
7.3.1	Preliminary notes	45

7.3.2	Syntax Proper	45
7.4	Syntax of $\mathcal{A}^{\min}$	47
7.5	Desugaring $\mathcal{A}^H$ into $\mathcal{A}^{\min}$	47
7.5.1	Desugaring Inline Roles in Plays	48
7.5.1.1	Utility Functions	51
7.5.1.2	Rewrite Rule	51
7.5.1.3	A Note on Handlers	51
7.5.2	Desugaring ImportTasks and ImportRole	51
7.5.2.1	Utility Functions	53
7.5.2.2	A Note on Handlers	54
7.5.2.3	Rewrite Rule	55
7.5.3	Desugaring Static Variables	55
7.5.3.1	Note on Type Constraints	56
7.5.3.2	Utility Functions	57
7.5.3.3	Rewrite Rule	59
7.5.4	Desugaring Blocks	59
7.5.4.1	Utility Functions	70
7.5.4.2	Rewrite Rule	73
7.5.5	Desugaring Loops	73
7.5.5.1	Rewrite Rule	75
7.5.6	Desugaring Handlers	75
7.5.6.1	Rewrite Rule	79
7.5.7	Desugaring Plays and Playbooks	79
7.5.7.1	Rewrite Rule	81
7.6	Summary of Rewrite Rules	81
8	From $\mathcal{A}^{\min}$ to BBFS	83
8.1	Translating Modules	83
8.1.1	Syntax	83
8.1.2	Semantics	83
8.1.3	Goalkeepers	84
8.2	Expanding Translation from Modules to Tasks	86
8.2.1	when Conversion	94
8.2.1.1	Formalization	95
8.2.2	ignore_errors Alteration	95
8.2.2.1	Formalization	96
9	Discussion	97
9.1	Effect Map, Invariant Checking and Stronger Properties	97
9.2	Meaning of BBFS	97
9.3	Practicality of BBFS	97
9.4	Time Complexity	98
9.5	Manual Translation and Translation Correction	98
9.6	Implementation	98
10	Future Work	99
10.1	Dynamic Variables	99
10.2	Support for include	99
10.3	Automating Module Translation	99
10.4	Regexes and Groups of Paths	99
10.5	Implementation	99
10.5.1	Expression Interpretation	100

Bibliography	101
--------------------	-----

1 Abstract

Ansible is a popular Infrastructure as Code (IaC) tool used widely in industry, whose DSL provides no checks or guarantees of idempotence, one of the most important properties for IaC tools. We present BBFS as a core calculus for black-box heavy filesystem operations, and design an idempotence/invariant analysis based on symbolic execution using BBFS, for which we prove correctness. Then, we identify a focus subset of Ansible, $\mathcal{A}^H$, and present rewrite rules to translationally define a minimal subset of Ansible, $\mathcal{A}^{\min}$. Finally, we present algorithms to translate $\mathcal{A}^{\min}$ to BBFS.

2 Introduction

2.1 Infrastructure as Code (IaC)

Infrastructure as Code, or IaC, is the practice of using machine-readable scripts to automate the process of provisioning instances and configuring software dependencies. Common examples include Chef, Puppet and Ansible.

Let us consider a student building a web service for his friends in his free time. He might decide to use an old laptop as his one and only server, upload his code onto it, and manually configure his home firewall to allow safe access. In doing so, his web service is now online. If he finds a bug, we can simply open the laptop and fix the code manually. If his friends' data are lost, there are likely no lasting consequences.

However, if that student is now a system administrator working at a tech company, whose job it is to maintain the computing infrastructure needed to provide the company's services, or to run internal processes, he needs a much higher degree of consistency, automation, and safety. When the tech company releases a new web service, he must provision perhaps 50 new servers appropriately, configure firewalls for each one depending on their region, and ensure that customer data is backed up. If the web service then receives spikes or lapses in traffic, he must then quickly provision more machines or take down machines in order to service the demand with the appropriate running cost. When a security risk is found in the software, he might have to patch the code on the servers and restart them in batches to minimize downtime.

In this case, manually performing these tasks can be cumbersome and prone to human error. Instead, IaC scripts might be used to automate provisioning, with the number of machines encapsulated as a parameter. If a firewall configuration or buggy script needs to be changed, it is edited once and the provisioning script is re-deployed to all machines.

Furthermore, if there are now two system administrators, or a whole team, the procedures for provisioning new servers are now codified and agreed upon in a more standardized form, rather than having to consult natural language descriptions of the appropriate procedures.

2.2 Examples of existing tools

Puppet

Puppet provides a declarative DSL for specifying resources (such as files or accounts), their desired end states, and interdependencies [1].

Nix

Nix is a package manager where deployment scripts are written in a functional DSL (the Nix language). It uses hashing to ensure exact reproducibility of builds and configurations. More recently, NixOS and NixOps have arisen to extend the use cases of Nix's fundamental system to cover more IaC use cases [2].

Chef

Chef uses a Ruby-based DSL in an imperative manner to directly modify the target system. In this sense it is most similar to Ansible among the examples we have discussed here; however, it uses a server-client architecture (as does Puppet) [3].

Ansible

Ansible uses a YAML-based DSL in an imperative manner. Unlike Chef, the computer running the script uses SSH to remote into target machines to perform tasks [4].

Ansible is owned by Red Hat, which has also made available a suite of IaC tools under the "Ansible" name, built around the core of the DSL and its interpreter. Such tools include Event-Driven Ansible,

which allows users to set up event-driven responses, as well as AI tools to generate Ansible's DSL. In this thesis, "Ansible" will be used to mean the core mechanism of the DSL and its interpreter, a system known more specifically as *Ansible Playbooks*.

2.3 Why Idempotence?

If a program is idempotent, it means that after executing it *successfully* n times, where $n > 1$, the result is the same as executing the program *successfully* once. This is a nice property that makes automated deployments and debugging easier by producing an effect similar to determinism, in that given a certain initial state, a program represents a predictable transform to a singular other state, because running the program 1 time, 5 times or 50 times will all have the same effect.

Let us return to our system administrator, who is now patching a bug on 50 instances, using an IaC tool such as Ansible. He writes a script that deletes the original buggy file, and creates a new, patched file. We can represent this script in pseudocode:

Non-idempotent IaC Script

```
| For all 50 machines:  
|   delete old_file  
|   create new_file
```

When he deploys this script to the 50 instances, all of them succeed in deleting the old file. However, due to a network issue, 10 of them fail to create the new file. We now have two machine states:

- 40 machines have new_file but not old_file
- 10 machines have neither new_file nor old_file

Our system administrator is now faced with a decision: He can either write a new, one-off script to target those 10 machines, or he can simply run the IaC script again on all 50 machines. If the latter were possible, it would not only be easier on the system administrator, but also easier to automate failure recovery: If this network issue is common, one may simply create a cron job that runs the IaC script again for all machines after 5 minutes.

However, if we run the above non-idempotent IaC pseudocode on all 50 machines, we will encounter an error. Notice that none of the machines have old_file; when the script is run again, it will attempt to delete old_file, which errors out on all machines. Our system administrator amends the IaC script:

Non-idempotent IaC Script (For Hotfix)

```
| For all 50 machines:  
|   create new_file
```

This script should run correctly on any of the 10 machines which have neither file. However, it will error out on any of the 40 machines which already have new_file (since it cannot create a file which already exists). If, by some bad luck, the IaC is run serially one machine at a time, and the 40 successful machines are run first, then the script might error out before reaching any of the 10 machines which need it (or, arguably worse, might reach some but not others if the order is interspersed).

Therefore, tired and annoyed, our system administrator amends the code once more:

Idempotent IaC Script (For Hotfix)

```
| For all 50 machines:
```

```
| | if new_file does not exist:
| | | create new_file
```

This runs successfully on all 50 machines. Our system administrator, having gained insight with regards to idempotence, wants to write a script that would have been idempotent from the start. He writes:

Idempotent IaC Script

```
| For all 50 machines:
| | if old_file exists:
| | | delete old_file
| | if new_file does not exist:
| | | create new_file
```

Now, **given that all 50 machines start off with the same state**, our system administrator can now run this script 4 times on 10 machines, 5 times on 5 more machines, and 100 times on the rest, and it would have been as if he had run it once on all machines.

The next day, our system administrator hears of a different tool, which uses a declarative style of language. In it, he could simply write the end state which he intends old_file and new_file to have:

Declarative IaC Script

```
| End state for all 50 machines:
| | Exist: new_file
| | NotExist: old_file
```

Declarativity is relatively poorly-defined compared to idempotence, but it can be thought of as a style of programming that emphasizes the *what* over the *how*. In this way, a declarative language is also idempotent, because no matter how many times it is run, there is only one same outcome, which is the end state that was specified. These languages might require sophisticated systems to calculate the imperative commands used to reach a specified state.

To our system administrator's chagrin, the new tool is too expensive to purchase. Instead, he tries to approximate declarativity by encapsulating his own imperative code. He defines his own Exist and NotExist:

Define Exist(file):

```
| if file does not exist:
| | create file
```

Define NotExist(file):

```
| if file exists:
| | delete file
```

Now, he can write:

Declarative IaC Script

```
| For all 50 machines:
|   Exist(new_file)
|   NotExist(old_file)
```

When trying to design such reusable modules of code that that can be used in a declarative way despite the internal fixed imperative definition, checking for idempotence is a useful thing to be able to do, since it is a necessary condition for a declarative style.

Furthermore, idempotence without complete declarativity can still provide declarativity-like benefits. Imagine that some machines run Linux while others run Windows, and our system administrator for some reason has to run an ad-hoc script on one of them, to install an OS-sensitive patch. He writes:

Declarative IaC Script for one machine

```
| if machine runs Linux:
|   Exists(linux_file1)
|   NotExists(linux_file2)
|   Exists(linux_file3)
| else if machine runs Windows:
|   Exists(windows_file1)
|   NotExists(windows_file2)
|   NotExists(windows_file3)
```

When run on a single machine of unknown OS, this code produces different end states depending on the OS of that machine. As such, it is arguably not declarative (again, declarativity as we have defined it is a *style* and so might factor in subjective things like how one thinks about a certain syntax in a certain situation). However, if we only consider Linux machines, it is declarative; and if we only consider Windows machines, it is declarative. In a way, idempotence helps design code that has per-initial-state declarativity. If these initial states are thought through well, it can be used to take into account multiple possible situations.

2.4 Goal

Ansible, being a simple, imperative language, does not guarantee idempotence. As such, **our goal is to build a symbolic execution tool for checking a given Ansible script for idempotence.**

3 A Brief Look at Ansible Playbook

(The information regarding Ansible’s behavior is gathered from Ansible documentation [4].)

Revisiting our system administrator, we find that he has adopted Ansible to write his script. He writes a file, `myscript.yml`:

```
--- # three dashes in YAML indicate beginning of a file
- name: My playbook to patch my servers
  hosts: all

  tasks:
  - name: Delete old_file if it exists
    ansible.builtin.file:
      path: /home/remotemachine/old_file.py
      state: absent
  - name: Copy new_file to target if it doesn't exist
    ansible.builtin.copy:
      src: /home/localmachine/new_file.py
      dest: /home/remotemachine/new_file.py
```

He then uses the `ansible-playbook` command (probably along with some options, which we will not go into for reasons of scope) to run this file. Despite the lack of explicit conditional checks, this Ansible file has the same runtime behavior as the “Idempotent IaC Script” pseudocode detailed earlier.

Indeed, it seems as if Ansible already has idempotence built in.

3.1 Ansible’s Module System

Ansible does not in fact have idempotence built in. What the script has done is to call two separate *modules*, `ansible.builtin.file` and `ansible.builtin.copy`, with their individual parameters. These modules are individual programs, written in any language that can communicate with the Ansible API. There is nothing guaranteeing that these modules are idempotent, though in this case they are — most of the core standard library of Ansible is informally regarded to have been written in an idempotent fashion by the developers of Ansible, who use Python, rigorous unit tests and stalwart community feedback to preserve these properties.

It might seem strange that Ansible already uses modules — calls to external, non-Ansible programs — even for fundamental IaC tasks such as creating or copying files. In contrast, Puppet, Nix and Chef all use built-in constructs to express these tasks. Indeed, Ansible chooses a highly modular approach: If Javascript has adopted the ontology of “objects all the way down”, Ansible is “modules and plugins all the way down”. Ansible can often be seen as a simple listing of calls to a series of external programs, with the parameters for each module in Ansible reflecting the idiosyncratic APIs of each program.

These modules are external programs written in any language that can communicate with the Ansible API; they have been written in everything from Javascript to Haskell and can even be pre-compiled binary programs. As such, Ansible relies almost entirely on its community to ensure the quality of each individual module, along with a battle-tested core of modules written by Ansible developers. Among these qualities is the one we are interested in, idempotence.

3.2 Ansible’s DSL

Ansible’s DSL is expressed using YAML syntax. The most basic unit within Ansible is the *task*. Each task encapsulates one or more calls to a single *module*, which as elaborated earlier is an external

program. Any Ansible script is essentially a list of tasks executed one-by-one in the order they are written in, with predicated control flow rather than conditional constructs.

3.2.1 Tasks and Modules

A task is a YAML item that consists of one or more fields.

```
- name: my task           # a field
  mymodule:               # a field with nesting
    myparam1: myvalue1    # a nested field
    myparam2: myvalue2    # a nested field
  retries: 3              # a field
  loop:                   # a field with nesting
    - 1                   # a nested field
    - 2                   # a nested field
  loop_control:           # a field with nesting
    loop_var: loopitem    # a nested field
```

Most of these are attributes of the *task*, and one field is the *module* itself.

There is no human-convenient syntactic way to differentiate which line in a task is the module; one simply has to know what task attributes exist in Ansible, and by elimination identify the module and its parameters (which will be nested under the module).

We have already seen an example of a task containing a single call to the module `ansible.builtin.file`:

```
- name: Delete my_file.py if it exists
  ansible.builtin.file:
    path: /home/targetmachine/my_file.py
    state: absent
```

A task can be written with its fields in any order, and all task fields are optional (with the constraint that nested fields must be under their parent field). The only requirement for a task is that it contains a **single** module; a task cannot contain zero or more than one module. Indeed,

```
- ansible.builtin.file:
  path: /home/targetmachine/my_file.py
  state: absent
  name: Delete my_file.py if it exists
```

and

```
- ansible.builtin.file: # A module with no parameters will
                       # still have a colon at the end
```

are both perfectly syntactically correct tasks. (In the latter example, the module `ansible.builtin.file` might throw an error due to lack of required parameters, but that obviously depends on the module itself.)

3.2.2 Loops

Tasks can have certain control flow features, such as loops. Task loops in Ansible are always finite, and all loop variables are fixed at the start of the loop (meaning a task cannot, for instance, add elements to the loop as it runs and loop forever in that fashion). An example of a task with a loop is:

```
- name: Delete my_file1.py, my_file2.py and my_file3.py.
  ansible.builtin.file:
    path: /home/targetmachine/my_file{item}.py
    state: absent
  loop:
```

- 1
- 2
- 3

3.2.3 Blocks

Blocks are another method of control flow in Ansible. They serve partially to allow users to organize their code into coherent blocks, but more importantly act as a try-catch-finally construct for error handling.

3.2.4 Plays and Playbooks

All Ansible programs are written as a *playbook*; this is like the main function in other languages. A playbook is a list of plays. For our purposes, a play is mostly just a container where more interesting constructs like tasks, handlers and blocks find themselves due to the mandated structure of Ansible.

3.2.5 Handlers

In this thesis, we simplify the real Ansible behavior slightly.

Handlers in Ansible are tasks which run only when they are *notified* to do so. Each handler subscribes to a set of *topics*. Tasks can notify topics, which will tell its handlers to run.

3.2.6 Roles

Roles in Ansible are like a cross between libraries and imported functions. For the purposes of this thesis, a role is nothing more than a collection of tasks and handlers, which is given a name like `myRole`.

A role is used by being imported into a play, whereupon its tasks will run as part of that play, and its handlers will be exposed to tasks within the play.

3.2.7 Variables

Plays and tasks can have the `vars` attribute. This is a key-value map, which allows the variables defined to be used in any more local scope.

3.2.8 Registers and when

A task can have a `register` attribute, which is a special variable that other tasks can check to find out:

- If the task was run
- If the task succeeded
- If the task failed

The way to check this is to use the `when` attribute. A task with this attribute will only attempt to run when the expression given to `when` evaluates to true. For example:

```
- name: first task
  try_delete_file:
    file: /home/remotemachine/old_file.py
    register: reg1

- name: second task
  debug:                                     # this module just prints the contents of msg
    msg: "first task was successful!"
  when: reg1 is defined and reg1 is succeeded
```

3.3 When will we see these?

Later on in Section 5.2, we will see that the approach of this thesis involves:

- First establishing a focus subset of Ansible called $\mathcal{A}^H$, **that our user writes in**

- Then, establishing a minimal subset of Ansible called $\mathcal{A}^{\min}$, **that $\mathcal{A}^H$ can be desugared into**
- Each subset makes use of some features of Ansible; the following table summarizes which.

User-written Ansible ($\mathcal{A}^H$)	Minimal Ansible ($\mathcal{A}^{\min}$)
• Tasks and Modules • Loops • Blocks • Plays • Handlers • Roles • Variables	• Tasks and Modules • Registers and when

It is not surprising that most constructs appear in $\mathcal{A}^H$ but not in $\mathcal{A}^{\min}$, since the latter is a desugaring of the former.

However, noticeably, registers are used within $\mathcal{A}^{\min}$ but not $\mathcal{A}^H$. This is because registers are a powerful and flexible low-level construct that allow us to encode a lot of higher-level constructs, and are useful for $\mathcal{A}^{\min}$. However, if we allowed our user to manipulate registers manually in $\mathcal{A}^H$, it might make desugaring to $\mathcal{A}^{\min}$ relatively complex.

In this thesis, for consistency, we will maintain $\mathcal{A}^H$ to be a superset of $\mathcal{A}^{\min}$. What this means is that registers will technically be included within the syntax of $\mathcal{A}^H$, but a note will be made that the user is not allowed to manipulate it, and it will be assumed to take a default value for $\mathcal{A}^H$.

A similar approach will be taken for some smaller Ansible functionality such as `ignore_errors`, or even constructs that we create ourselves such as *goalkeepers* (all these will be explained in Section 7.5.4). These constructs are useful for the intermediate or final steps of desugaring, but cannot be manually written by the user.

4 Existing Work

4.1 Code Smells

Much of the existing work regarding analyses of IaC tools involve detecting and categorizing code smells, which can range from security risks like hardcoded secrets to maintainability issues like redundant abstraction. For instance, *Puppeteer* and *SecureCode* are tools which use rule-based methods to detect smells. *Puppeteer* focuses on Puppet and *SecureCode* on Ansible [5], [6].

Formalization of data flow or architecture of IaC tool scripts has also been used to help identify smells; *GLITCH*, a rule-based code smell detector like *Puppeteer* and *SecureCode*, supports multiple IaC tools by designing a common intermediate representation [7]. The TOSCA standard is another useful formalization of IaC that allows checking against validity constraints or compliance rules [8], [9]. More recently, Opdebeeck developed a graph-based representation of data flow within Ansible in order to detect code smells [10].

4.2 Property Checking

Several analyses of IaC tools have been written based on semantic formalizations; Häyhä defines a representation of IaC configurations, and models semantics for interactions between resources as well as upgrades of configurations in order to detect security vulnerabilities [11]. Puppet is particularly popular; in addition to μ Puppet, a formalization of Puppet’s core operational semantics [12], Tortoise solves for ways to help system administrators safely repair configurations with the help of their ΔP modeling language as a representation of the semantics of Puppet manifests, shell commands, and possible repairs [13]. Idempotence has also been a useful property to design intelligent fault tolerance during configuration by converging eventually to a final desired state [14].

Of particular relevance to this thesis are two works which have use semantic formalizations to check for idempotence. The first is Rehearsal, a tool that formalizes Puppet semantics as a language called FS, with which it builds an efficient equivalence check, as well as SMT constraints that are only satisfiable if the script is deterministic; together, these two checks allow checking for idempotence of a Puppet script [1]. The second is a paper by Waldemar et al, where an abstract system model was developed for IaC tools in general for the purposes of testing idempotence; their focus is on test optimization to target idempotence using a model testing approach [15].

Indeed, Rehearsal’s approach was the main starting point for this thesis, since its relatively focused approach on formalizing one language’s interaction with the filesystem seemed extensible to the relatively simple language of Ansible. However, symbolic execution was chosen for our method instead of Rehearsal’s SMT constraint building since Ansible is already deterministic. In relation to symbolic execution, guidance for our proof of correctness was taken from a formalization of the method by de Boer et al [16].

5 Approach

5.1 Key Properties of Ansible

Black box

As previously seen, Ansible’s most fundamental building block is a call to an external program which can be written in a multitude of languages. Without inspecting the source code of these modules oneself, they are essentially black boxes whose parameters and outputs are made known via documentation.

Modularity and Community

We have also seen that Ansible’s modules can be written by its users. In fact, Ansible takes a community-centered approach to ensure the richness and quality of its module pool; the developers of Ansible maintain a repository called *Ansible Galaxy* [17], where community members upload and document collections, which often include modules along with their documentation.

5.2 Approach

We have thus far seen that the most fundamental units of the Ansible DSL are black boxes, each of which might originate from a different member of the Ansible community, whose natural language documentation users must rely on. This obviously makes a conventional semantic analysis of Ansible difficult, since the ultimate common reference for the semantics of modules is informal documentation.

With this in mind, our approach in this thesis is:

- Define a core language BBFS to express generic filesystem operations in a way that is compatible with the black-box nature of Ansible
- Develop an idempotence analysis using that core language. We will take a symbolic execution approach, and prove correctness.
- Define a focus subset of Ansible, $\mathcal{A}^H$, that represents the Ansible program, written by a human, that we wish to check for idempotence.
- Define rewrite rules to desugar $\mathcal{A}^H$ into a minimal subset of Ansible, $\mathcal{A}^{\min}$.
- Define a procedure to translate $\mathcal{A}^{\min}$ to BBFS.

6 BBFS

BBFS is a simple calculus whose main data type is a mapping from paths to booleans, representing a filesystem, and whose core operation is a map update similar to `dictionary update()` in Python.

A map update is meant to represent a filesystem operation done by a black-box, with the contract that given a certain starting filesystem state (before the update), the ending state of the update can be achieved; any internal logic or edge cases are handled internally by the black-box. Thus, encapsulation is the default. Any error conditions the black-box does not handle must be explicitly stated using if-else guards.

Section 6.1 to Section 6.4, plus Section 6.6, explain how BBFS relates to a real-life filesystem. Section 6.5 explains the symbolic analysis, and Section 6.7 is a summary of syntax and semantics for reference.

6.1 Filesystems and Paths in BBFS

We can represent a concrete filesystem σ as a total mapping from paths to booleans, where every possible path has a corresponding boolean that describes whether or not it is present. For example, returning to our system administrator from earlier, one possible representation of the initial filesystem of one of his machines might be:

$$\begin{aligned}\sigma = \{ \\ & \text{/home : true,} \\ & \text{/home/remotemachine : true,} \\ & \text{/home/remotemachine/old_file.py : true,} \\ & \text{/home/remotemachine/new_file.py : false,} \\ & \dots \\ & \}\end{aligned}$$

This represents that, among other files, the machine has `old_file.py`, but not `new_file.py`. Note that although we have only shown the relevant paths in the mapping above, in reality, the “...” above contains a mapping for *every possible path*. We will notate the set of all possible paths as $\mathcal{P}$. The domain $\text{dom}(\sigma)$ is therefore $\mathcal{P}$.

When expressing transformations to a concrete filesystem, it is often useful to have *incomplete* mappings of paths to booleans; that is, mappings where the domain is not $\mathcal{P}$. By convention, we will notate these as $\hat{\sigma}$ to distinguish them from the *complete* mapping σ , although they are really the same fundamental type of object. The domains of $\hat{\sigma}$ can be finite or infinite. An example of a finite $\hat{\sigma}$ is:

$$\begin{aligned}\hat{\sigma}_f = \{ \\ & \text{/home/remotemachine/old_file.py : false} \\ & \}\end{aligned}$$

and an example of an infinite $\hat{\sigma}$ is:

$$\hat{\sigma}_i = \{ \begin{array}{l} \text{/home/remotemachine/myfolder : false} \\ \text{/home/remotemachine/myfolder/a : false} \\ \text{/home/remotemachine/myfolder/b : false} \\ \text{/home/remotemachine/myfolder/c : false} \\ \dots \\ \end{array} \}$$

As we can observe from $\hat{\sigma}_i$, once we indicate that `/home/remotemachine/myfolder` is absent, it *must* mean that *every possible path* that is a child of `/home/remotemachine/myfolder` is also absent, which gives us an infinite domain (but still incomplete in the sense that it is not all of $\mathcal{P}$).

One realizes, then, that σ and $\hat{\sigma}$ need to uphold certain properties to maintain sensible representations of actual filesystems. For instance:

$$\sigma_e = \{ \begin{array}{l} \text{/home : false,} \\ \text{/home/remotemachine : true,} \\ \dots \\ \end{array} \}$$

does not make sense, since we are asserting that a subdirectory exists of a nonexistent directory. We may establish this constraint formally. Firstly, we define paths and a parent function on paths, and some predicates:

$$p ::= s \mid s/p$$

$$\text{par}(p) = \begin{cases} p_1/p_2/\dots/p_{n-1} & \text{if } p = p_1/p_2/\dots/p_{n-1}/p_n \\ p_1 & \text{if } p = p_1 \end{cases}$$

Note that formally, paths are represented rather verbosely in BBFS. For instance, the path

`/home/remotemachine/old_file.py`

would really be represented as

`"root"/"home"/"remotemachine"/"old_file.py"`

This formal method of representation was chosen to explicitly indicate that the root directory need not be the conventional `/`. Later on in Section 8.2, we hijack the filesystem to serve as a store for some abstract states, so this allows us to more cleanly separate them by giving the store a completely different root.

For the sake of readability, we will use the informal notation `/home/remotemachine/old_file.py` whenever the meaning is obvious, and return to the formal notation only when it is salient.

Also, we can apply a mapping σ or $\hat{\sigma}$ to any path within its domain in order to get the boolean value assigned to that path. For example, $\hat{\sigma}_f(\text{/home/remotemachine/old_file.py}) = \text{false}$.

With these defined, we may then define what we shall call filesystem consistency, expressed as a predicate `fsc` on a filesystem. In words, a filesystem satisfies `fsc` if:

For all paths within the domain of the filesystem:

- If the path is present, then all its parents must also be in the domain of the filesystem, and be marked as present.

- If the path is absent, then all its possible children must also be in the domain of the filesystem, and be marked as absent.
 - Note: Since only paths which are directories can have children, we only need check this for directories.

We express this formally as:

$$\begin{aligned} \text{fsc}(\hat{\sigma}) \Leftrightarrow \forall p, p_c : p \in \text{dom}(\hat{\sigma}) \wedge p = \text{par}(p_c) (\\ & \hat{\sigma}(p) \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}) \wedge \hat{\sigma}(\text{par}(p)) \\ & \wedge \neg \hat{\sigma}(p) \Rightarrow p_c \in \text{dom}(\hat{\sigma}) \wedge \neg \hat{\sigma}(p_c) \\ &) \end{aligned}$$

6.2 Filesystem Transformations in BBFS

One useful application of $\hat{\sigma}$ parameters is in the *override* function. The *override* function is the core mechanism of *unsafeTransform*, which is the only construct in BBFS that is used to represent real-life filesystem transformations/effects.

override is simply a map/dictionary update as found in many languages; the intuition is the same as dictionary update() in Python, except *override* returns a new mapping and is not in-place. *override* uses one mapping as a parameter to update another. For any path in the domain of the overrider, we assign the corresponding path in the overridee with the overrider's boolean value. For instance, using the values of $\hat{\sigma}_f$ and σ from earlier:

$$\begin{aligned} \text{override}(\hat{\sigma}_f, \sigma) = \{ \\ & \text{/home} : \text{true}, \\ & \text{/home/remotemachine} : \text{true}, \\ & \text{/home/remotemachine/old_file.py} : \text{false}, \\ & \text{/home/remotemachine/new_file.py} : \text{false}, \\ & \dots \\ & \} \end{aligned}$$

The above $\text{override}(\hat{\sigma}, \sigma)$ operation represents the deletion of `home/remotemachine/old_file.py` from the filesystem.

What happens if, like the system administrator previously, we then attempt to delete a file that does not exist? If we override the above result again with the same $\hat{\sigma}_f$, there will be no change (the same intuition as a dictionary update() in Python). By definition, our *override* function cannot produce an error; in BBFS, errors are handled explicitly by if-else guards (we will see this with the *ask* construct later in Section 6.4), and idempotence is the default.

We may also use *override* where both parameters are incomplete mappings; in this case, just like update() for dictionaries in Python, we combine the two mappings' domains, and the boolean values of the overrider is given priority wherever a path exists in the domain of both mappings. For instance, if we have:

$$\begin{aligned}
\hat{\sigma}_Q &= \{ \\
&\quad /home/remotemachine/file1.py : \text{true}, \\
&\quad /home/remotemachine/file2.py : \text{false} \\
&\} \\
\hat{\sigma} &= \{ \\
&\quad /home/remotemachine/file2.py : \text{true}, \\
&\quad /home/remotemachine/file3.py : \text{true} \\
&\}
\end{aligned}$$

Then,

$$\begin{aligned}
\text{override}(\hat{\sigma}_Q, \hat{\sigma}) &= \{ \\
&\quad /home/remotemachine/file1.py : \text{true}, \\
&\quad /home/remotemachine/file2.py : \text{false}, \\
&\quad /home/remotemachine/file3.py : \text{true} \\
&\}
\end{aligned}$$

By convention, we usually represent the *override* parameter with a subscript Q whenever possible.

Having developed some intuitive understanding of the *override* operation in BBFS, we can now formally define it:

$$\text{override}(\hat{\sigma}_Q, \sigma) \stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}$$

As mentioned earlier, *override* is not itself part of the syntax of BBFS, but is the core mechanism of *unsafeTransform*, which is the only construct in BBFS that represents real-life filesystem transformations/effects.

6.3 Executing our First BBFS Expression

Let us take our first look at executing a BBFS expression. We surround the BBFS expression with brackets $\llbracket \cdot \rrbracket_C$ to indicate that we are interpreting the expression's semantics with respect to the concrete filesystem. Later on in Section 6.6, we will develop a symbolic semantics where we instead use $\llbracket \cdot \rrbracket_S$.

When interpreted in the context of manipulating a particular filesystem σ , as we are doing now, all BBFS expressions take in a pair $\langle \hat{\sigma}_E, \sigma \rangle$, and output either another pair $\langle \hat{\sigma}'_E, \sigma' \rangle$ or an error err.

- $\hat{\sigma}_E$ is a mapping for bookkeeping, that tracks the effects that have been applied to the concrete filesystem thus far. This will come in handy for the formal analysis in Section 6.8 and can be ignored for now.
- σ is the concrete filesystem itself.

We thus have:

$$\llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle$$

where:

$$\hat{\sigma}_Q = \{ /home/remotemachine/old_file.py : \text{false} \}$$

Let us say that at this point,

$$\begin{aligned}
\hat{\sigma}_E &= \{\} \\
\sigma &= \{ \\
&\quad /home : true, \\
&\quad /home/remotemachine : true, \\
&\quad /home/remotemachine/old_file.py : true, \\
&\quad /home/remotemachine/new_file.py : false, \\
&\quad \dots \\
&\}
\end{aligned}$$

The concrete semantic for *unsafeTransform* is as such:

$$\llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \sigma) \rangle$$

After executing, we are left with an empty expression $\llbracket \emptyset \rrbracket_C$ and a resulting pair $\langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \sigma) \rangle$. Recalling the definition of *override*, we see that:

$$\begin{aligned}
\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) &= \{ /home/remotemachine/old_file.py : false \} \\
\text{override}(\hat{\sigma}_Q, \sigma) &= \{ \\
&\quad /home : true, \\
&\quad /home/remotemachine : true, \\
&\quad /home/remotemachine/old_file.py : false, \\
&\quad /home/remotemachine/new_file.py : false, \\
&\quad \dots \\
&\}
\end{aligned}$$

Note that $\hat{\sigma}_E$, which started off as an empty set, has undergone the exact same transformation as σ . This is useful as a bookkeeping device when σ might not be known to us.

6.4 Control Flow in BBFS

As observed earlier, *override*, and thus by extension *unsafeTransform*, can never return an error.

Recall that BBFS is meant as a representation of the filesystem behavior of real programs. If our only means of representing filesystem effects has no error mechanism, how might we then represent real programs using it?

To see this, we must first examine the other main construct of BBFS, which is the *ask* guard. This is the only source of control flow in BBFS; *ask* is always used within an if-else expression, and is in fact the only predicate available in the language.

ask allows us to ask questions about the state of the concrete filesystem. In fact, there is only one type of question that can be asked – whether some mapping $\hat{\sigma}_Q$ is a subset of the concrete filesystem. *ask* takes an arbitrary mapping $\hat{\sigma}_Q$ as a parameter, and returns a function. This function takes in a concrete filesystem σ , and returns true if $\hat{\sigma}_Q \subseteq \sigma$, and false otherwise. Formally:

$$\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_1 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \hat{\sigma}_Q \subseteq \sigma \\ \llbracket e_2 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \hat{\sigma}_Q \not\subseteq \sigma \end{cases}$$

To see the utility of *ask*, let us try to model a non-idempotent function. In our example of the system administrator:

Non-idempotent IaC Script

```
1 | For all 50 machines:  
2 | | delete old_file
```

Recall that when he tried to delete `old_file` from a machine where it was already not present, he encountered an error. This suggests that the delete method he was using was not itself idempotent; it could have been anything from `rm` in Bash, to `Files.delete()` in Java, or even a raw syscall like `unlink()` on Linux. The important detail is that it errors out when an attempt is made to delete a file that does not exist.

We can express this behavior as such:

$$\text{if ask}(\hat{\sigma}_Q) \text{ then unsafeTransform}(\hat{\sigma}_T) \text{ else err}$$

where:

$$\hat{\sigma}_Q = \{ /home/remotemachine/old_file.py : \text{true} \}$$
$$\hat{\sigma}_T = \{ /home/remotemachine/old_file.py : \text{false} \}$$

In words, we first check if `/home/remotemachine/old_file.py` is present in our concrete filesystem. If it is, then we delete it through our transform. If it is absent, we error out.

Here we see that although *unsafeTransform* can never return an error, we use the *ask* guard to explicitly state an error condition if it exists. The thinking is that once execution reaches an *unsafeTransform* call, we trust that whatever program that *unsafeTransform* represents will have the requisite error handling internally to ensure that the transform specification is respected (hence the “unsafe” designation).

For example, consider the below Python program:

```
def errors_internally_handled():  
    try:  
        os.makedirs("/home/remotemachine/myfolder")  
    except FileExistsError:  
        print("file already exists, do nothing")  
  
    Path("/home/remotemachine/myfolder/fl.txt").touch(exist_ok=True)
```

This program does its own internal error handling for cases where the file already exists. We can simply express this in BBFS as $\text{unsafeTransform}(\hat{\sigma}_Q)$, where:

$$\hat{\sigma}_Q = \{ \begin{array}{l} /home/remotemachine/myfolder : \text{true} \\ /home/remotemachine/myfolder/fl.txt : \text{true} \end{array} \}$$

6.5 The Symbolic Analysis

Thus far, we have seen how BBFS serves as a means to model generic filesystem operations that read, modify or error out based on a given filesystem state. However, this thesis seeks to conduct an analysis that checks if an Ansible (BBFS) program is always idempotent when run, *regardless* of what the user’s initial filesystem state might be.

In order to do this, we take a symbolic execution approach, where at each point in execution we model all possibilities. We still want to use BBFS as a model for filesystem operations; to this end, we will give it a separate semantics for use in this symbolic analysis.

Therefore, BBFS has two semantics:

- Concrete semantics, which is what we have just seen and which pertains to what a BBFS program “really means”, given a specific concrete filesystem
- Symbolic semantics, which is the semantics we will give BBFS for the purposes of conducting our idempotence analysis. Instead of dealing with filesystems σ , symbolic semantics allow BBFS to operate on the level of *filesystem constraints*, which we notate as κ .

In order to understand the symbolic semantics, we must first examine the form of our idempotence analysis; and in order to understand that, we must first define idempotence for our purposes.

In this thesis, idempotence for Ansible will mean:

- When the program is run more than one time, the end state of the filesystem is identical as if it were only run once.
- We allow programs to error out on the first run — we will only analyze idempotence for filesystem states where the first run succeeds.

To elaborate on the second point, take this program:

```

1 Idempotent Program
2 if file A exists:
3   | error out
4 else:
5   | if file B does not exist:
6     | | create file B

```

Figure 1

If, on the filesystem that runs this program, file A exists, this program will error out immediately. Otherwise, it does an idempotent creation of file B.

This program is idempotent, because we allow programs to error out on the first run. Our criteria for idempotence is that **for all runs which succeed on the first run**, the second, third, etc runs do not further alter filesystem state after they are complete.

With this notion of idempotence in mind, let us examine the analysis we will design. In order to analyse an Ansible program A , we first translate it into a BBFS program B (the translation process will be detailed later in Section 8). Let us say we have the following BBFS program:

```

if ask({/home/fileA: true}) then
  | err
else
  | if ask({/home/fileB: true}) then
    | | unsafeTransform({/home/fileC: true})
  | else
    | | unsafeTransform({/home/fileD: true})

```

Figure 2

If file A exists, this program errors out immediately. Otherwise, the program will either create file C or D, depending on whether file B exists. The creation of file C or D is idempotent.

Let us imagine running this program once. There are 3 possible outcomes:

1. If file A exists, we error out.
2. If file A does not exist, and file B exists, then we create file C.
3. If file A does not exist, and file B does not exist, then we create file D.

For **each** of these outcomes which is not an error, we will run the program again, using as input the filesystem state left behind after the first run.

As mentioned before, Outcome 1 is ignored and progresses no further in our analysis.

For Outcome 2, we run the program again, using as input the filesystem state left behind after the first run. Now, we *do* know some things about the filesystem:

- In order to have arrived at Outcome 2 in the first place, we know that file B must exist in the filesystem.
- We know that file C must exist now, since we created it in the first run.

Therefore, in our second run for Outcome 2, we **must** get Outcome 2 again, because file B exists. The presence of file C does not matter, since its creation is idempotent.

Therefore, Outcome 2 passes the idempotence check; but we are not done yet, since we still have to check Outcome 3. Only if all non-error outcomes pass the check do we consider the program as a whole idempotent.

For Outcome 3, we run the program again, using as input the filesystem state left behind after the first run. Again, now we *do* know some things about the filesystem:

- In order to have arrived at Outcome 3 in the first place, we know that file B must not exist in the filesystem.
- We know that file D must exist now, since we created it in the first run.

Therefore, in our second run for Outcome 3, we **must** get Outcome 3 again, since we know file B exists. This is the only possible result of the second run. Outcome 3 also passes the check.

Since all non-error outcomes pass the check, the BBFS program in Figure 2 is idempotent.

Let us recap the process below.

- The first run:
 - Represents the corresponding Ansible script being ran for the first time
 - Assumes nothing about the initial filesystem state
 - *This is represented by the filesystem constraint `true` in the green zone of Figure 3, which is trivially satisfiable by all filesystems.*
 - Outputs a set of possible outcomes, where each outcome can either be a filesystem constraint, or an error.
 - *This corresponds to κ_1 , κ_2 and `err` in the blue zone of Figure 3.*
- Then, for each possible outcome that is not an error, a second run is done:
 - This represents the user running the same Ansible script the second (or third, fourth etc.) time
 - Now, we know some initial constraints about the filesystem state for each second run
 - *This is represented in Figure 3 by the fact that instead of `true`, our second runs start with κ_1 and κ_2 .*
 - Outputs a set of possible outcomes, where each outcome can either be a filesystem constraint, or an error.

- Finally, in order to check if the program is idempotent, for EACH second run individually, we check if:
 - The output set only contains one possible outcome
 - That outcome is not an error, but a filesystem constraint.
 - That filesystem constraint is the same as the beginning of that second run.
 - Figure 3 shows an example of an idempotent program, because both outcomes (which were not errors in the first run) fulfill these criteria.

Figure 3 below is a diagram representing this process. What we have done is an informal approximation of the symbolic execution step, exploring all 3 options; this is represented by the *symbolic execution tree*. When a real user runs the program, his real filesystem will only explore 1 of those 3 paths; this is represented by the *concrete execution tree*.

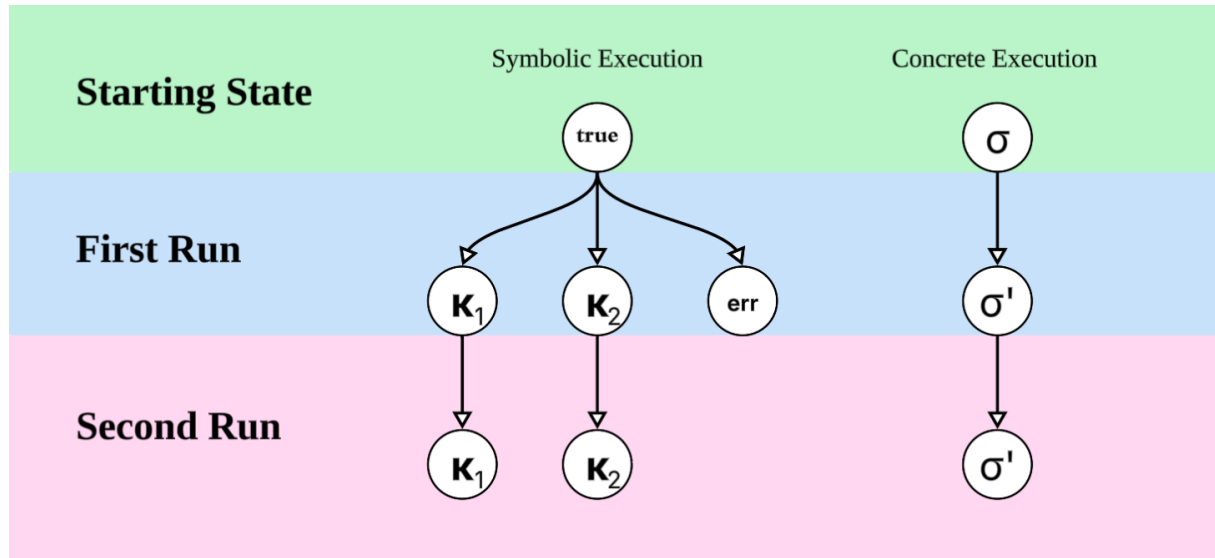


Figure 3

Let us now slowly bring in the formalization.

Instead of a filesystem, symbolic execution of BBFS manipulates a predicate formula, which we notate as κ . In this formula, paths are the variables, which represents a constraint that concrete filesystems can satisfy. An example κ is:

$$(\text{./}) \wedge (\text{./home}) \wedge \neg(\text{./home/myfolder}) \wedge \neg(\text{./home/myfolder/a}) \wedge \neg(\text{./home/myfolder/b}) \dots$$

Recall that filesystems like σ are mappings of paths to booleans. To check if a σ satisfies a κ , we substitute the boolean assigned to each path in σ to the path variable in κ , and evaluate κ normally. Symbolically, we express this as $\sigma(\kappa)$, which we call *concrete-symbolic satisfaction* if true. At any point in the symbolic execution, if the current constraint formula is some κ , it means that all possible σ which satisfy κ can reach that point in the program execution. Note that for all σ , $\sigma(\text{true})$ is trivially true.

In addition to the constraint formula κ , we also keep track of a second variable $\hat{\sigma}_E$ in our analysis. We have already seen a glimpse of this in Section 6.3 for concrete semantics; in fact, both concrete and symbolic semantics make use of this construct.

$\hat{\sigma}_E$ is, as the notation suggests, a mapping from paths to booleans. The function of $\hat{\sigma}_E$ is to keep track of the effects which have been applied to the filesystem at each point in program execution, since we cannot make any assumption or have any knowledge of σ itself at the start. $\hat{\sigma}_E$ starts as an

empty set $\{\}$ at the start of our analysis of some σ , and any effect that is applied to σ is also applied to $\hat{\sigma}_E$.

As such, when a BBFS expression is evaluated using symbolic semantics, it takes in a pair $\langle \hat{\sigma}_E, \kappa \rangle$, and outputs a pair $\langle \hat{\sigma}'_E, \kappa' \rangle$. We call this pair a *symbolic state*. Similarly, when a BBFS expression is evaluated using concrete semantics, it takes in a pair $\langle \hat{\sigma}_E, \sigma \rangle$, and outputs a pair $\langle \hat{\sigma}'_E, \sigma' \rangle$. We call these *concrete states*.

In our idempotence analysis, we always start from an initial symbolic state of $\langle \{\}, \text{true} \rangle$.

- $\hat{\sigma}_E = \{\}$ represents the fact that no effects have been applied to the filesystem at the start of the program.
- $\kappa = \text{true}$, which is satisfiable by any σ , represents the fact that all possible filesystems are allowed at the start of the program; i.e. we do not know anything about the filesystem at this point.

With this understanding, we can add $\hat{\sigma}_E$ to the situation pictured in Figure 3.

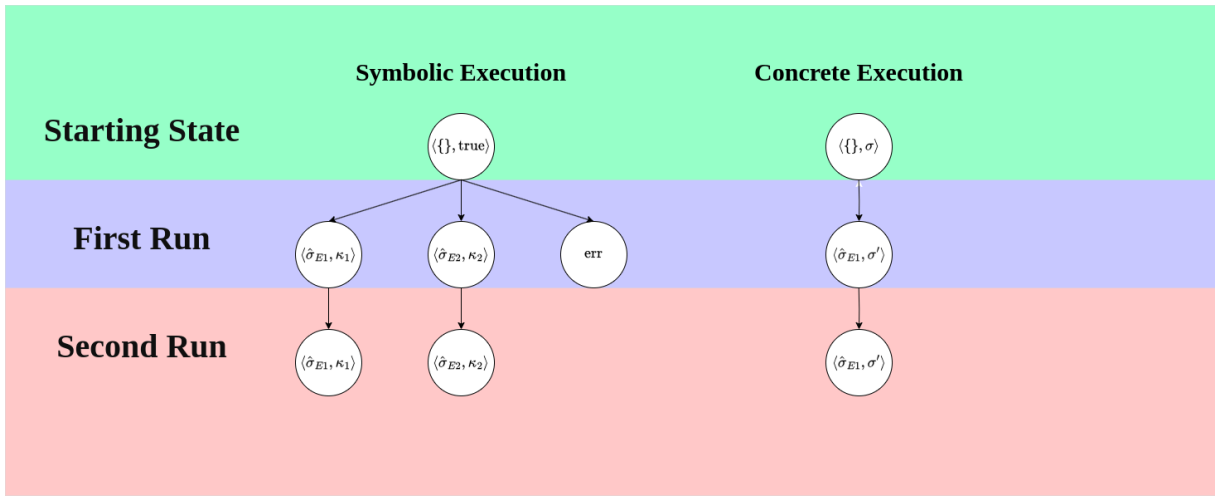


Figure 4

6.6 Symbolic Semantics

Finally, we shall examine the symbolic semantics of BBFS, which are summarized in Section 6.7.3.

Firstly, we see *unsafeTransform* is essentially the same:

$$\llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \kappa) \rangle$$

Because we are dealing with κ , we overload *override*:

$$\text{override}(\hat{\sigma}_Q, \kappa) \stackrel{\text{def}}{=} \bigvee \{ \text{toForm}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_v)) \mid \hat{\sigma}_v \in \text{satTT}(\kappa) \} \quad (1)$$

and define some helper functions *toForm* and *satTT*.

toForm transforms a mapping $\hat{\sigma}$ into a predicate formula:

$$\begin{aligned} \text{toForm} &\in \hat{\sigma} \rightarrow \kappa \\ \text{toForm}(\hat{\sigma}) &= \bigwedge \left\{ \begin{cases} p & \text{if } \hat{\sigma}(p) \\ \neg p & \text{if } \neg \hat{\sigma}(p) \end{cases} \mid p \in \hat{\sigma} \right\} \end{aligned}$$

so if we have

$$\hat{\sigma} = \{ \\
\quad /home/file1 : \text{true} \\
\quad /home/file2 : \text{false} \\
\quad /home/file3 : \text{true} \\
\}$$

then

$$\text{toForm}(\hat{\sigma}) = (/home/file1) \wedge \neg(/home/file2) \wedge (/home/file3)$$

Note that formulae can be infinite.

satTT stands for “satisfying truth table”, and generates all mappings $\hat{\sigma}$ that have the same domain as a predicate formula, and satisfy that formula.

$$\begin{aligned} \text{satTT} &\in \kappa \rightarrow \overline{\hat{\sigma}} \\ \text{satTT}(\kappa) &\stackrel{\text{def}}{=} \{\hat{\sigma} \mid \hat{\sigma}(\kappa) \wedge \text{dom}(\hat{\sigma}) = \text{dom}(\kappa)\} \end{aligned}$$

If we have

$$\kappa = (/home/file1) \wedge (\neg(/home/file2) \vee \neg(/home/file3))$$

then

$$\begin{aligned} \text{satTT}(\kappa) = \{ \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{false}, /home/file3 : \text{false} \} \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{true}, /home/file3 : \text{false} \} \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{false}, /home/file3 : \text{true} \} \\
\} \end{aligned} \tag{2}$$

satTT is so named because it essentially generates the part of κ 's truth table for which κ evaluates to true. Returning to Equation 1, what it is essentially doing is:

- Generate the section of κ 's truth table for which κ evaluates to true at present. Each column of the truth table is a path, and each row is a $\hat{\sigma}_v$.
- For all columns (paths) that are in the domain of $\hat{\sigma}_Q$, override those columns of the truth table with the value from $\hat{\sigma}_Q$.
- Convert back into a formula.

For example, if we have

$$\hat{\sigma}_Q = \{ \\
\quad /home/file2 : \text{true} \\
\}$$

and use the value of $\text{satTT}(\kappa)$ in Equation 2, then:

$$\begin{aligned} \{\text{override}(\hat{\sigma}_Q, \hat{\sigma}_v) \mid \hat{\sigma}_v \in \text{satTT}(\kappa)\} = \{ \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{true}, /home/file3 : \text{false} \} \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{true}, /home/file3 : \text{false} \} \\
\quad \{ /home/file1 : \text{true}, /home/file2 : \text{true}, /home/file3 : \text{true} \} \\
\} \end{aligned} \tag{3}$$

where the “column” for `/home/file2` is overridden to be `true`.

Each row of the truth table is then converted back into a formula, and or-ed together.

$$\begin{aligned} & ((/home/file1) \wedge (/home/file2) \wedge \neg(/home/file3)) \\ \vee & ((/home/file1) \wedge (/home/file2) \wedge \neg(/home/file3)) \\ \vee & ((/home/file1) \wedge (/home/file2) \wedge (/home/file3)) \end{aligned}$$

This means that the new formula’s truth table is what we saw in Equation 3.

What about *ask*?

The first thing we realize is that *ask* is nondeterministic in symbolic semantics. This just means that both paths can be explored; whether each path does get explored depends on whether its precondition (e.g. if $\bar{\sigma}_T \neq \emptyset$) is fulfilled.

Expression for taking the if-branch:

$$\begin{aligned} \llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle &\stackrel{\text{def}}{=} \llbracket e_1 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_T \neq \emptyset \\ \text{where:} & \\ \bar{\sigma}_T &= \{ \hat{\sigma}_T \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \} \\ \kappa'_T &= \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_T) \end{aligned} \quad (4)$$

Expression for taking the else-branch:

$$\begin{aligned} \llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle &\stackrel{\text{def}}{=} \llbracket e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_F \neq \emptyset \\ \text{where:} & \\ \bar{\sigma}_F &= \{ \hat{\sigma}_F \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \} \\ \kappa'_F &= \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_F) \end{aligned} \quad (5)$$

What both equations do together is generate a truth table ($\bar{\sigma}_T$ and $\bar{\sigma}_F$ are the two halves of this truth table), where each row is a subset of σ that satisfies κ . Since κ is the filesystem constraint up until the if-else, this represents all possible subsets of σ that can possibly have reached this if-else juncture. Then, the two equations partition this truth table into:

- $\bar{\sigma}_T$: those values of σ which *also* satisfy $\text{toForm}(\hat{\sigma}_Q)$ (Equation 4)
- $\bar{\sigma}_F$: those values of σ which *also* do not satisfy $\text{toForm}(\hat{\sigma}_Q)$ (Equation 5)

Each equation then adds its respective half of the truth table to the current constraint κ , and then continues execution along its branch. If one of the halves is empty (i.e. no possible subset of σ satisfies it), that branch is not taken.

6.7 BBFS Summary

We will now present the syntax and semantics of BBFS, and thereafter explain the idempotence analysis in more detail.

6.7.1 Syntax

$$\begin{aligned} \text{Strings } s &\text{ ::= } \dots \\ \text{Paths } p &\text{ ::= } s \mid s/p \\ \text{Booleans } b &\text{ ::= } \text{true} \mid \text{false} \\ \text{Logical File System } \hat{\sigma} &\text{ ::= } \{p_1 \rightarrow b_1, p_2 \rightarrow b_2, \dots\} \\ \text{Expressions } e &\text{ ::= } \text{err} \mid \text{if ask}(\hat{\sigma}) \text{ then } e_1 \text{ else } e_2 \mid e_1; e_2 \mid \text{unsafeTransform}(\hat{\sigma}) \end{aligned}$$

6.7.2 Concrete Semantics

$$\begin{aligned}
\llbracket e \rrbracket_C &\in \langle \hat{\sigma}_E, \sigma \rangle \rightarrow \langle \hat{\sigma}_E, \sigma \rangle + \text{err} \\
\llbracket \text{err} \rrbracket_C &\langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \text{err} \\
\llbracket e_1; e_2 \rrbracket_C &\langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_2 \rrbracket_C \langle \hat{\sigma}'_E, \sigma' \rangle & \text{if } \llbracket e_1 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle = \langle \hat{\sigma}'_E, \sigma' \rangle \\ \text{err} & \text{if } \llbracket e_1 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle = \text{err} \end{cases} \\
\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_C &\langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_1 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \sigma(\text{toForm}(\hat{\sigma})) \\ \llbracket e_2 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \neg \sigma(\text{toForm}(\hat{\sigma})) \end{cases} \\
\llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_C &\langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \sigma) \rangle
\end{aligned}$$

6.7.3 Symbolic Semantics

$$\begin{aligned}
\llbracket e \rrbracket_S &\in \langle \hat{\sigma}_E, \kappa \rangle \rightarrow \langle \hat{\sigma}_E, \kappa \rangle + \text{err} \\
\llbracket \text{err} \rrbracket_S &\langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \text{err} \\
\llbracket e_1; e_2 \rrbracket_S &\langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_2 \rrbracket_S \langle \hat{\sigma}'_E, \kappa' \rangle & \text{if } \llbracket e_1 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle = \langle \hat{\sigma}'_E, \kappa' \rangle \\ \text{err} & \text{if } \llbracket e_1 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle = \text{err} \end{cases} \\
\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S &\langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \llbracket e_1 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_T \neq \emptyset \\
&\text{where:} \\
&\bar{\sigma}_T = \{ \hat{\sigma}_T \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \} \\
&\kappa'_T = \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_T) \\
\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S &\langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \llbracket e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_F \neq \emptyset \\
&\text{where:} \\
&\bar{\sigma}_F = \{ \hat{\sigma}_F \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \} \\
&\kappa'_F = \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_F) \\
\llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_S &\langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \kappa) \rangle
\end{aligned}$$

6.7.4 Utility Functions

$$\begin{aligned}
\text{dom} &\in \hat{\sigma} \rightarrow \{p\} \\
+\sigma &\rightarrow \{p\} \\
\text{dom}(\hat{\sigma}) &= \{p \mid \langle p \rightarrow b \rangle \in \hat{\sigma}\} \\
\text{dom}(\sigma) &= \mathcal{P} \\
&\text{where } \mathcal{P} \text{ is the set of all paths}
\end{aligned}$$

Note that since σ denotes an infinite set, its domain is trivially the set of all paths.

$$\begin{aligned}
\text{override} &\in \hat{\sigma} \rightarrow \kappa \rightarrow \kappa \\
&+\hat{\sigma} \rightarrow \sigma \rightarrow \sigma \\
&+\hat{\sigma} \rightarrow \hat{\sigma} \rightarrow \hat{\sigma} \\
\text{override}(\hat{\sigma}_Q, \kappa) &\stackrel{\text{def}}{=} \bigvee \{ \text{toForm}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_v)) \mid \hat{\sigma}_v \in \text{satTT}(\kappa) \} \\
\text{override}(\hat{\sigma}_Q, \sigma) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\} \\
\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}
\end{aligned}$$

$$\begin{aligned}
\text{satTT} &\in \kappa \rightarrow \bar{\hat{\sigma}} \\
\text{satTT}(\kappa) &\stackrel{\text{def}}{=} \{\hat{\sigma} \mid \hat{\sigma}(\kappa) \wedge \text{dom}(\hat{\sigma}) = \text{dom}(\kappa)\} \\
\text{toForm} &\in \hat{\sigma} \rightarrow \kappa \\
\text{toForm}(\hat{\sigma}) &= \bigwedge \left\{ \begin{cases} p & \text{if } \hat{\sigma}(p) \\ \neg p & \text{if } \neg \hat{\sigma}(p) \end{cases} \mid p \in \hat{\sigma} \right\}
\end{aligned}$$

6.8 Proof of Correctness

To prove correctness, we must prove both that our idempotence check does indeed imply idempotence if satisfied, and that our symbolic system is sound with respect to the concrete system.

6.8.1 Idempotence Proof

We will now more formally define what we mean by idempotence in the context of Ansible. Let us say that we have some Ansible program A , which we have translated into a BBFS program B (we will deal with how this translation works later in Section 8).

For B to be idempotent, there must be no difference between running B once on the user's machine, versus running B more than once in sequence.

To remove the “more than once” semantic, we can simplify this using a fixed point: Running B once on the user's filesystem σ must produce the same result as running B twice in succession (or in other words, $B; B$).

We can express this using the semantic notation we have used thus far. We define the property *idem* on a BBFS program B :

$$\begin{aligned}
\text{idem}(B) \Leftrightarrow \forall \sigma : \llbracket B \rrbracket_C \langle \{\}, \sigma \rangle \neq \text{err} \ (\\
&\llbracket B \rrbracket_C \langle \{\}, \sigma \rangle = \llbracket B \rrbracket_C \circ \llbracket B \rrbracket_C \langle \{\}, \sigma \rangle \\
&)
\end{aligned}$$

Using transition notation, we express the same idea as:

$$\begin{aligned}
\text{idem}(B) \Leftrightarrow \forall \sigma, \hat{\sigma}_E, \sigma' : \langle B, \{\}, \sigma \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \sigma' \rangle (\\
&\langle B; B, \{\}, \sigma \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \sigma' \rangle \\
&)
\end{aligned}$$

In order to check for this property, we check for a similar property in the symbolic transition system.

$$\begin{aligned}
\text{passSymCheck}(B) \Leftrightarrow \forall \hat{\sigma}_E, \kappa, \hat{\sigma}'_E, \kappa' : \langle B; B, \{\}, \text{true} \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \kappa \rangle (\\
&\langle B, \hat{\sigma}_E, \kappa \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}'_E, \kappa' \rangle \\
&)
\end{aligned}$$

We would like to show that:

$$\text{passSymCheck}(B) \Rightarrow \text{idem}(B)$$

Proof:

We assume the premise; that is, we assume that for some B , $\text{passSymCheck}(B)$ is true:

$$\forall \hat{\sigma}_E, \kappa, \hat{\sigma}'_E, \kappa' : \langle B; B, \{\}, \text{true} \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \kappa \rangle \left(\begin{array}{l} \langle B, \hat{\sigma}_E, \kappa \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \kappa' \rangle \\ \end{array} \right) \quad (6)$$

From Lemma 6.8.2.1, we know that for any transition of the symbolic system, there must be a corresponding concrete transition.

Therefore, given that Equation 6 asserts that there exists a symbolic transition

$$\langle B; B, \{\}, \text{true} \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \kappa \rangle \quad (7)$$

There will exist some concrete transition:

$$\langle B; B, \{\}, \sigma \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \sigma' \rangle \quad (8)$$

where σ' satisfies κ .

We have thus far examined the first symbolic run from $\langle B; B, \{\}, \text{true} \rangle$ to $\langle B, \hat{\sigma}_E, \kappa \rangle$, and shown that a corresponding concrete run exists. Now, we want to prove that:

- Given that the second symbolic run exists, from $\langle B, \hat{\sigma}_E, \kappa \rangle$ to $\langle \emptyset, \hat{\sigma}_E, \kappa' \rangle$
- There will exist a corresponding second concrete run from $\langle B, \hat{\sigma}_E, \sigma' \rangle$ to $\langle \emptyset, \hat{\sigma}_E, \sigma'' \rangle$, such that σ'' also satisfies κ' . In other words, *concrete-symbolic satisfaction* is preserved in the second run.

This is basically the exact same proof as for soundness (Lemma 6.8.2.1), but:

- Instead of beginning with symbolic state $\langle B; B, \{\}, \text{true} \rangle$ and concrete state $\langle B; B, \{\}, \sigma \rangle$
- We now begin with symbolic state $\langle B, \hat{\sigma}_E, \kappa \rangle$ and concrete state $\langle B, \hat{\sigma}_E, \sigma' \rangle$.

As such, since we know that σ' already satisfies κ at the start of this section, we establish our base case with a nop. Letting $B = \text{unsafeTransform}(\{\}); B'$ and applying Lemma 6.8.2.3,

$$\langle B, \hat{\sigma}_E, \kappa \rangle \rightarrow \langle B', \hat{\sigma}_E, \kappa \rangle, \quad \sigma(\kappa)$$

Reusing Lemma 6.8.2.3 as our inductive case, we prove that $\langle B, \hat{\sigma}_E, \sigma' \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \sigma'' \rangle$ exists, and that $\sigma''(\kappa')$.

What exactly is σ'' ? From Lemma 6.8.2.4, we know some additional information regarding the symbolic transition in Equation 7. Most importantly, we know that σ' in Equation 8 is actually $\text{override}(\hat{\sigma}_E, \sigma)$. As such, we can rewrite Equation 8 to

$$\langle B; B, \{\}, \sigma \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle$$

and we know that $\text{override}(\hat{\sigma}_E, \sigma)$ satisfies κ .

Lemma 6.8.2.8 also tells us that $\hat{\sigma}_E \subseteq \text{override}(\hat{\sigma}_E, \sigma)$. This means we satisfy the precondition for Lemma 6.8.2.5, which in turn tells us that $\sigma'' = \text{override}(\hat{\sigma}_E, \sigma)$.

Recall that σ' is also $\text{override}(\hat{\sigma}_E, \sigma)$. This means that overall:

- At the beginning:
 - *Concrete-symbolic satisfaction*: $\sigma(\text{true})$
- First run:
 - Given that there exists symbolic transition: $\langle B; B, \{\}, \text{true} \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \kappa \rangle$
 - This implies there exists concrete transition: $\langle B; B, \{\}, \sigma \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle$
 - Such that *concrete-symbolic satisfaction* is preserved: $\text{override}(\hat{\sigma}_E, \sigma)(\kappa)$
- Second run:

- Given that there exists symbolic transition: $\langle B, \hat{\sigma}_E, \kappa \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \kappa' \rangle$
- This implies there exists concrete transition: $\langle B, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle$
- Such that *concrete-symbolic satisfaction* is preserved: $\text{override}(\hat{\sigma}_E, \sigma)(\kappa')$

The important bit is that in the second run, the concrete filesystem does not change.

Thus far, we have shown that given $\text{passSymCheck}(B)$ is true, we can run a program $B;B$ on the concrete filesystem, which will go through the transitions:

$$\langle B;B, \{\}, \sigma \rangle \xrightarrow{*} \langle B, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle \quad (9)$$

Finally, we realize that in BBFS, running a program $B;B$ to B is the exact same as running a program B to completion. Symbolically, $\forall \hat{\sigma}_E, \hat{\sigma}'_E, \sigma, \sigma'$:

$$\begin{aligned} \langle B;B, \hat{\sigma}_E, \sigma \rangle &\xrightarrow{*} \langle B, \hat{\sigma}'_E, \sigma' \rangle \\ \Leftrightarrow \langle B, \hat{\sigma}_E, \sigma \rangle &\xrightarrow{*} \langle \emptyset, \hat{\sigma}'_E, \sigma' \rangle \end{aligned}$$

because there is no construct within the semantics of BBFS that transcends sequencing boundaries.

Therefore, we can deduce from Equation 9 that:

$$\langle B, \{\}, \sigma \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle \quad (10)$$

and

$$\langle B;B, \{\}, \sigma \rangle \xrightarrow{*} \langle \emptyset, \hat{\sigma}_E, \text{override}(\hat{\sigma}_E, \sigma) \rangle$$

Thus we prove that $\text{passSymCheck}(B) \Rightarrow \text{idem}(B)$.

6.8.2 Soundness Proof

For our system to be *correct*, it must not only facilitate idempotence checking, but must also be *sound*. We have seen the two semantics that can be applied to two transition systems: A logical one and a concrete one. If our system is *sound*, it means that every reachable logical state corresponds to a reachable concrete state.

Formally, we want to show:

Lemma 6.8.2.1 (Soundness):

- If there exists a multi-step symbolic transition from the standard starting state $\langle B, \{\}, \text{true} \rangle$ to some ending symbolic state $\langle B', \hat{\sigma}_E, \kappa \rangle$, then
- it implies that there must exist a corresponding multi-step concrete transition from the standard starting state $\langle B, \{\}, \sigma \rangle$ to some ending concrete state $\langle B', \hat{\sigma}_E, \sigma' \rangle$
- such that σ' satisfies κ .

Formally:

$$\forall B, B', \hat{\sigma}_E, \kappa, \sigma \left(\begin{aligned} &\langle B, \{\}, \text{true} \rangle \xrightarrow{*} \langle B', \hat{\sigma}_E, \kappa \rangle \\ \Rightarrow &\exists \sigma' : \sigma'(\kappa) \wedge \langle B, \{\}, \sigma \rangle \xrightarrow{*} \langle B', \hat{\sigma}_E, \sigma' \rangle \end{aligned} \right)$$

We will prove this via induction; Lemma 6.8.2.2 below allows us to establish our base case with a nop applied to Lemma 6.8.2.3:

$$\langle \text{unsafeTransform}(\{\}); B', \{\}, \text{true} \rangle \rightarrow \langle B', \{\}, \text{true} \rangle, \quad \sigma(\text{true})$$

And then Lemma 6.8.2.3 is our inductive step.

Lemma 6.8.2.2 (Initial conditions):

For any concrete filesystem σ , it satisfies the trivial filesystem constraint $\kappa = \text{true}$.

Formally:

$$\forall \sigma (\sigma(\text{true}))$$

Lemma 6.8.2.3 (One-step):

- Given that:
 - there exists a single-step symbolic transition from some symbolic state $\langle B, \hat{\sigma}_E, \kappa \rangle$ to some ending symbolic state $\langle B', \hat{\sigma}_E, \kappa \rangle$,
 - and σ satisfies κ ,
- this implies that:
 - there must exist a corresponding single-step concrete transition from some concrete state starting state $\langle B, \hat{\sigma}_E, \sigma \rangle$ to some concrete state $\langle B', \hat{\sigma}'_E, \sigma' \rangle$
 - such that σ' satisfies κ' .

Formally:

$$\begin{aligned} \forall B, B', \hat{\sigma}_E, \kappa, \sigma (\\ & \sigma(\kappa) \wedge \langle B, \hat{\sigma}_E, \kappa \rangle \rightarrow \langle B', \hat{\sigma}_E, \kappa \rangle \\ \Rightarrow \exists \sigma' : \sigma'(\kappa') \wedge \langle B, \hat{\sigma}_E, \sigma \rangle \rightarrow \langle B', \hat{\sigma}'_E, \sigma' \rangle \\) \end{aligned}$$

Proof:

We have two cases:

- `unsafeTransform`($\hat{\sigma}_Q$)
- if `ask`($\hat{\sigma}_Q$) then e_1 else e_2

For `unsafeTransform`($\hat{\sigma}_Q$):

$$\begin{aligned} \llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle &\stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \kappa) \rangle \\ \llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle &\stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \sigma) \rangle \end{aligned}$$

We want to show that $\text{override}(\hat{\sigma}_Q, \sigma)(\text{override}(\hat{\sigma}_Q, \kappa))$. Recall that:

$$\begin{aligned} \text{override}(\hat{\sigma}_Q, \kappa) &\stackrel{\text{def}}{=} \bigvee \{ \text{toForm}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_v)) \mid \hat{\sigma}_v \in \text{satTT}(\kappa) \} \\ \because \sigma(\kappa) &\because \exists \hat{\sigma}_s : \hat{\sigma}_s = \hat{\sigma}_v \wedge \hat{\sigma}_s \subseteq \sigma \end{aligned}$$

In other words, we know that one of the or'ed elements in $\text{override}(\hat{\sigma}_Q, \kappa)$ is $\text{toForm}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_s))$.

We rewrite:

$$\begin{aligned} \text{override}(\hat{\sigma}_Q, \kappa) &= \text{toForm}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_s)) \vee \bar{\kappa} \\ &\text{where } \bar{\kappa} \text{ denotes any other elements.} \end{aligned}$$

Since $\hat{\sigma}_s \subseteq \sigma$, we apply Lemma 6.8.2.15

$$\text{override}(\hat{\sigma}_Q, \hat{\sigma}_s) \subseteq \text{override}(\hat{\sigma}_Q, \sigma)$$

And applying Lemma 6.8.2.14:

$$\begin{aligned} &\text{override}(\hat{\sigma}_Q, \sigma)(\text{toForm}(\hat{\sigma}_Q, \hat{\sigma}_s)) \\ \Rightarrow &\text{override}(\hat{\sigma}_Q, \sigma)(\text{override}(\hat{\sigma}_Q, \kappa)) \end{aligned}$$

For if ask($\hat{\sigma}_Q$) then e_1 else e_2 :

We recall that

$$\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_1 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \sigma(\text{toForm}(\hat{\sigma})) \\ \llbracket e_2 \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle & \text{if } \neg\sigma(\text{toForm}(\hat{\sigma})) \end{cases}$$

$$\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \llbracket e_1 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_T \neq \emptyset$$

where:

$$\bar{\sigma}_T = \{ \hat{\sigma}_T \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \}$$

$$\kappa'_T = \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_T)$$

$$\llbracket \text{if ask}(\hat{\sigma}_Q) \text{ then } e_1 \text{ else } e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa \rangle \stackrel{\text{def}}{=} \llbracket e_2 \rrbracket_S \langle \hat{\sigma}_E, \kappa' \rangle \text{ if } \bar{\sigma}_F \neq \emptyset$$

where:

$$\bar{\sigma}_F = \{ \hat{\sigma}_F \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \}$$

$$\kappa'_F = \kappa \wedge \bigvee \text{map}(\text{toForm}, \bar{\sigma}_F)$$

We wish to show that $\sigma(\kappa'_T) \vee \sigma(\kappa'_F)$.

$$\begin{aligned} \kappa'_T &= \kappa \wedge \bigvee \text{map}(\text{toForm}, \{ \hat{\sigma}_T \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \}) \\ &= \kappa \wedge \bigvee \{ \text{toForm}(\hat{\sigma}_T) \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \} \\ \kappa'_F &= \kappa \wedge \bigvee \text{map}(\text{toForm}, \{ \hat{\sigma}_F \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \}) \\ &= \kappa \wedge \bigvee \{ \text{toForm}(\hat{\sigma}_F) \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \} \end{aligned}$$

Since we know $\sigma(\kappa)$, this means we only need to check:

$$\begin{aligned} &\sigma\left(\bigvee \{ \text{toForm}(\hat{\sigma}_T) \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \}\right) \\ &\vee \sigma\left(\bigvee \{ \text{toForm}(\hat{\sigma}_F) \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \}\right) \end{aligned}$$

Recall:

$$\text{satTT}(\kappa) \stackrel{\text{def}}{=} \{ \hat{\sigma} \mid \hat{\sigma}(\kappa) \wedge \text{dom}(\hat{\sigma}) = \text{dom}(\kappa) \}$$

Since we know $\sigma(\kappa)$, we know $\exists \hat{\sigma}_s \subseteq \sigma : \hat{\sigma}_s \in \text{satTT}(\kappa)$.

We know that:

- $\hat{\sigma}_s \in \text{satTT}(\kappa)$
- $\hat{\sigma}_s(\text{toForm}(\hat{\sigma}_Q)) \vee \neg \hat{\sigma}_s(\text{toForm}(\hat{\sigma}_Q))$ is a tautology

We can conclude that:

$$\begin{aligned} \hat{\sigma}_s &\in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \\ \vee \hat{\sigma}_s &\in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \end{aligned}$$

Therefore, one of the $\hat{\sigma}_T$ or the $\hat{\sigma}_F$ will be our $\hat{\sigma}_s$. Either way, we know that:

$$\begin{aligned} &\hat{\sigma}_s\left(\bigvee \{ \text{toForm}(\hat{\sigma}_T) \mid \dots \}\right) \\ &\vee \hat{\sigma}_s\left(\bigvee \{ \text{toForm}(\hat{\sigma}_F) \mid \dots \}\right) \end{aligned}$$

Applying Lemma 6.8.2.14 shows that we have achieved our goal:

$$\begin{aligned} & \sigma \left(\bigvee \left\{ \text{toForm}(\hat{\sigma}_T) \mid \hat{\sigma}_T \in \text{satTT}(\kappa \wedge \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_T) \right\} \right) \\ & \vee \sigma \left(\bigvee \left\{ \text{toForm}(\hat{\sigma}_F) \mid \hat{\sigma}_F \in \text{satTT}(\kappa \wedge \neg \text{toForm}(\hat{\sigma}_Q)) \wedge \text{fsc}(\hat{\sigma}_F) \right\} \right) \end{aligned}$$

Lemma 6.8.2.4 (First-Run Override Relationship):

Recall that the first step of our idempotence analysis is to run our BBFS program with symbolic semantics once, starting from the standard symbolic starting point $\langle B, \{\}, \text{true} \rangle$ and ending at $\langle \emptyset, \hat{\sigma}_E, \kappa \rangle$, which corresponds to a standard concrete transition from $\langle B, \{\}, \sigma \rangle$ to $\langle \emptyset, \hat{\sigma}_E, \sigma' \rangle$.

This lemma states that if we have such a multi-step concrete transition, from the standard concrete starting point $\langle B, \{\}, \sigma \rangle$ to some $\langle B', \hat{\sigma}_E, \sigma' \rangle$, then we can actually calculate $\sigma' = \text{override}(\hat{\sigma}_E, \sigma)$.

Formally:

$$\begin{aligned} & \forall B, \sigma, \hat{\sigma}_E : \\ & \langle B, \{\}, \sigma \rangle \xrightarrow{*} \langle B', \hat{\sigma}_E, \sigma' \rangle (\\ & \qquad \qquad \qquad \sigma' = \text{override}(\hat{\sigma}_E, \sigma) \\ & \qquad \qquad \qquad) \end{aligned}$$

Proof:

Corollary of Lemma 6.8.2.6. .

Lemma 6.8.2.5 (Second-Run Override Relationship):

Recall that the second phase of our idempotence analysis is to re-run our BBFS program with symbolic semantics. The symbolic starting point is where the first step left off, which is some $\langle B, \hat{\sigma}_E, \kappa \rangle$; and in order for $\text{passSymbolicCheck}(B)$ to be true, it must end at $\langle \emptyset, \hat{\sigma}_E, \kappa' \rangle$. This corresponds to a concrete transition from $\langle B, \hat{\sigma}_E, \sigma' \rangle$ to $\langle \emptyset, \hat{\sigma}_E, \sigma' \rangle$.

This lemma states that for such a concrete transition, we can actually calculate that $\sigma = \sigma'$; i.e. the filesystem does not change when running program B from that point.

$$\begin{aligned} & \forall B, \sigma, \hat{\sigma}_E : \\ & \hat{\sigma}_E \subseteq \sigma \wedge \langle B, \hat{\sigma}_E, \sigma \rangle \xrightarrow{*} \langle B', \hat{\sigma}_E, \sigma' \rangle (\\ & \qquad \qquad \qquad \sigma' = \sigma \\ & \qquad \qquad \qquad) \end{aligned}$$

Proof:

Corollary of Lemma 6.8.2.6.

$$\sigma' = \text{override}(\hat{\sigma}_E, \sigma)$$

Applying Lemma 6.8.2.13:

$$\text{override}(\hat{\sigma}_E, \sigma) = \sigma$$

Lemma 6.8.2.6 (Generic Many-Step Override Relationship):

This is a generalization of Lemma 6.8.2.4 and Lemma 6.8.2.5.

This lemma states that if we have some multi-step concrete transition from $\langle B, \hat{\sigma}_E, \sigma \rangle$ to $\langle B', \hat{\sigma}'_E, \sigma' \rangle$, and if $\hat{\sigma}_E \subseteq \sigma$ at the start, then we can actually calculate $\sigma' = \text{override}(\hat{\sigma}'_E, \sigma)$.

$$\begin{aligned} \forall B, \sigma, \hat{\sigma}_E : \\ \hat{\sigma}_E \subseteq \sigma \wedge \langle B, \hat{\sigma}_E, \sigma \rangle \xrightarrow{*} \langle B', \hat{\sigma}'_E, \sigma' \rangle (\\ \sigma' = \text{override}(\hat{\sigma}'_E, \sigma) \\) \end{aligned}$$

Proof:

This is an inductive proof. Our base case is given by Lemma 6.8.2.7 where

$$B = \text{unsafeTransform}(\{\}); B'$$

Note that $\text{unsafeTransform}(\{\})$ is essentially a nop. As such:

$$\langle B, \hat{\sigma}_E, \sigma \rangle \rightarrow \langle B', \hat{\sigma}_E, \sigma \rangle$$

where $\sigma = \text{override}(\hat{\sigma}_E, \sigma)$ since $\hat{\sigma}_E \subseteq \sigma$ according to Lemma 6.8.2.13.

Now, let us examine two steps:

$$\langle B, \hat{\sigma}_E, \sigma \rangle \rightarrow \langle B', \hat{\sigma}'_E, \sigma' \rangle \rightarrow \langle B'', \hat{\sigma}''_E, \sigma'' \rangle$$

From Lemma 6.8.2.7:

$$\sigma' = \text{override}(\hat{\sigma}'_E, \sigma)$$

$$\sigma'' = \text{override}(\hat{\sigma}''_E, \sigma')$$

$$\text{dom}(\hat{\sigma}_E) \subseteq \text{dom}(\hat{\sigma}'_E) \subseteq \text{dom}(\hat{\sigma}''_E)$$

We would like to show that $\sigma'' = \text{override}(\hat{\sigma}''_E, \sigma)$.

$$\sigma'' = \text{override}(\hat{\sigma}''_E, \text{override}(\hat{\sigma}'_E, \sigma))$$

Applying Lemma 6.8.2.11:

$$\begin{aligned} & \text{override}(\hat{\sigma}''_E, \text{override}(\hat{\sigma}'_E, \sigma)) \\ &= \text{override}(\text{override}(\hat{\sigma}''_E, \text{override}(\hat{\sigma}'_E, \hat{\sigma}_E)), \sigma) \end{aligned}$$

Applying Lemma 6.8.2.9 twice:

$$\begin{aligned} & \text{override}(\text{override}(\hat{\sigma}''_E, \text{override}(\hat{\sigma}'_E, \hat{\sigma}_E)), \sigma) \\ &= \text{override}(\text{override}(\hat{\sigma}''_E, \hat{\sigma}_E), \sigma) \\ &= \text{override}(\hat{\sigma}''_E, \sigma) \end{aligned}$$

Lemma 6.8.2.7 (Generic Single-Step Override Relationship):

This is a single-step version of Lemma 6.8.2.6, and is used in it as an inductive step.

This lemma states that if we have some single-step concrete transition from $\langle B, \hat{\sigma}_E, \sigma \rangle$ to $\langle B', \hat{\sigma}'_E, \sigma' \rangle$, and if $\hat{\sigma}_E \subseteq \sigma$ at the start, then we know several things which are useful for proving Lemma 6.8.2.6:

- We can calculate $\sigma' = \text{override}(\hat{\sigma}'_E, \sigma)$.
- We know that $\text{dom}(\hat{\sigma}_E) \subseteq \text{dom}(\hat{\sigma}'_E)$
- We know that $\hat{\sigma}'_E \subseteq \sigma'$

Formally:

$$\begin{aligned} \forall B, \sigma, \hat{\sigma}_E : \\ \hat{\sigma}_E \subseteq \sigma \wedge \langle B, \hat{\sigma}_E, \sigma \rangle \rightarrow \langle B', \hat{\sigma}'_E, \sigma' \rangle (\\ \quad \sigma' = \text{override}(\hat{\sigma}'_E, \sigma) \\ \quad \wedge \text{dom}(\hat{\sigma}_E) \subseteq \text{dom}(\hat{\sigma}'_E) \\ \quad \wedge \hat{\sigma}'_E \subseteq \sigma' \\) \end{aligned}$$

Proof: The only construct in our language that mutates $\hat{\sigma}_E$ and σ is $\text{unsafeTransform}(\hat{\sigma}_Q)$ for some parameter $\hat{\sigma}_Q$. For this case, for any $\hat{\sigma}_Q$, we wish to show that $\sigma' = \text{override}(\hat{\sigma}'_E, \sigma)$.

$$\begin{aligned} \llbracket \text{unsafeTransform}(\hat{\sigma}_Q) \rrbracket_C \langle \hat{\sigma}_E, \sigma \rangle &\stackrel{\text{def}}{=} \langle \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \text{override}(\hat{\sigma}_Q, \sigma) \rangle \\ \therefore \sigma' &= \text{override}(\hat{\sigma}_Q, \sigma) \\ \therefore \hat{\sigma}'_E &= \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) \end{aligned}$$

Since $\hat{\sigma}_E \subseteq \sigma$, we apply Lemma 6.8.2.11 to the definition of σ' :

$$\begin{aligned} \sigma' &= \text{override}(\hat{\sigma}_Q, \sigma) \\ &= \text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma) \\ &= \text{override}(\hat{\sigma}'_E, \sigma) \end{aligned}$$

Also, because of Lemma 6.8.2.8, $\hat{\sigma}'_E \subseteq \sigma'$.

Lastly, applying Lemma 6.8.2.10 to the expansion of σ' above:

$$\begin{aligned} \text{dom}(\hat{\sigma}_E) &\subseteq \text{dom}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) \\ \text{dom}(\hat{\sigma}_E) &\subseteq \text{dom}(\hat{\sigma}'_E) \end{aligned}$$

Lemma 6.8.2.8 (Overrider Subset Relation):

This lemma states that the *overrider* will always be a subset of the *overridee*.

$$\forall \hat{\sigma}, \sigma (\hat{\sigma} \subseteq \text{override}(\hat{\sigma}, \sigma))$$

Proof: Recall:

$$\text{override}(\hat{\sigma}, \sigma) \stackrel{\text{def}}{=} \hat{\sigma} \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}$$

Trivially, $\hat{\sigma} \subseteq \hat{\sigma} \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}$.

Lemma 6.8.2.9 (Override Domain Dominance):

This states that if the *overrider* has a domain that subsumes the *overridee*, then the result will just be the *overrider*.

$$\forall \hat{\sigma}_1, \hat{\sigma}_2 : \text{dom}(\hat{\sigma}_1) \subseteq \text{dom}(\hat{\sigma}_2) (\\ \text{override}(\hat{\sigma}_2, \hat{\sigma}_1) = \hat{\sigma}_2 \\)$$

Recall:

$$\text{override}(\hat{\sigma}_2, \hat{\sigma}_1) \stackrel{\text{def}}{=} \hat{\sigma}_2 \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_2) \wedge p \in \text{dom}(\hat{\sigma}_1), b = \hat{\sigma}_1(p)\}$$

If $\text{dom}(\hat{\sigma}_1) \subseteq \text{dom}(\hat{\sigma}_2)$, it means that there are no paths p such that $p \notin \text{dom}(\hat{\sigma}_2) \wedge p \in \text{dom}(\hat{\sigma}_1)$.

In other words, the second term of the union is empty, so $\text{override}(\hat{\sigma}_2, \hat{\sigma}_1) \stackrel{\text{def}}{=} \hat{\sigma}_2$.

Lemma 6.8.2.10 (Overridee Domain Relationship):

This states that the result of overriding will have a domain that subsumes that of the *overridee*.

$$\forall \hat{\sigma}_1, \hat{\sigma}_2 (\text{dom}(\hat{\sigma}_2) \subseteq \text{dom}(\text{override}(\hat{\sigma}_1, \hat{\sigma}_2)))$$

Proof:

Corollary of Lemma 6.8.2.12, from which we get:

$$\text{dom}(\text{override}(\hat{\sigma}_1, \hat{\sigma}_2)) = \text{dom}(\hat{\sigma}_1) \cup \text{dom}(\hat{\sigma}_2)$$

Trivially, $\text{dom}(\hat{\sigma}_1) \subseteq \text{dom}(\hat{\sigma}_1) \cup \text{dom}(\hat{\sigma}_2)$.

Lemma 6.8.2.11 (Finite Segment Override):

This lemma provides a way to refactor an override between $\hat{\sigma}_Q$ and σ , to express it in terms of an override between $\hat{\sigma}_Q$ and some subset of σ first, before chaining in σ . This is useful because we encounter situations where we know things about a subset of σ , but not σ itself (since σ represents any concrete filesystem and has an infinite domain).

$$\forall \hat{\sigma}_Q, \sigma, \hat{\sigma}_E : \hat{\sigma}_E \subset \sigma (\\ \text{override}(\hat{\sigma}_Q, \sigma) = \text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma) \\)$$

Proof:

Firstly, let us prove that $\text{dom}(\text{override}(\hat{\sigma}_Q, \sigma)) = \text{dom}(\text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma))$.

From Lemma 6.8.2.12:

$$\begin{aligned}\text{dom}(\text{override}(\hat{\sigma}_Q, \sigma)) &= \mathcal{P} \\ \text{dom}(\text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma)) &= \mathcal{P}\end{aligned}$$

Now, let us prove that for each path in the domain, the valuation b is the same.

Recall:

$$\begin{aligned}\text{override}(\hat{\sigma}_Q, \sigma) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\} \\ \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}\end{aligned}$$

We know that $\text{dom}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) = \text{dom}(\hat{\sigma}_Q) \cup \text{dom}(\hat{\sigma}_E)$.

Therefore:

$$\begin{aligned}\text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\} \\ &\quad \cup \{p \rightarrow b \mid p \notin \text{dom}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\} \\ &= \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\} \\ &\quad \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}\end{aligned}$$

We can naively enumerate several cases for paths:

- $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
- $p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
- $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
- $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \notin \text{dom}(\sigma)$
- $p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
- $p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \notin \text{dom}(\sigma)$
- $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \notin \text{dom}(\sigma)$

However, because $\hat{\sigma}_E \subset \sigma$, it is not possible for a path p to exist where

- $p \in \text{dom}(\hat{\sigma}_E) \wedge p \notin \text{dom}(\sigma)$
- $p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$

Thus, we can reduce our cases to:

1. $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
2. $p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma)$
3. $p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \notin \text{dom}(\sigma)$

Since the expansions of both $\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)$ and $\text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma)$ contain the term $\hat{\sigma}_Q$, paths in case 1 and 3 have the valuation $\hat{\sigma}_Q(p)$ in both.

In the expansion of $\text{override}(\hat{\sigma}_Q, \sigma)$, paths in case 2 are contained in the term

$$\cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}$$

which gives those paths the valuation $\sigma(p)$.

In the expansion of $\text{override}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E), \sigma)$, paths in case 2 are contained in the term

$$\begin{aligned}&\{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\} \\ &\cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}\end{aligned}$$

which gives those paths either the valuation $\hat{\sigma}_E(p)$, or $\sigma(p)$.

However, since $\hat{\sigma}_E \subset \sigma$, $\forall p(\hat{\sigma}_E(p) = \sigma(p))$.

Lemma 6.8.2.12 (Domain of Override):

This lemma states that the result of an override operation has a domain that is the union of its two operands.

$$\forall \hat{\sigma}_Q, \hat{\sigma}_E (\text{dom}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) = \text{dom}(\hat{\sigma}_Q) \cup \text{dom}(\hat{\sigma}_E))$$

Proof:

Recall:

$$\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) \stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}$$

We have 3 cases for paths:

1. $p \in \text{dom}(\hat{\sigma}_1) \wedge p \in \text{dom}(\hat{\sigma}_2)$
2. $p \in \text{dom}(\hat{\sigma}_1) \wedge p \notin \text{dom}(\hat{\sigma}_2)$
3. $p \notin \text{dom}(\hat{\sigma}_1) \wedge p \in \text{dom}(\hat{\sigma}_2)$

Paths in case 1 and 2 are trivially included in $\text{dom}(\text{override}(\hat{\sigma}_1, \hat{\sigma}_2))$ because $\text{dom}(\text{override}(\hat{\sigma}_1, \hat{\sigma}_2))$ is comprised of $\hat{\sigma}_1$ itself unioned with another term.

Paths in case 3 are included in the second term, $\{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_1) \wedge p \in \text{dom}(\hat{\sigma}_2), b = \hat{\sigma}_2(p)\}$.

Lemma 6.8.2.13 (Subset Override Identity):

This lemma states that using a filesystem to override a subset of itself will simply yield the same filesystem.

$$\forall \hat{\sigma}_E, \sigma : \hat{\sigma}_E \subseteq \sigma(\text{override}(\hat{\sigma}_E, \sigma) = \sigma)$$

Proof:

We first prove equality of domain. From Lemma 6.8.2.12, $\text{dom}(\text{override}(\hat{\sigma}_E, \sigma)) = \mathcal{P}$. Also, by definition, $\text{dom}(\sigma) \stackrel{\text{def}}{=} \mathcal{P}$.

Next, we prove equality of valuation. Recall:

$$\text{override}(\hat{\sigma}_E, \sigma) \stackrel{\text{def}}{=} \hat{\sigma}_E \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_E) \wedge p \in \text{dom}(\sigma), b = \sigma(p)\}$$

Because $\text{dom}(\sigma) = \mathcal{P}$, $\forall p(p \in \text{dom}(\sigma))$. We can thus rewrite the above as:

$$\text{override}(\hat{\sigma}_E, \sigma) = \hat{\sigma}_E \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_E), b = \sigma(p)\}$$

We thus have just two cases for paths in $\text{override}(\hat{\sigma}_E, \sigma)$:

1. $p \in \text{dom}(\hat{\sigma})$
2. $p \notin \text{dom}(\hat{\sigma})$

Paths in case 1 correspond to the term in the expansion: $\hat{\sigma}_E$, which gives them the valuation $\hat{\sigma}_E(p)$. However, since $\hat{\sigma}_E \subseteq \sigma$, $\forall p(\hat{\sigma}_E(p) = \sigma(p))$. Therefore paths in case 1 have the valuation $\sigma(p)$.

Paths in case 2 correspond to the term in the expansion:

$$\{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_E), b = \sigma(p)\}$$

which gives them the valuation $\sigma(p)$.

Trivially, all paths in σ have the valuation $\sigma(p)$.

Lemma 6.8.2.14 (Formula Superset Solvability):

This lemma states that if a predicate expression is formed by calling toForm on a filesystem, then that expression can be satisfied by any filesystem that is equal to or which contains the original.

$$\forall \hat{\sigma}_E, \sigma : \hat{\sigma}_E \subseteq \sigma \left(\begin{array}{c} \sigma(\text{toForm}(\hat{\sigma}_E)) \\ \end{array} \right)$$

Proof:

Recall:

$$\text{toForm}(\hat{\sigma}_E) \stackrel{\text{def}}{=} \bigwedge \left\{ \begin{array}{ll} p & \text{if } \hat{\sigma}_E(p) \\ \neg p & \text{if } \neg \hat{\sigma}_E(p) \end{array} \mid p \in \text{dom}(\hat{\sigma}_E) \right\}$$

If $\hat{\sigma}_E \subseteq \sigma$, this implies that $\forall p \in \text{dom}(\hat{\sigma}_E) (p \in \text{dom}(\sigma) \wedge \hat{\sigma}_E(p) = \sigma(p))$.

This means that all paths p in $\text{toForm}(\hat{\sigma}_E)$ will be found in σ , and have the same valuation in σ as they do under $\hat{\sigma}_E$. Thus, substituting the valuations of σ to them must satisfy the expression as a whole.

Lemma 6.8.2.15 (Override Subset Preservation):

This lemma states that overriding a subset of a filesystem will produce a subset of overriding the whole filesystem.

$$\forall \hat{\sigma}_E, \sigma : \hat{\sigma}_E \subseteq \sigma \left(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) \subseteq \text{override}(\hat{\sigma}_Q, \sigma) \right)$$

Proof:

Recall:

$$\begin{aligned} \text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\} \\ \text{override}(\hat{\sigma}_Q, \sigma) &\stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\sigma), b = \sigma(p)\} \end{aligned}$$

First, we show that $\text{dom}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) \subseteq \text{dom}(\text{override}(\hat{\sigma}_Q, \sigma))$. Since both have $\hat{\sigma}_Q$, we focus on the right-hand side of each. We would like to show that

$$\begin{aligned} & \text{dom}(\{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}) \\ & \subseteq \text{dom}(\{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\sigma), b = \sigma(p)\}) \end{aligned}$$

We find 3 cases for paths p :

1. $p \in \hat{\sigma}_E \wedge p \in \sigma$
2. $p \notin \hat{\sigma}_E \wedge p \in \sigma$
3. $p \in \hat{\sigma}_E \wedge p \notin \sigma$

Since $\hat{\sigma}_E \subseteq \sigma$, Case 3 is impossible. Note that

$$\text{dom}(\{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\})$$

can only contain Case 2, while

$$\text{dom}(\{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\sigma), b = \sigma(p)\})$$

can contain both Cases 1 and 2.

Next, we prove that for each path that does exist in $\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)$, it has the same value assigned to it as it does in $\text{override}(\hat{\sigma}_Q, \sigma)$. Note that for $\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)$, every path's valuation is given either by:

1. the $\hat{\sigma}_Q$ term
 - in which case its value is $\hat{\sigma}_Q(p)$
2. or else it is given by $\{p \rightarrow b \mid p \in \text{dom}(\hat{\sigma}_Q) \wedge p \notin \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}$
 - in which case its value is $\hat{\sigma}_E(p)$.

In Case 1, both $\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)$ and $\text{override}(\hat{\sigma}_Q, \sigma)$ use $\hat{\sigma}_Q$. Since $\hat{\sigma}_E \subseteq \sigma$, for all paths p , $\hat{\sigma}_E(p) = \sigma(p)$.

Lemma 6.8.2.16 (Override FSC Preservation):

This lemma is not used explicitly in this proof, but is an important result that shows that the override operation preserves the FSC constraint, so that our idempotence analysis only outputs filesystems (or calculates intermediate ones) that make sense.

$$\begin{aligned} & \forall \hat{\sigma}_Q : \text{fsc}(\hat{\sigma}_Q), \hat{\sigma}_E : \text{fsc}(\hat{\sigma}_E) (\\ & \qquad \qquad \qquad \text{fsc}(\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E)) \\ & \qquad \qquad \qquad) \end{aligned}$$

Proof: Recall:

$$\begin{aligned} \text{fsc}(\hat{\sigma}) \Leftrightarrow \forall p, p_c : p \in \text{dom}(\hat{\sigma}) \wedge p = \text{par}(p_c) (\\ & \qquad \qquad \qquad \hat{\sigma}(p) \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}) \wedge \hat{\sigma}(\text{par}(p)) \\ & \qquad \qquad \qquad \wedge \neg \hat{\sigma}(p) \Rightarrow p_c \in \text{dom}(\hat{\sigma}) \wedge \neg \hat{\sigma}(p_c) \\ & \qquad \qquad \qquad) \end{aligned}$$

Note that it is impossible for a path to be a child of another path ending with a file; that is, $\forall p_1, p_2 (\neg \text{isFile}(p_1) \vee p_1 \neq \text{par}(p_2))$. Thus, we will not have to check for this case in any of our work below.

We know:

$$\begin{aligned} & \forall p, p_c : p = \text{par}(p_c) (\\ & \quad p \in \text{dom}(\hat{\sigma}_Q) \wedge \hat{\sigma}_Q \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}_Q) \wedge \hat{\sigma}_Q(\text{par}(p)) \\ & \quad p \in \text{dom}(\hat{\sigma}_Q) \wedge \neg \hat{\sigma}_Q \Rightarrow p_c \in \text{dom}(\hat{\sigma}_Q) \wedge \neg \hat{\sigma}_Q(p_c) \\ & \quad p \in \text{dom}(\hat{\sigma}_E) \wedge \hat{\sigma}_E \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}_E) \wedge \hat{\sigma}_E(\text{par}(p)) \\ & \quad p \in \text{dom}(\hat{\sigma}_E) \wedge \neg \hat{\sigma}_E \Rightarrow p_c \in \text{dom}(\hat{\sigma}_E) \wedge \neg \hat{\sigma}_E(p_c) \\ &) \end{aligned}$$

We also know:

$$\text{override}(\hat{\sigma}_Q, \hat{\sigma}_E) \stackrel{\text{def}}{=} \hat{\sigma}_Q \cup \{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}$$

And due to Lemma 6.8.2.12:

$$\text{dom}(\hat{\sigma}_R) = \text{dom}(\hat{\sigma}_Q) \cup \text{dom}(\hat{\sigma}_E)$$

We have 2 cases of $p \in \text{dom}(\hat{\sigma}_R)$:

1. $p \in \text{dom}(\hat{\sigma}_Q)$
2. $p \notin \text{dom}(\hat{\sigma}_Q)$

In Case 1 we have two subcases:

1. $\hat{\sigma}_Q(p)$
2. $\neg \hat{\sigma}_Q(p)$

In Subcase 1:

$$p \in \text{dom}(\hat{\sigma}_Q) \wedge \hat{\sigma}_Q(p) \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}_Q) \wedge \hat{\sigma}_Q(\text{par}(p))$$

Since $\hat{\sigma}_Q \subseteq \hat{\sigma}_R$, this implies:

$$\text{par}(p) \in \text{dom}(\hat{\sigma}_R) \wedge \hat{\sigma}_R(\text{par}(p))$$

In Subcase 2, for all $p_c : p = \text{par}(p_c)$:

$$p \in \text{dom}(\hat{\sigma}_Q) \wedge \neg \hat{\sigma}_Q(p) \Rightarrow p_c \in \text{dom}(\hat{\sigma}_Q) \wedge \neg \hat{\sigma}_Q(p_c)$$

Since $\hat{\sigma}_Q \subseteq \hat{\sigma}_R$, this implies:

$$p_c \in \text{dom}(\hat{\sigma}_R) \wedge \neg \hat{\sigma}_R(p_c)$$

In Case 2, we again have two subcases:

1. $\hat{\sigma}_E(p)$
2. $\neg \hat{\sigma}_E(p)$

In Subcase 1:

$$p \in \text{dom}(\hat{\sigma}_E) \wedge \hat{\sigma}_E(p) \Rightarrow \text{par}(p) \in \text{dom}(\hat{\sigma}_E) \wedge \hat{\sigma}_E(\text{par}(p))$$

$\text{par}(p)$ must be an element of $\hat{\sigma}_E$, and may or may not be an element of $\hat{\sigma}_Q$. If it is not an element of $\hat{\sigma}_Q$, then it is an element of the $\{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}$ term of $\hat{\sigma}_R$. Since this is a subset of $\hat{\sigma}_R$, this implies:

$$\text{par}(p) \in \text{dom}(\hat{\sigma}_R)$$

Its boolean value also comes from $\hat{\sigma}_E(\text{par}(p))$. Since in our premise we assume that $\text{fsc}(\hat{\sigma}_E) = \text{true}$, $\hat{\sigma}_E(\text{par}(p)) = \text{true}$. Therefore,

$$\text{par}(p) \in \text{dom}(\hat{\sigma}_R) \wedge \hat{\sigma}_R(p)$$

If on the other hand $\text{par}(p) \in \hat{\sigma}_Q$, then since $\hat{\sigma}_Q \subseteq \hat{\sigma}_R$, this implies:

$$\text{par}(p) \in \text{dom}(\hat{\sigma}_R)$$

However, its value $\hat{\sigma}_R(\text{par}(p))$ will be given by the $\hat{\sigma}_Q$ term, as $\hat{\sigma}_Q(\text{par}(p))$. Luckily, this is not a problem. If $\hat{\sigma}_Q(\text{par}(p)) = \text{true}$, then we already satisfy the fsc condition.

On the other hand, it can never occur that $\hat{\sigma}_Q(\text{par}(p)) = \text{false}$, because in our premise, $\text{fsc}(\hat{\sigma}_Q) = \text{true}$, which implies that $p \in \text{dom}(\hat{\sigma}_Q)$, which contradicts the Case 2 premise that $p \notin \text{dom}(\hat{\sigma}_Q)$.

Therefore, we see that:

$$\text{par}(p) \in \text{dom}(\hat{\sigma}_R) \wedge \hat{\sigma}_R(\text{par}(p))$$

In Subcase 2, for all $p_c : p = \text{par}(p_c)$:

$$p \in \text{dom}(\hat{\sigma}_E) \wedge \neg \hat{\sigma}_E(p) \Rightarrow p_c \in \text{dom}(\hat{\sigma}_E) \wedge \neg \hat{\sigma}_E(p_c)$$

For each p_c , either:

- $p_c \in \text{dom}(\hat{\sigma}_Q)$
- $p_c \notin \text{dom}(\hat{\sigma}_Q)$

If $p_c \notin \text{dom}(\hat{\sigma}_Q)$, then it is an element of the $\{p \rightarrow b \mid p \notin \text{dom}(\hat{\sigma}_Q) \wedge p \in \text{dom}(\hat{\sigma}_E), b = \hat{\sigma}_E(p)\}$ term of $\hat{\sigma}_R$. Since this is a subset of $\hat{\sigma}_R$, this implies:

$$p_c \in \text{dom}(\hat{\sigma}_R)$$

The boolean value of p_c also comes from $\hat{\sigma}_E(p_c)$. Since we assume in our premise that $\text{fsc}(\hat{\sigma}_E) = \text{true}$, then $\hat{\sigma}_E(p_c) = \text{false}$. Thus,

$$p_c \in \text{dom}(\hat{\sigma}_R) \wedge \neg \hat{\sigma}_R(p_c)$$

If $p_c \in \text{dom}(\hat{\sigma}_Q)$, then since $\hat{\sigma}_Q \subseteq \hat{\sigma}_R$, $p_c \in \text{dom}(\hat{\sigma}_R)$.

However, the boolean value of p_c comes from $\hat{\sigma}_Q(p_c)$. Again, luckily we see that this is not a problem. If $\hat{\sigma}_Q(p_c) = \text{false}$, then we already satisfy the fsc condition.

On the other hand, we see that it is impossible for $\hat{\sigma}_Q(p_c) = \text{true}$ in this situation. Since we assumed in our premise that $\text{fsc}(\hat{\sigma}_Q) = \text{true}$, this would imply that $p \in \text{dom}(\hat{\sigma}_Q)$, which contradicts the premise of Case 2. Therefore:

$$p_c \in \text{dom}(\hat{\sigma}_R) \wedge \neg \hat{\sigma}_R(p_c)$$

7 Formalizing and Desugaring Ansible

Now that we have seen BBFS and how it facilitates idempotence checking, we shall begin the process of transforming Ansible into BBFS for analysis. In order to do this, we must first attempt to formalize Ansible to some extent.

7.1 Extent of Formalization

Ansible is a large and active language. It is not yet formalized to the best of knowledge of the author, and its implementation, which is in Python, has undergone many iterations and changes over its lifespan. It has a wide variety of features with complicated interactions, some of which seem to not be completely documented, as evidenced by the author's personal attempts to search for them, as well as the fact that multiple Ansible users appear to have done informal empirical experimentation in order to figure the details and edge cases out for themselves. These details include surprisingly core aspects of Ansible, such as the order of execution of handlers, and the

As such, we will only explicitly formalize the syntax of Ansible. The semantics of Ansible will be implied through the way we desugar and eventually translate it into BBFS, whose semantics we have already formalized. This desugaring and translation process is informed by the author's own empirical experience and experimentation with Ansible.

7.2 Simplification

Being a very active and industry-oriented language, Ansible contains a vast amount of small, disparate features and tiny, unconnected sub-systems, many of which have case-by-case interactions between each other which must be memorized; many of which furthermore are not completely documented.

Therefore, for reasons of scope, we have attempted to isolate only the most core parts of Ansible into a subset which we call $\mathcal{A}^H$. We have also taken the liberty of simplifying some aspects of Ansible whose subtleties do not contribute much to the formalization.

For instance, in Ansible, a task may notify a handler either by the task directly specifying the *name* of the handler:

```
- tasks:
  - name: task1
    debug:
      msg: "I am a task"
    notify: handler1

- handlers:
  - name: handler1
    debug:
      msg: "I am a handler"
```

or by having both the handler and the task subscribe to the same *topic*:

```
- tasks:
  - name: task1
    debug:
      msg: "I am a task"
    notify: topic1

- handlers:
  - name: handler1
    debug:
```

```
msg: "I am a handler"
listen: topic1
```

Furthermore, when there are multiple handlers, both with the same name (which is, for some reason, allowed in Ansible) and both subscribed to the same topic, the behavior between *name* and *topic* notifications differs very subtly. When a topic notifies the *name*, only the last handler with that name is executed. When a topic notifies the *topic*, both are executed. What does “last” handler mean? Ansible prescribes the following order:

1. Handlers from the roles section of a play
2. Handlers from the handlers section of a play
3. Statically imported handlers
4. Dynamically imported handlers

These details do not contribute much to the core functionality of the language, and needlessly complicate formalization. Thus, in our subset $\mathcal{A}^H$, we simplify this by:

- making it such that handlers can only be notified by topic
- removing the name attribute altogether, since it serves no other purpose

These are the kinds of simplifications we have made in order to arrive at $\mathcal{A}^H$.

7.3 Syntax of $\mathcal{A}^H$

7.3.1 Preliminary notes

Recall from Section 3.3 that the `goalkeeper`, `when`, `ignoreError` and `register` fields below (they will be marked) are not accessible to the human who is writing the $\mathcal{A}^H$ program. They are present in the syntax because they are involved in the intermediate or final steps of desugaring into $\mathcal{A}^{\min}$. In $\mathcal{A}^H$, a program from the user will start with those fields set to default values.

Note that the exact default value for `register` is not important as long as it is unique per task, and so we will assume that some UID subsystem exists to auto-assign these values.

Finally, since a lot of Ansible constructs are record types with set named fields, we use notation suitable for this. For example:

$$\begin{aligned} \text{MD} ::= & \text{GenericModDecl } \{ \text{name} : s, \text{params} : \overline{\langle s \rightarrow \text{VAR} \rangle} \} \\ & | \text{ImportRole } \{ \text{name} : \text{VAR}, \text{tasksFrom} : \text{VAR}, \text{handlersFrom} : \text{VAR} \} \\ & | \text{ImportTasks } \{ \text{file} : \text{VAR} \} \end{aligned}$$

Here,

- ALLCAPS denotes the symbol for this grammar type
- UpperCamelCase is for the name of each subtype
- lowerCamelCase is for the name of a field

7.3.2 Syntax Proper

$$\text{ROOT} ::= \text{RootDir } \{ \text{playbook} : \overline{\text{P}}, \text{roledir} : \overline{\langle s \rightarrow \text{R} \rangle}, \text{looseTaskFiles} : \overline{\langle s \rightarrow \overline{\text{T}} \rangle} \}$$

$$\text{P} ::= \text{Play } \{ \text{vars} : \overline{\langle s \rightarrow \text{VAR} \rangle} \mid \varepsilon, \text{roleNames} : \overline{\text{S}}, \text{tasks} : \overline{\text{T}}, \text{handlers} : \overline{\text{T}} \}$$

$$\text{R} ::= \text{Role } \{ \text{tasksDir} : \overline{\langle \text{N} \rightarrow \overline{\text{T}} \rangle}, \text{handlersDir} : \overline{\langle \text{N} \rightarrow \overline{\text{T}} \rangle} \}$$

$$\text{N} ::= \text{MainName} \mid \text{OtherName } s$$

$$\text{T} ::= \text{Atomic } \{ \text{attSet} : \text{AAS}, \text{modDecl} : \text{MD} \}$$

$$| \text{Blocktask } \{ \text{attSet} : \text{BAS}, \text{block} : \text{BLOCK} \}$$

BLOCK $\coloneqq$ Block $\{ \text{blockMain} : \overline{\text{T}},$
 $\text{rescue} : \overline{\text{T}} \mid \varepsilon,$
 $\text{always} : \overline{\text{T}} \mid \varepsilon,$
 $\text{goalkeeper} : \varepsilon \mid \text{T} \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value } \varepsilon$
 $\}$

AAS $\coloneqq$ AtomicAttributeSet $\{$
 $\text{notify} : \overline{\text{VAR}}$
 $\text{loop} : \text{KWLOOP} \mid \varepsilon$
 $\text{listen} : \overline{\text{VAR}}$
 $\text{vars} : \langle s \rightarrow \text{VAR} \rangle$

 $\text{when} : \text{JBE} \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value} : \text{true}$

 $\text{ignoreErrors} : b \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value} : \text{false}$
 $\text{register} : s \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value auto-assigned}$
 $\}$

BAS $\coloneqq$ BlockAttributeSet $\{$
 $\text{notify} : \overline{\text{VAR}}$
 $\text{vars} : \langle s \rightarrow \text{VAR} \rangle$

 $\text{when} : \text{JBE_EXP} \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value} : \text{true}$
 $\text{ignoreErrors} : b \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value} : \text{false}$
 $\text{register} : s \quad \text{Unavailable to user in } \mathcal{A}^H, \text{ default value auto-assigned}$
 $\}$

KWLOOP $\coloneqq$ KwLoop $\{ \text{loopList} : \overline{\text{VAR}}, \text{loopVar} : \text{VAR}, \text{indexVar} : \text{VAR} \mid \varepsilon \}$

MD $\coloneqq$ GenericModDecl $\{ \text{name} : s, \text{params} : \langle s \rightarrow \text{VAR} \rangle \}$

$\mid \text{ImportRole} \{ \text{name} : \text{VAR}, \text{tasksFrom} : \text{VAR}, \text{handlersFrom} : \text{VAR} \}$
 $\mid \text{ImportTasks} \{ \text{file} : \text{VAR} \}$

VAR $\coloneqq b \mid s \mid \text{JP}$

JP $\coloneqq \overline{\text{JSE}} \mid \text{JBE_EXP} \mid \text{Uvr } s$

JSE $\coloneqq s \mid \text{JseUvr } s$

Note that the “ s is JBE_TEST” construct is not allowed to be used in $\mathcal{A}^H$.

$$\begin{aligned} \text{JBE_EXP} ::= & s \text{ is JBE_TEST} \\ & | \text{JBE_EXP JBE_BINOP JBE_EXP} \\ & | \neg \text{JBE_EXP} \\ & | (\text{JBE_EXP}) \\ & | b \end{aligned}$$

$$\text{JBE_BINOP} ::= \wedge \mid \vee$$

$$\text{JBE_TEST} ::= \text{succeeded} \mid \text{failed} \mid \text{defined}$$

7.4 Syntax of $\mathcal{A}^{\min}$

$$\text{T} ::= \text{Atomic} \{ \text{attSet} : \text{AS}, \text{modDecl} : \text{MD} \}$$

$$\text{MD} ::= \text{GenericModDecl} \{ \text{name} : s, \text{params} : \overline{\langle s \rightarrow \text{VAR} \rangle} \}$$

$$\begin{aligned} \text{AAS} ::= & \text{AtomicAttributeSet} \{ \\ & \quad \text{when} : \text{JBE_EXP} \\ & \quad \text{register} : s \\ & \quad \text{ignoreErrors} : b \\ & \} \end{aligned}$$

$$\begin{aligned} \text{JBE_EXP} ::= & s \text{ is JBE_TEST} \\ & | \text{JBE_EXP JBE_BINOP JBE_EXP} \\ & | \neg \text{JBE_EXP} \\ & | (\text{JBE_EXP}) \\ & | b \end{aligned}$$

$$\text{JBE_BINOP} ::= \wedge \mid \vee$$

$$\text{JBE_TEST} ::= \text{succeeded} \mid \text{failed} \mid \text{defined}$$

7.5 Desugaring $\mathcal{A}^H$ into $\mathcal{A}^{\min}$

$\mathcal{A}^H$ represents the Ansible as it is written by a human, using many constructs such as loops and defining procedures (with roles) to increase ease of expression. However, to facilitate our eventual translation to BBFS, we will first desugar $\mathcal{A}^H$ into a minimal subset of Ansible, $\mathcal{A}^{\min}$. This mostly involves inlining imports, unrolling loops and replacing Ansible constructs like blocks with equivalent but more verbose rewrites in order to reduce the scope of Ansible syntax required to convey the same semantics.

That being said, the focus of this thesis is not on formalizing the semantics of Ansible. The Ansible interpreter is not formally specified, written in Python and constantly updated, so it would be a monumental effort for a moving target. As such, this desugaring should be taken as a translational definition of $\mathcal{A}^{\min}$ based on documentation and experimentation with Ansible, rather than a verified translation.

The constructs in $\mathcal{A}^H$ that we can desugar are:

- Inline roles
- Imports of roles and tasks
- Static variables
- Blocks
- Loops
- Handlers

- Plays and Playbooks
 - Technically, Ansible cannot function without plays and playbooks, which serve similar purposes to the main function in other languages. Once we desugar these, we are firmly in $\mathcal{A}^{\min}$.

Our desugaring process handles one construct at a time. However, there are certain dependencies between these constructs, which gives us a topological ordering of sorts for our desugaring process.

The first dependency is that inline roles and imports of roles and tasks must be desugared first before anything else, because desugaring them might yield any other construct.

The second dependency is that static variables must be desugared (resolved) before loops can be unrolled, because the number of iterations might be determined by the variable.

These are the only two necessary dependencies for us to perform desugaring in a single pass. Our order of transformations is thus:

- Desugar Roles in Plays
- Desugar Imports
- Static Variable Desugaring
- Desugar Blocks
- Desugar Loops
- Desugar Handlers
- Desugar Plays and Playbooks

The following sections will each tackle one of these points; each section will also present one rewrite rule that does desugaring of its construct. Together, these rewrite rules form a translational definition of $\mathcal{A}^{\min}$ from $\mathcal{A}^H$, which we propose as a minimal subset of Ansible. These rewrite rules are summarized in Section 7.6.

7.5.1 Desugaring Inline Roles in Plays

Roles are essentially a package of tasks and handlers that are used in a play. There are two kinds of roles; the first is what we will call *inline roles*. This means writing the name of the role in the `roles` section of a play.

```
---
- roles:
  - r1
```

Code Block 1

The second is through using a task containing a special module called `import_role`. In $\mathcal{A}^H$, we call these `ImportRoles`.

```
---
- tasks:
  - name: import myRole
    import_role:
      name: r1
```

Code Block 2

Both methods do essentially the same thing, with some very minor differences.

When a role is used in a play, its list of tasks are essentially copy-pasted into the play statically before the program executes. If the role is inline, then its tasks will all run before the `tasks` section of the play. If the role is added through an `ImportRole`, then the role's tasks will run at the point in the `tasks` list.

As for its handlers, they will be similarly copy-pasted into the handler list of the play and can be notified by tasks in the play. We have already seen from Section 3.2.5 that inline roles' handlers run first, followed by the handlers list, and lastly handlers imported by `import_role`.

An Ansible project has a mandated file structure when using `import_role` and `import_tasks`:

```
myAnsibleProject/
├─ playbook.yml
├─ tasks1.yml
├─ tasks2.yml
└─ roles/
    ├─ r1/
    │   ├─ tasks/
    │   │   ├─ main.yml
    │   │   └─ cleanup.yml
    │   └─ handlers/
    │       └─ main.yml
    └─ r2/
        └─ tasks/
            └─ main.yml
```

Code Block 3

`import_tasks` is used to import loose task files such as `tasks1.yml` or `tasks2.yml`, and `import_role` is used to import roles such as `r1` or `r2`. As suggested by the file structure shown above, a role is comprised of:

- A set of files, each containing a list of tasks
- A set of files, each containing a list of handlers

For example, the contents of `roles/r1/tasks/main.yml` might be:

```
---
- debug:
    msg: "first task from r1/tasks/main.yml"

- debug:
    msg: "second task from r1/tasks/main.yml"
```

Code Block 4

and the contents of `roles/r1/handlers/main.yml` might be:

```
---
- debug:
    msg: "first handler from r1/handlers/main.yml"
    listen: topic1

- debug:
    msg: "second handler from r1/handlers/main.yml"
    listen: topic1
```

Code Block 5

These are not playbooks; the top level of each file is just a list of tasks/handlers.

For inline roles, only the file with the name `main.yml` will be imported; one for tasks and one for handlers, if they exist. For `import_role`, there are optional parameters `tasks_from` and `handlers_from`, which allow a file to be selected by name.

Recall that we use the term “inline roles” to mean roles listed in the `roles` section of a play. For instance, let us say we have a play:

```

---
- roles:
  - r1

  tasks:
    - debug:
      msg: "first task from play1"

  handlers:
    - debug:
      msg: "first handler from play1"
      listen: topic1

```

Code Block 6

To desugar this, tasks from `r1` are inlined and prepended to the tasks list of the play; similarly, handlers from `r1` are prepended to the handlers list of the play. The end result of desugaring Code Block 6 would be:

```

---
- tasks:
  - debug:
    msg: "first task from r1/tasks/main.yml"

  - debug:
    msg: "second task from r1/tasks/main.yml"

  - debug:
    msg: "first task from play1"

  handlers:
  - debug:
    msg: "first handler from r1/handlers/main.yml"
    listen: topic1

  - debug:
    msg: "second handler from r1/handlers/main.yml"
    listen: topic1

  - debug:
    msg: "first handler from play1"
    listen: topic1

```

Code Block 7

However, recall that we also have `ImportRoles`. Instead of having:

- One rewrite rule for desugaring inline roles into tasks and handlers
- Another rewrite rule for desugaring `ImportRoles` into tasks and handlers

We will instead convert inline roles into `ImportRoles`, and then desugar all `ImportRoles` into tasks and handlers. Converting inline roles into `ImportRoles` is relatively trivial, whereas desugaring directly to tasks and handlers requires us to include context which would necessitate reaching all the way up to `RootDir`.

This section therefore covers transforming **inline roles** into **calls to `import_role`**. Instead of transforming Code Block 6 directly into Code Block 7, the rewrite rule in this section will transform Code Block 6 into:

```

---
- tasks:
  - import_role:
      name: r1

  - debug:
      msg: "first task from play1"

- handlers:
  - debug:
      msg: "first handler from play1"
      listen: topic1

```

Code Block 8

Desugaring Code Block 8 to Code Block 7 is done in the next section on desugaring ImportRoles.

7.5.1.1 Utility Functions

$$\text{toImport}(r) = \text{ImportRole } \{\text{name} : r\}$$

7.5.1.2 Rewrite Rule

$$\frac{\bar{t}' = \text{map}(\text{toImport}, \bar{r}) \# \bar{t}}{\text{Play } \{\text{roleNames} : \bar{r}, \text{tasks} : \bar{t}\} \rightarrow \text{Play } \{\text{roleNames} : [], \text{tasks} : \bar{t}'\}}$$

7.5.1.3 A Note on Handlers

Ansible specifies a certain ordering on its handler constructs:

1. Handlers from the `roles` section of a play
2. Handlers from the `handlers` section of a play
3. Handlers from calls to `import_role` (static)

Since handlers are executed in this order, we would ideally have a way to preserve it. If this is desired, `toImport()` can include a mechanism to mark all the `import_role` tasks created through inline roles as originating from an inline role, so that the following step of desugaring `import_role` tasks will be able to handle them separately.

This will not be heavily covered in the formalization and is presumed to have been done using some simple UID system.

7.5.2 Desugaring ImportTasks and ImportRole

Here is where we transform Code Block 8 into Code Block 7, completing the desugaring process for any inline roles, as well as performing desugaring for any roles from `import_role` calls. The core idea is very simple — it is simply taking the contents of files and copying it into the play. The complexity mainly comes from covering the various ways that Ansible allows this to be done; either through `import_role` or `import_tasks`.

In this step, we also desugar any `import_tasks` calls, which are very similar to `import_role` calls, but instead of a role, takes in the name of a single loose task file. For example, if we have a play:

```

---
- tasks:
  - import_tasks:
      file: tasks1.yml

```

Code Block 9

with the same Ansible project structure from Code Block 3, and where the content of `tasks1.yml` is:

```

---
- debug:
  msg: "first task from tasks1.yml"

- debug:
  msg: "second task from tasks1.yml"

```

Code Block 10

Then Code Block 9 will be desugared to:

```

---
- tasks:
  - debug:
    msg: "first task from tasks1.yml"

  - debug:
    msg: "second task from tasks1.yml"

```

Code Block 11

The last detail is that any attributes of an `import_tasks` or `import_role` task should be passed down to its desugared tasks. For instance, if it has some variable declarations (we will elaborate on how these work more in Section 7.5.3):

```

---
- tasks:
  - import_tasks:
    file: tasks2.yml
    vars:
    myvar: "hello"

```

Code Block 12

and the contents of `tasks2.yml` are:

```

---
- debug:
  msg: "{myvar} from the first task from tasks2.yml"

- debug:
  msg: "{myvar} from the second task from tasks2.yml"

```

Code Block 13

this will be desugared to:

```

---
- tasks:
  - debug:
    msg: "{myvar} from the first task from tasks1.yml"
    vars:
    myvar: "hello"

  - debug:
    msg: "{myvar} from the second task from tasks1.yml"
    vars:
    myvar: "hello"

```

Code Block 14

Let us first deal with a single play. We want a function `desugarImportsInPlay` that takes in a play like Code Block 8 and outputs an unrolled version like Code Block 7.

As a side note, in order to reference the contents of the role that we wish to import, we must also take in the root directory rd (of type RootDir in $\mathcal{A}^H$), which has access to both the subdirectory where all roles are stored, as well as all task files.

7.5.2.1 Utility Functions

A note on syntax:

- The symbol $\oplus$ is used to describe a map update in the context of an object with named fields.
- A subscript such as $\text{func}_{\text{context}}$ is used to denote parameters which are passed to the function but might not be used immediately in the function itself and mostly serve to be passed down to sub-functions, similar to information stored in a Reader monad.

The below function is the parent function that does the desugaring for ImportTasks and ImportRoles . It searches for ImportTasks and ImportRoles within the play's tasks, and desugars each import, creating a new task list and handler list, with which it creates a modified version of the play.

$$\text{desugarImportsInPlay}_{rd}(p) = p \oplus \{\text{tasks} : \text{newTasks}, \text{handlers} : p.\text{handlers} \# \text{newHandlers}\}$$

where:

$$\langle \text{newTasks}, \text{newHandlers} \rangle = \text{desugarImports}_{rd}(p.\text{tasks})$$

It does so by calling desugarImports , which is simply a map of the real work-doing function over the task list:

$$\text{desugarImports}_{rd}(\bar{t}) = \text{unzip}(\text{map}(\text{getImportsFromTask}_{rd}, \bar{t}))$$

$\text{getImportsFromTask}$ is a function that converts an individual task into its constituent tasks and handlers. There are 4 cases of what an individual task can be:

1. Atomic task that is not an import
2. ImportRole
3. ImportTasks
4. A block-containing task (which cannot be an import)

$\text{getImportsFromTask}$ also makes sure any relevant attributes in the attribute sets of the ImportRole or ImportTasks get passed down to the desugared tasks and handlers by calling parAtts (which stands for parent attributes).

$$\text{getImportsFromTask}(t) = \begin{cases} \langle [t], [] \rangle & \text{if Case 1} \\ \langle \text{tasksFromRole}, \text{handlersFromRole} \rangle & \text{if Case 2} \\ \langle \text{tasksFromFile}, [] \rangle & \text{if Case 3} \\ \langle [\text{newBlocktask}], \text{handlersFromBlock} \rangle & \text{if Case 4} \end{cases}$$

where:

$\text{tasksFromRole} = \text{map}(\lambda t'. \text{parAtts}(t.\text{attSet}, t'), \text{getTasksFromRole}_{\text{rd}}(t.\text{modDecl.name}, t.\text{modDecl.tasksFrom}))$

$\text{handlersFromRole} = \text{map}(\lambda t'. \text{parAtts}(t.\text{attSet}, t'), \text{getHandlersFromRole}_{\text{rd}}(t.\text{modDecl.name}, t.\text{modDecl.handlersFrom}))$

$\text{tasksFromFile} = \text{map}(\lambda t'. \text{parAtts}(t.\text{attSet}, t'), \text{getTasksFromFile}_{\text{rd}}(t.\text{name}, t'))$

$\text{newBlock} = \text{Block} \{ \text{blockMain} : \text{ebm}, \text{rescue} : \text{er}, \text{always} : \text{ea} \}$

$\text{newBlocktask} = \text{Blocktask} \{ \text{attSet} : t.\text{attSet}, \text{block} : \text{newBlock} \}$

$\langle \text{ebm}, \text{handlersFromBM} \rangle = \text{desugarImports}_{\text{rd}}(t.\text{blockMain})$

$\langle \text{er}, \text{handlersFromR} \rangle = \text{desugarImports}_{\text{rd}}(t.\text{rescue})$

$\langle \text{ea}, \text{handlersFromA} \rangle = \text{desugarImports}_{\text{rd}}(t.\text{always})$

$\text{handlersFromBlock} = \text{map}(\lambda t'. \text{parAtts}(t.\text{attSet}, t'), \text{handlersFromBM} \# \text{handlersFromR} \# \text{handlersFromA})$

Case 1 is when: t is Atomic $\wedge t.\text{modDecl}$ is GenericModDecl

Case 2 is when: t is Atomic $\wedge t.\text{modDecl}$ is ImportRole

Case 3 is when: t is Atomic $\wedge t.\text{modDecl}$ is ImportTasks

Case 4 is when: t is Blocktask

The definition of parAtts is below. The three attributes that can be passed down are notify , when and vars (all of which we will see later on in Section 7.5.6, Section 7.5.4 and Section 7.5.3 respectively).

$\text{parAtts}(a, t) = t \oplus \{ \text{attSet} : t.\text{attSet} \oplus \text{modifiedAttSet} \}$

where:

$\text{modifiedAttSet} = \begin{cases} \text{AtomicAttributeSet} \{ \text{notify} : \text{newNotify}, \text{when} : \text{newWhen}, \text{vars} : \text{newVars} \} & \text{if } t \text{ is Atomic} \\ \text{BlockAttributeSet} \{ \text{notify} : \text{newNotify}, \text{when} : \text{newWhen}, \text{vars} : \text{newVars} \} & \text{if } t \text{ is Blocktask} \end{cases}$

$\text{newNotify} = a.\text{notify} \# t.\text{attSet}.\text{notify}$

$\text{newWhen} = a.\text{when} \wedge t.\text{attSet}.\text{when}$

$\text{newVars} = a.\text{vars} \# t.\text{attSet}.\text{vars}$

Finally, we have the functions that grab the tasks and handlers from the relevant places in the project folder:

$\text{getTasksFromRole}_{\text{rd}}(\text{roleName}, \text{taskFileName}) = \text{rd.roleDir}[\text{roleName}].\text{tasksDir}[\text{taskFileName}]$

$\text{getHandlersFromRole}_{\text{rd}}(\text{roleName}, \text{handlerFileName}) = \text{rd.roleDir}[\text{roleName}].\text{handlersDir}[\text{handlerFileName}]$

$\text{getTasksFromFile}_{\text{rd}}(\text{fileName}) = \text{rd.looseTaskFiles}[\text{fileName}]$

7.5.2.2 A Note on Handlers

Recall the order of handlers mentioned in the previous section on inline roles. If we want to preserve this ordering, we can simply run this desugaring step twice:

- Once with a version of $\text{desugarImportsInPlay}$ that only targets import_role tasks marked as having originated from inline roles, and which prepends newHandlers to $p.\text{handlers}$, instead of appending it to the back.
- Then again with the version detailed above.

7.5.2.3 Rewrite Rule

$$\begin{array}{c}
 \text{rd}_1 = \text{RootDir } \{\text{playbook} : \bar{p}\}, \\
 \exists p \in \bar{p} : \exists t \in p.\text{tasks} : t.\text{modDecl of type ImportRole} \vee t.\text{modDecl of type ImportTasks} \\
 \bar{p}' = \text{map}(\text{desugarImportsInPlay}_{\text{rd}}, p) \\
 \text{rd}_2 = \text{rd}_1 \oplus \{\text{playbook} : \bar{p}'\} \\
 \hline
 \text{rd}_1 \rightarrow \text{rd}_2
 \end{array}$$

7.5.3 Desugaring Static Variables

An example of a variable defined on the play level is:

```

---
- vars:
    myvar: "hello"

tasks:
- debug:
    msg: myvar

```

Code Block 15

An example of a variable defined on the blocktask level is:

```

---
- tasks:
  - vars:
      myvar: "hello"
    block:
      - debug:
          msg: myvar

```

Code Block 16

An example of a variable defined on the atomic task level is:

```

---
- tasks:
  - debug:
      msg: myvar
    vars:
      myvar: "hello"

```

Code Block 17

Variables declared at higher levels are overridden by more local definitions.

Variables can also either be used directly, or can be templated into a larger string. For instance,

```

---
- tasks:
  - debug:
      msg: "John says {myvar}!"
    vars:
      myvar: "hello"

```

Code Block 18

In this thesis, variables can take the form of booleans and strings. If a boolean is used in a template, it will be coerced into a string form.

In this thesis, we only consider static variables; i.e. variables whose values are known at “compile time” (Ansible is not compiled).

First, we will establish a type to represent variables. As mentioned, a variable can either be a string or a boolean. To this we will add an additional type JP that represents a template or a variable that is instantiated somewhere else.

$$\text{VAR} ::= b \mid s \mid \text{JP}$$

JP can either be:

- A single unresolved variable. We call this UVR.
- A string template. This must eventually evaluate into a string. We call this JSE.
- A boolean template. This must eventually evaluate into a boolean. We call this JBE_EXP.

Why do we need these 3 subcategories of JP? In essence,

- When we see a parameter like `param1: x` and `y`, we know it has to resolve to a boolean eventually, so it must be a JBE_EXP.
- When we see `param1: "x and {y}"`, we know it has to resolve to a string eventually, so it must be a JSE.
- When we just see `param1: x`, we do not know what type it is, so we call it a UVR.

An example of UVR has already been seen in Code Block 15; the type of `myvar` in `msg: myvar` is a UVR. Note that the `myvar` in `myvar: "hello"` is **not** a UVR; that is simply the key for the string "hello". UVRs are the key name being used *as a reference value* in a parameter such as `msg`.

The defining characteristic of a UVR, compared to a JSE or JBE_EXP, is that UVRs only have one part: an unquoted name (such as `myvar`) which references a variable name defined elsewhere.

An example of JSE has already been seen in Code Block 18 — `"John says {myvar}!"` is of type JSE. JSEs are always enclosed within quotes, and may or may not include templating such as `{myvar}`. They must be resolved to strings eventually; if `{myvar}` was a boolean, it would be coerced to a string so that the whole JSE would read `"John says true!"` or `"John says false!"`.

An example of JBE_EXP is below:

```

---
- vars:
  safe_mode: true
  use_cloud: true

- tasks:
  - delete_folder:
    backup_in_cloud: safe_mode and cloud_backup

```

Code Block 19

`reg1 is defined` and `reg1 is succeeded` is a JBE_EXP. JBE_EXPs are not quoted, but are differentiated from UVRs because they contain more than one element.

We will use a conventional symbol table approach for variable resolution. When we reach a JS, we must be able to resolve it using the variables already gathered thus far.

This thesis does not consider dynamic variables (i.e variables determined at runtime), so after this variable resolution step, there should be no JS left in the code anywhere.

7.5.3.1 Note on Type Constraints

The resolved variables must also fit the constraints of the context. For instance, although the type of `file` in `importTasks` is VAR:

```
importTasks {file : VAR}
```


the following code:

```

---
- vars:
  safe_mode: true
  use_cloud: true

- tasks:
  - import_tasks:
    file: safe_mode and use_cloud

```

Code Block 20

will trigger an error during variable resolution analysis.

For constructs which are hardcoded in $\mathcal{A}^H$ such as `import_tasks`, we can check this immediately.

However, for generic modules, we will rely on a different check that is done by a templating system. This will be explained further in Section 8.1.

7.5.3.2 Utility Functions

These utility functions are very verbose, but the vast majority are simply propagating variables down the hierarchy (from plays to tasks to modules etc) while collecting new ones in a conventional symbol-table style. The subscript, usually $\bar{v}$, denotes the collected variable mappings thus far in each function. Only when we reach `resolveVar` to we look in the mapping and resolve each variable type in its own way.

$$\begin{aligned}
 \text{resolvePlay}(p) &= p \oplus \{ \\
 &\quad \text{tasks} : \text{map}(\text{resolveTask}_{p.\text{vars}}, (p.\text{tasks})), \\
 &\quad \text{handlers} : \text{map}(\text{resolveTask}_{p.\text{vars}}, (p.\text{handlers})) \\
 &\quad \} \\
 \text{resolveTask}_{\bar{v}}(t) &= \begin{cases} t \oplus \{ \\ \quad \text{attSet} : \text{resolveAAS}_{\bar{v}'}(t.\text{attSet}) \\ \quad \text{modDecl} : \text{resolveModDecl}_{\bar{v}'}(t.\text{modDecl}) \\ \quad \} & \text{if } t \text{ is Atomic} \\ t \oplus \{ \\ \quad \text{attSet} : \text{resolveBAS}_{\bar{v}'}(t.\text{attSet}) \\ \quad \text{block} : \text{resolveBlock}_{\bar{v}'}(t.\text{block}) \\ \quad \} & \text{if } t \text{ is Blocktask} \end{cases} \\
 \text{where } \bar{v}' &= \bar{v} \cup t.\text{attSet}.\text{vars}
 \end{aligned}$$

$$\begin{aligned}
 \text{resolveAAS}_{\bar{v}}(a) &= a \oplus \{ \\
 &\quad \text{notify} : \text{resolveVar}_{\bar{v}}(a.\text{notify}), \\
 &\quad \text{loop} : \text{resolveLoop}_{\bar{v}}(a.\text{loop}), \\
 &\quad \text{listen} : \text{resolveVar}_{\bar{v}}(a.\text{listen}), \}
 \end{aligned}$$

$$\begin{aligned} \text{resolveLoop}_{\bar{v}}(l) = l \oplus \{ \\ & \text{loopList} : \text{map}(\text{resolveVar}_{\bar{v}}, l.\text{loopList}), \\ & \text{loopVar} : \text{resolveVar}_{\bar{v}}(l.\text{loopVar}), \\ & \text{indexVar} : \text{resolveVar}_{\bar{v}}(l.\text{indexVar}), \\ & \} \end{aligned}$$

$$\begin{aligned} \text{resolveBAS}_{\bar{v}}(b) = b \oplus \{ \\ & \text{notify} : \text{resolveVar}_{\bar{v}}(b.\text{notify}), \\ & \} \end{aligned}$$

$$\begin{aligned} \text{resolveBlock}_{\bar{v}}(b) = b \oplus \{ \\ & \text{blockMain} : \text{map}(\text{resolveTask}_{\bar{v}}, b.\text{blockMain}), \\ & \text{rescue} : \text{map}(\text{resolveTask}_{\bar{v}}, b.\text{rescue}), \\ & \text{always} : \text{map}(\text{resolveTask}_{\bar{v}}, b.\text{always}), \\ & \} \end{aligned}$$

$$\text{resolveModDecl}_{\bar{v}}(m) = \begin{cases} m \oplus \{ \\ \quad \text{params} : \text{mapVals}(\text{resolveVar}_{\bar{v}}, m.\text{params}), \\ \} & \text{if } m \text{ is } \text{GenericModDecl} \\ \\ m \oplus \{ \\ \quad \text{name} : \text{resolveVar}_{\bar{v}}(m.\text{name}), \\ \quad \text{tasksFrom} : \text{resolveVar}_{\bar{v}}(m.\text{tasksFrom}), \\ \quad \text{handlersFrom} : \text{resolveVar}_{\bar{v}}(m.\text{handlersFrom}), \\ \} & \text{if } m \text{ is } \text{ImportRole} \\ \\ m \oplus \{ \\ \quad \text{file} : \text{resolveVar}_{\bar{v}}(m.\text{file}), \\ \} & \text{if } m \text{ is } \text{ImportTasks} \end{cases}$$

$$\text{resolveVar}_{\bar{v}}(v) = \begin{cases} \text{concatMap}(\text{resolveJSE}_{\bar{v}}, (v)) & \text{if } v \text{ of form } \overline{\text{JSE}} \\ \text{resolveJBE_EXP}_{\bar{v}}(v) & \text{if } v \text{ of form } \text{JVE_EXP} \\ \text{resolveUVR}_{\bar{v}}(v) & \text{if } v \text{ of form } \text{UVR} \\ v & \text{otherwise} \end{cases}$$

$$\text{resolveJSE}_{\bar{v}}(j) = \begin{cases} s & \text{if } j = s \\ \text{toString}(x) & \text{if } j = \text{JseUvr } s \wedge \bar{v}[s] = x \end{cases}$$

$$\text{resolveJBE}_{\bar{v}}(j) = \begin{cases} \text{resolveJBE}_{\bar{v}}(\text{left}) \text{ op } \text{resolveJBE}_{\bar{v}}(\text{right}) & \text{if } j \text{ of form } \text{left op right} \\ \neg \text{resolveJBE}_{\bar{v}}(e) & \text{if } j \text{ of form } \neg e \\ \text{resolveJBE}_{\bar{v}}(e) & \text{if } j \text{ of form } (e) \\ j & \text{if } j \text{ is plain boolean} \\ b & \text{if } j \text{ is plain string } \wedge \bar{v}[j] = b \end{cases}$$

$$\text{resolveUVR}_{\bar{v}}(v) = x \text{ if } \bar{v}[v] = x$$

7.5.3.3 Rewrite Rule

$$\frac{\exists \text{ VAR at some level within } p}{p \rightarrow \text{resolvePlay}(p)}$$

7.5.4 Desugaring Blocks

Recall that blocks in Ansible are a method of neatly structuring control flow related to error handling. `block`, `rescue` and `always` are similar to `try`, `catch` and `finally` respectively in many common languages.

A block is a special module that can be used by a task. It can have up to three components, all of which are simply lists of tasks:

- The main section, under the `block` keyword. This is like `try` in many languages. It is compulsory.
 - Note that this will be called the `blockMain` section in this thesis. This is not official Ansible terminology, but is useful.
- The `rescue` section. This is like `catch` in many languages. It is optional.
- The `always` section. This is like `finally` in many languages. It is optional.

An example is:

```
- name: a blocktask
  block:
    - name: backup folder
      backup_folder:
        folder_to_backup: /home/remotemachine/myfolder
    - name: attempt to delete folder
      delete_folder:
        folder_to_delete: /home/remotemachine/myfolder
  rescue:
    - name: recover backup if exists
      recover_backup:
        folder_to_recover: /home/remotemachine/myfolder
  always:
    - name: generate log file
      log:
```

Code Block 21

Note that the above is technically one task, named “a blocktask”. This task contains a block module, which in turn contains 4 tasks spread among the `blockMain`, `rescue` and `always` sections.

In this thesis, we will refer to a task containing the block module as a *blocktask*, and a task which does not as an *atomic task*.

Blocktasks have some restrictions on the attributes they can have compared to atomic tasks; for instance, blocktasks cannot have loops (although their constituent atomic tasks can).

In order to facilitate desugaring, we would like to use more primitive Ansible constructs to represent this control flow. Ansible has two that prove useful. Both are parts of the attribute set of each task:

- `ignore_errors`: When set to `true`, even if the task fails, the Ansible Playbook will continue executing instead of halting.
- `when`: When evaluation of the playbook reaches the task, the task will only be executed if the value of its `when` attribute evaluates to `true`.

If we set the `ignore_errors` attribute of every task to `true`, we can then explicitly set a condition for every single task to run using `when`. For instance, in a simple play:

```

---
- tasks:
  - debug:
      msg: "first task"

  - debug:
      msg: "second task"

  - debug:
      msg: "third task"

```

Code Block 22

if the first task fails, the second task will not run. This is simply due to the default behavior of Ansible; if any task fails whose `ignore_error` attribute is set to `false` (which is the default if not explicitly set), the entire playbook ceases execution. In other words:

- For the second task to run, the first task must have succeeded.
- For the third task to run, the first and second tasks must have succeeded.

Since the second task succeeding already implies that the first task succeeded, we can further simplify:

- For the second task to run, the first task must have succeeded.
- For the third task to run, the second task must have succeeded.

Recall that whether a task has succeeded is stored in its `register`. Also, before checking if a task has succeeded, we should also check if it has run in the first place (or else we will get an error for trying to check a nonexistent value). We may do this by checking if that task's `register` variable is defined.

We can therefore rewrite Code Block 22 as:

```

---
- tasks:
  - debug:
      msg: "first task"
      ignore_errors: true
      register: reg1
      when: true

  - debug:
      msg: "second task"
      ignore_errors: true
      register: reg2
      when: reg1 is defined and reg1 is succeeded

  - debug:
      msg: "third task"
      ignore_errors: true
      register: reg3
      when: reg2 is defined and reg2 is succeeded

```

Code Block 23

We can draw this out as:



Figure 5

Note that the rewriting of Code Block 22 to Code Block 23 comes with a caveat that is important to our idempotence analysis. In BBFS, we only have one `err` construct, which represents an unhandled error; i.e. one which would normally cause Ansible to immediately stop execution and report the script as a whole to have failed.

If we set `ignore_errors` to `true` for all tasks, we do not know which errors are caught and handled, and therefore should not be represented by `err` in BBFS, and which errors should. What our rewriting *does* preserve correctly is *which tasks end up being run* by the script. In Code Block 23, if the first task fails, Ansible would not treat the entire script as having failed, as it would in Code Block 22; instead, it would move on and try to run the second task, realize the run condition is not met, try to run the third task, realize the run condition is not met, and cease execution considering the script as a whole to have been executed successfully.

We shall ignore this caveat for now and address it later in this same section. For now, we only concern ourselves with ensuring that the correct tasks end up being run (i.e. the control flow is correct).

To further our rewriting idea, let us examine a play with a list of tasks; let us call it `blockrescueplay`:

```

---
- tasks:
  - debug:
    msg: "first task before block"
  - debug:
    msg: "second task before block"
  - debug:
    msg: "third task before block"
  - block:
    - debug:
      msg: "first task in block main part"
    - debug:
      msg: "second task in block main part"
    - debug:
      msg: "third task in block main part"
  rescue:
    - debug:
      msg: "first task in rescue"
    - debug:
      msg: "second task in rescue"
    - debug:
      msg: "third task in rescue"
  - debug:
    msg: "first task after block"
  - debug:
    msg: "second task after block"
  - debug:
    msg: "third task after block"
  
```

Code Block 24

Our dependency structure is now more complex than the previous simple chain. Tasks in `rescue` will only activate if any task in `blockMain` fails.

We might make this more concise. The first task in `rescue` will only activate if either:

- the first task of `blockMain` fails, *or*
- the second task of `blockMain` fails, *or*
- the third task of `blockMain` fails.

The rest of the tasks in `rescue` simply chain off of the first task in `rescue`.

In addition, the first task *after* the block will only run if either:

- the last task of `blockMain` succeeds (implying that the whole `blockMain` is successful), *or*
- the last task of `rescue` succeeds (implying that `blockMain` failed somewhere, but the whole `rescue` is successful)

Note that technically speaking, `blockrescueplay` has only 7 tasks on the top level: 3 atomic, then 1 containing a blocktask, then 3 atomic. The blocktask then itself contains 6 atomic tasks; 3 in the main part, and 3 in `rescue`.

Nevertheless, it is illustrative to only draw out the atomic tasks as if they were all raised to the top level, and then draw the conditions for each task to be executed.

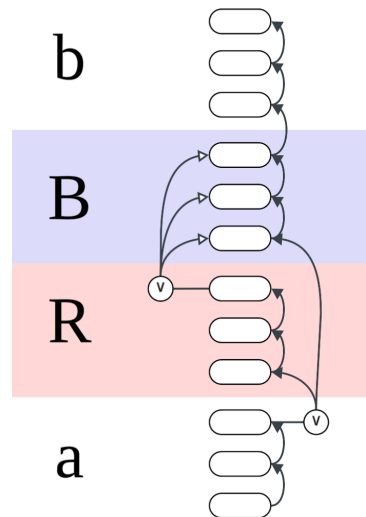


Figure 6

In Figure 6 above, we use:

- lowercase *b* to indicate tasks before the blocktask
- uppercase *B* and a blue zone to indicate the `blockMain` section of the blocktask
- uppercase *R* and a red zone to indicate the `rescue` section of the blocktask
- lowercase *a* to indicate tasks after the blocktask

We have also expressed the *or* conditions using circles labelled with $\vee$ that unify arrows. Also, solid-head arrows signify success dependencies, while hollow-head arrows signify failure dependencies.

This corresponds to:

```

---
- tasks:
- debug:
  msg: "first task before block"
  ignore_errors: true
  register: reg1
  when: true

- debug:
  msg: "second task before block"
  ignore_errors: true
  register: reg2
  when: reg1 is defined and reg1 is succeeded

- debug:
  msg: "third task before block"
  ignore_errors: true
  register: reg3
  when: reg2 is defined and reg2 is succeeded

- debug:
  msg: "first task in block main part"
  ignore_errors: true
  register: reg4
  when: reg3 is defined and reg3 is succeeded

- debug:
  msg: "second task in block main part"
  ignore_errors: true
  register: reg5
  when: reg4 is defined and reg4 is succeeded

- debug:
  msg: "third task in block main part"
  ignore_errors: true
  register: reg6
  when: reg5 is defined and reg5 is succeeded

- debug:
  msg: "first task in rescue"
  ignore_errors: true
  register: reg7
  when:
    (reg4 is defined and reg4 is failed) or
    (reg5 is defined and reg5 is failed) or
    (reg6 is defined and reg6 is failed)

- debug:
  msg: "second task in rescue"
  ignore_errors: true
  register: reg8
  when: reg7 is defined and reg7 is succeeded

- debug:
  msg: "third task in rescue"
  ignore_errors: true
  register: reg9
  when: reg8 is defined and reg8 is succeeded

- debug:
  msg: "first task after block"
  ignore_errors: true
  register: reg10
  when:
    (reg6 is defined and reg6 is succeeded) or
    (reg9 is defined and reg9 is succeeded)

- debug:
  msg: "second task after block"
  ignore_errors: true
  register: reg11
  when: reg10 is defined and reg10 is succeeded

- debug:
  msg: "third task after block"
  ignore_errors: true
  register: reg12
  when: reg11 is defined and reg11 is succeeded

```

Code Block 25

Next, let us observe a use of the `always` section:

```
---
- tasks:
  - debug:
    msg: "first task before block"
  - debug:
    msg: "second task before block"
  - debug:
    msg: "third task before block"
  - block:
    - debug:
      msg: "first task in block main part"
    - debug:
      msg: "second task in block main part"
    - debug:
      msg: "third task in block main part"
  always:
    - debug:
      msg: "first task in always"
    - debug:
      msg: "second task in always"
    - debug:
      msg: "third task in always"
  - debug:
    msg: "first task after block"
  - debug:
    msg: "second task after block"
  - debug:
    msg: "third task after block"
```

Code Block 26

Here, whether the `always` section runs does not depend on `blockMain` at all; as long as the last task *before* the `blocktask` runs, the `always` section will run. This is just like a `finally` block in other languages.

However, recall that in Ansible, if a task fails (and the entire `blocktask` is considered a single task), nothing after it will run. In order to determine whether the `blocktask` as a whole has succeeded, both the `blockMain` and the `always` sections must succeed. This is despite the fact that even if the `blockMain` section fails, the `always` section will still run (and can be used, for example, for graceful cleanup tasks).

Drawing this out:

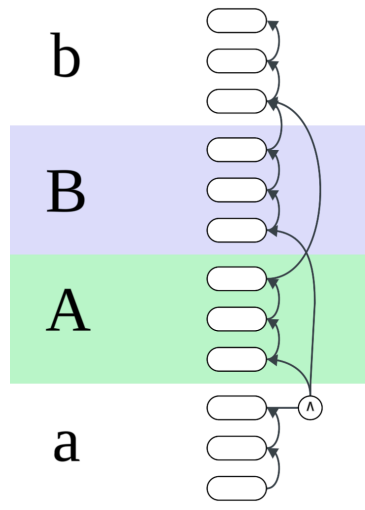


Figure 7

Note that we have used a circle labelled with $\wedge$ to indicate the *and* success dependency. The green zone, labelled with uppercase *A*, represents the *always* section of the blocktask.

If we observe a block with `blockMain`, `rescue` and `always`, the graph then looks like this:

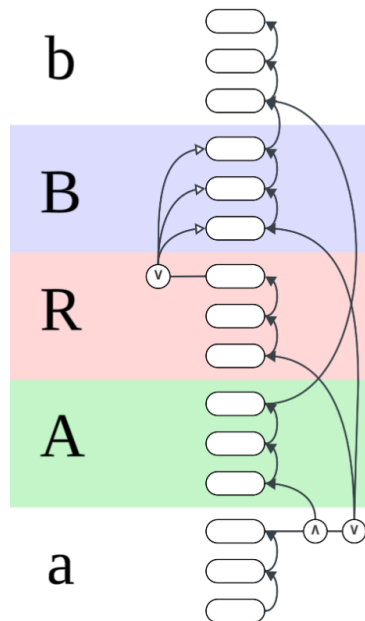


Figure 8

We would like to establish some more general rules for blocktasks for encapsulation purposes. This will help us deal with nested blocktasks.

Which rules should we seek to establish? Let us examine what we can do with an atomic task that helps us draw the dependency arrows above:

- We can check if a task was successful
 - This is required for the head end of the arrow
- We can set the when condition of a task
 - This is required for the tail end of the arrow

Let us examine how we might do this with blocktasks. We first note that there are a limited number of varieties of blocktasks:

- containing `blockMain` section only; let us call these B blocktasks.
- containing `blockMain` and `rescue` sections; let us call these BR blocktasks.
- containing `blockMain` and `always` sections; let us call these BA blocktasks.
- containing `blockMain`, `rescue` and `always` sections; let us call these BRA blocktasks.

When is a blocktask as a whole considered successful? From experimentation with Ansible:

- B: when `blockMain` is successful
- BR: when either `blockMain` *or* `rescue` are successful
- BA: when `always` *and* `blockMain` are successful
- BRA: when `always` *and* (either `blockMain` *or* `rescue`) are successful

Recall that when we say “`blockMain` is successful”, we can equivalently say “the last task of `blockMain` is successful”, since the last task in `blockMain` will only run if all the previous tasks in `blockMain` were successful.

Moving on, how can we set the `when` condition of a blocktask? For this, we can look at the arrows we have already drawn, and find out which ones we needed to draw from tasks within the blocktask, to the task *before* the blocktask. This will tell us which sub-tasks within the blocktask we need to change the `when` attribute for.

- B: first task of `blockMain`
- BR: first task of `blockMain`
- BA: first task of `blockMain` and first task of `always`
- BRA: first task of `blockMain` and first task of `always`.

Let us try to chain two BRA blocktasks together; that is, we would simply like to express



Figure 9

where each of the above 2 tasks is actually a BRA blocktask.

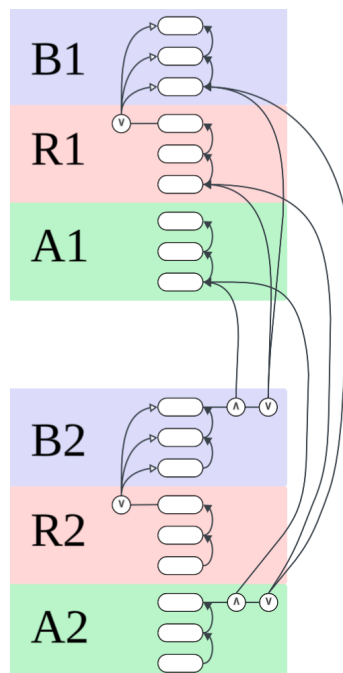


Figure 10

We can also nest blocktasks now. Let us try to express the following:

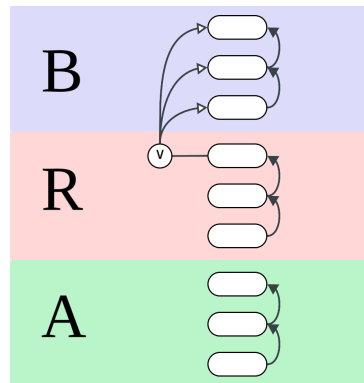


Figure 11

but where the first task of rescue is actually another blocktask.

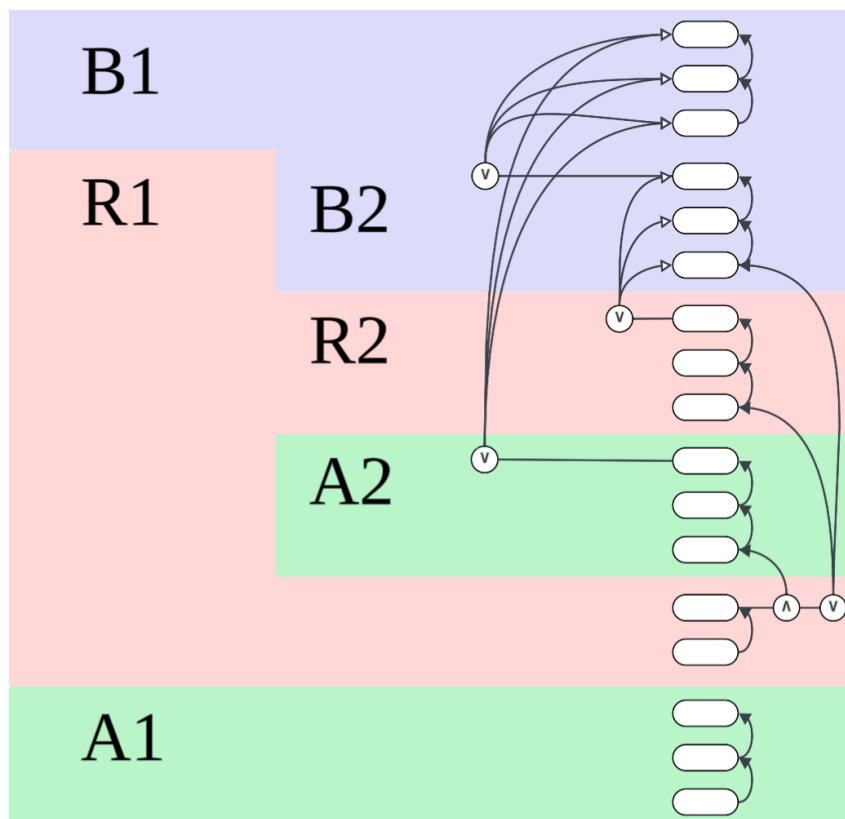


Figure 12

Having done all this, we now return to the caveat stated at the beginning: We cannot yet distinguish between handled and unhandled errors.

Firstly, we can return all tasks which are outside of any blocktask to `ignore_errors: false`. We do not allow the user to specify `ignore_errors` manually (it is not within $\mathcal{A}^H$), and in Ansible, all tasks have `ignore_errors: false` by default. We can simply retain this behavior.

For blocktasks, we will keep `ignore_errors` set to `true` for all tasks in `blockMain`, `rescue` and `always`. However, we will add an additional task to the end of the blocktask which has `ignore_error: false` and `when: true` by default. This special task represents the failure state of the blocktask as a whole; it will call a special module that takes in all the other tasks' statuses, and error out on behalf of the whole block if it detects that it should. We will call this the *goalkeeper* task.

Until now, after every blocktask, the very next task has had arrows that determine whether it should run by calculating whether the preceding blocktask succeeded as a whole. For instance, in Figure 8, the first task of section *a* only runs if the last *A* task and either the last *B* or the last *R* task are successful.

This calculation will now be done internally by the goalkeeper of each blocktask, and if it determines that the blocktask has failed as a whole, the goalkeeper will error out normally.

Let us return to Figure 6 with these changes made:

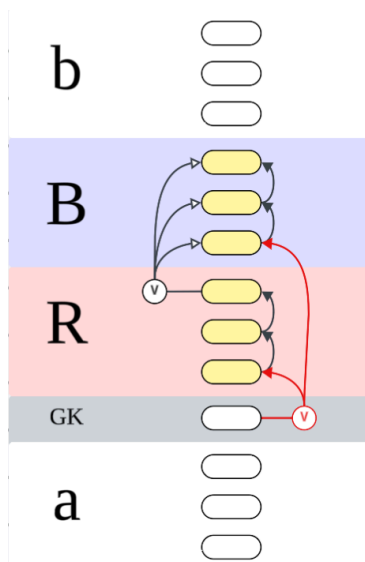


Figure 13

Now we must keep certain things in mind to interpret this diagram.

Firstly, recall that Ansible tries to execute each task in turn from top to bottom. When it is a task's turn, it decides whether or not it should be executed based on whether its when condition is fulfilled or not.

In our diagrams, if a task is contacted by the *tail* of an arrow, it means that that task's when condition depends on the success/failure of the task at the *head* of that same arrow. Conversely, any task that is *not* contacted by the *tail* of any arrow has when: `true`; i.e. it will always attempt to run when execution reaches it. This means for instance that the goalkeeper will always attempt to run if execution reaches it.

Next, we have used yellow to indicate tasks `ignore_errors: true`. If a task with `ignore_errors: true` fails, execution continues on the the next module down. If a task with `ignore_errors: false` fails, execution halts and the script as a whole is deemed to have failed.

Finally, GK and a grey zone are used to indicate the goalkeeper. The goalkeeper in Figure 13 does the same calculation as the first task in the *a* section in Figure 6, but instead of the calculation deciding if the goalkeeper will run or not, it decides if the goalkeeper will succeed or fail. This is indicated by the red arrows.

Overall, we can provide equivalent “methods” for the blocktasks so we can encapsulate them and treat them like atomic tasks:

Atomic Task	B Blocktask	BR Blocktask	BA Blocktask	BRA Blocktask
set when	set when of:	set when of:	set when of:	set when of:

	• first blockMain task • goalkeeper	• first blockMain task • goalkeeper	• first blockMain task • first always task • goalkeeper	• first blockMain task • first always task • goalkeeper
check if succeeded/failed	check if goalkeeper succeeded/failed	check if goalkeeper succeeded/failed	check if goalkeeper succeeded/failed	check if goalkeeper succeeded/failed
set ignore_errors	set ignore_errors of goalkeeper	set ignore_errors of goalkeeper	set ignore_errors of goalkeeper	set ignore_errors of goalkeeper

Let us see this in action. Take this task:

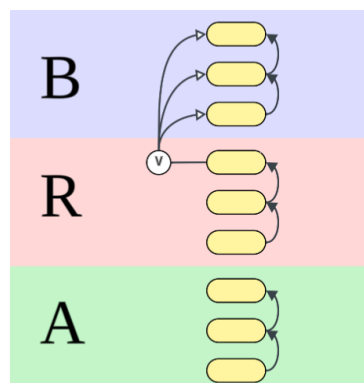


Figure 14

Let us replace the first task of rescue with another BRA blocktask.

The first task of rescue:

- has when set to *or* together the failure conditions of each blockMain task
- has ignore_errors: true
- is being checked for success by the second rescue task

To replace it with a nested blocktask, we:

- set the when of the following nested tasks to *or* together the failure conditions of each outer blockMain task:
 - first nested blockMain task
 - first nested always task
 - nested goalkeeper
- set ignore_errors: true for the nested goalkeeper
- the second outer rescue task should check the nested goalkeeper for success

We arrive at the diagram:

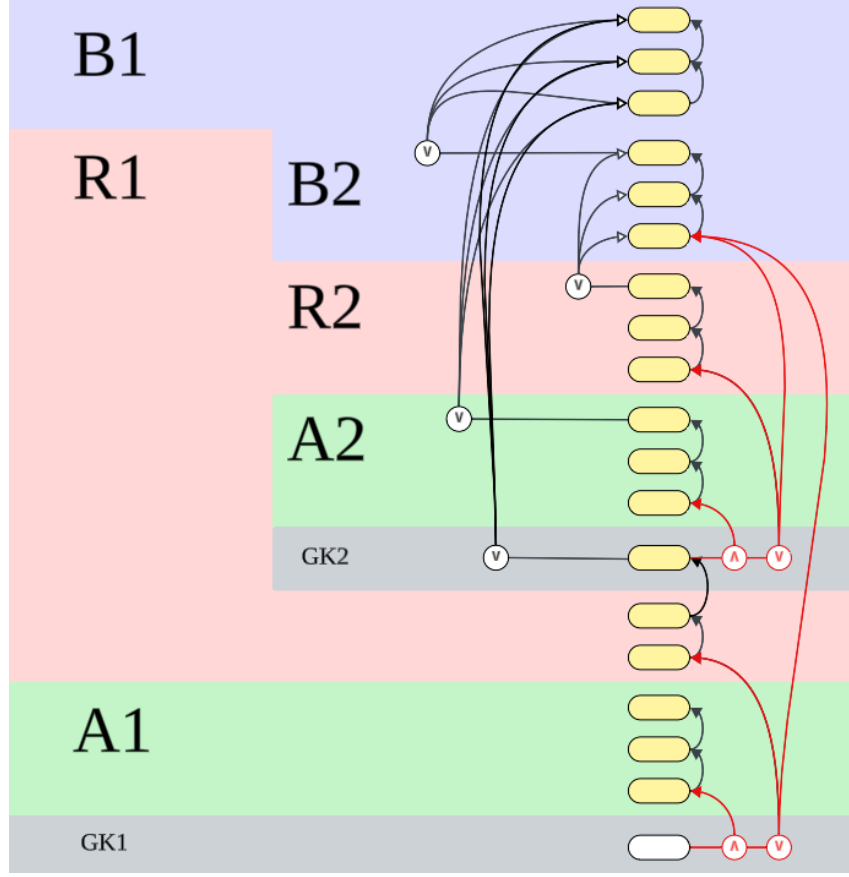


Figure 15

7.5.4.1 Utility Functions

We have some functions which serve to draw the solid (success) and hollow (failure) arrows by updating the when attributes of a task:

$$\text{drawSuccessArrow}(t_1, t_2) = \text{updateWhen}(t_2, \lambda w : w \wedge \text{getSuccessIndicator}(t_1))$$

$$\text{drawFailureArrows}(\bar{t}, t) = \text{updateWhen}(t, \lambda w : w \wedge \bigvee \text{map}(\text{getFailureIndicator}, \bar{t}))$$

These functions return the appropriate when attribute to indicate if a specific task has succeeded or failed:

$$\text{getSuccessIndicator}(t) = \begin{cases} t.\text{register is defined} \wedge t.\text{register is succeeded} & \text{if } t \text{ is Atomic} \\ \text{getSuccessIndicator}(t.\text{goalkeeper}) & \text{if } t \text{ is Blocktask} \end{cases}$$

$$\text{getFailureIndicator}(t) = \begin{cases} t.\text{register is defined} \wedge t.\text{register is failed} & \text{if } t \text{ is Atomic} \\ \text{getFailureIndicator}(t.\text{goalkeeper}) & \text{if } t \text{ is Blocktask} \end{cases}$$

And this function handles the updating of when itself. It recurses appropriately if the task it is trying to update is actually a blocktask.

$$\text{updateWhen}(t, f) = \begin{cases} t \oplus \{ \\ \quad \text{attSet} : t.\text{attSet} \oplus \{\text{when} : f(t.\text{attSet}.\text{when})\} \\ \} & \text{if } t \text{ is Atomic} \\ \\ t \oplus \{ \\ \quad \text{blockMain} : \text{updateWhen}(t.\text{blockMain}[0], f) \# t.\text{blockMain}[1:], \\ \quad \text{always} : \begin{cases} \text{updateWhen}(t.\text{always}[0], f) \# t.\text{always}[1:] & \text{if } b.\text{always} \neq \varepsilon \\ \varepsilon & \text{otherwise} \end{cases}, \\ \quad \text{goalkeeper} : \text{updateWhen}(t.\text{goalkeeper}, f), \\ \} & \text{if } t \text{ is Blocktask} \end{cases}$$

Putting these together, we can draw all the arrows within a blocktask:

$$\text{drawArrowsInBlock}(b) = b \oplus \left\{ \begin{array}{l} \text{blockMain} : \text{chainTogether}(b.\text{blockMain}), \\ \text{rescue} : \begin{cases} \text{chainTogether}(\text{newRescue}) & \text{if } b.\text{rescue} \neq \varepsilon \\ \varepsilon & \text{otherwise} \end{cases}, \\ \text{always} : \begin{cases} \text{chainTogether}(b.\text{always}) & \text{if } b.\text{always} \neq \varepsilon \\ \varepsilon & \text{otherwise} \end{cases}, \end{array} \right\}$$

where $\text{newRescue} = \text{drawFailureArrows}(b.\text{blockMain}, b.\text{rescue}[0]) \# b.\text{rescue}[1:]$

$\text{chainTogether}(\bar{t}) = \bar{t}[0] \# \text{zipWith}(\text{drawSuccessArrow}, \bar{t}, \bar{t}[1:])$

We have the function that creates a goalkeeper and adds it to a blocktask:

$$\text{addGoalkeeper}(b) = b \oplus \{ \text{goalkeeper} : \text{Atomic} \{ \\ \quad \text{attSet} : \{\}, \\ \quad \text{modDecl} : \text{GenericModDecl} \{ \text{name} : _gk, \text{params} : \text{createGoalkeeperParams}(b) \} \\ \} \}$$

$$\text{createGoalkeeperParams}(b) = \left[\begin{array}{l} \langle \text{mode} \rightarrow \begin{cases} \text{"B"} & \text{if } b.\text{rescue} = \varepsilon \wedge b.\text{always} = \varepsilon \\ \text{"BR"} & \text{if } b.\text{rescue} \neq \varepsilon \wedge b.\text{always} = \varepsilon \\ \text{"BA"} & \text{if } b.\text{rescue} = \varepsilon \wedge b.\text{always} \neq \varepsilon \\ \text{"BRA"} & \text{if } b.\text{rescue} \neq \varepsilon \wedge b.\text{always} \neq \varepsilon \end{cases} \rangle, \\ \langle b \rightarrow \text{getSuccessPath}(b.\text{blockMain}[-1]) \rangle, \\ \langle r \rightarrow \begin{cases} \text{getSuccessPath}(b.\text{rescue}[-1]) & \text{if } b.\text{rescue} \neq \varepsilon \\ \text{dummyPath} & \text{otherwise} \end{cases} \rangle, \\ \langle a \rightarrow \begin{cases} \text{getSuccessPath}(b.\text{always}[-1]) & \text{if } b.\text{always} \neq \varepsilon \\ \text{dummyPath} & \text{otherwise} \end{cases} \rangle \end{array} \right]$$

Like with `ImportTasks` and `ImportRole`, we also need to make sure any attributes of a block get passed down to its constituent tasks:

$$\text{propagateBAS}_a(t) = \begin{cases} t \oplus \{\text{attSet} : a'\} & \text{if } t \text{ is Atomic} \\ t \oplus \begin{cases} \text{blockMain} : \text{map}(\text{propagateBAS}_{a'}, t.\text{blockMain}) \\ \text{rescue} : \text{map}(\text{propagateBAS}_{a'}, t.\text{rescue}) \\ \text{always} : \text{map}(\text{propagateBAS}_{a'}, t.\text{always}) \end{cases} & \text{if } t \text{ is Blocktask} \end{cases}$$

where $a' = t.\text{attSet} \oplus \{$
 $\text{notify} : t.\text{attSet}.\text{notify} \cup a.\text{notify}$
 $\text{listen} : t.\text{attSet}.\text{listen} \cup a.\text{listen}$
 $\text{vars} : t.\text{attSet}.\text{vars} \cup a.\text{vars}$
 $\}$

Next, we must also remember to set `ignore_errors: true`:

$$\text{setIgnoreError}(t) = \begin{cases} t \oplus \{\text{attSet} : t.\text{attSet} \oplus \{\text{ignoreError} : \text{true}\}\} & \text{if } t \text{ is Atomic} \\ t \oplus \{\text{goalkeeper} : \text{setIgnoreError}(t.\text{goalkeeper})\} & \text{if } t \text{ is Blocktask} \end{cases}$$

And lastly, we turn a blocktask into a list of its constituent tasks, and recurse if the blocktask is has further nested blocktasks.

$$\text{flattenBlock}(t) = \begin{cases} t & \text{if } t \text{ is Atomic} \\ \begin{aligned} &\text{map}(\text{flattenBlock}, t.\text{blockMain}) \# \\ &\text{map}(\text{flattenBlock}, t.\text{rescue}) \# \\ &\text{map}(\text{flattenBlock}, t.\text{always}) \# \\ &t.\text{goalkeeper} \end{aligned} & \text{if } t \text{ is Blocktask} \end{cases}$$

In this way we arrive at a function we can apply to a blocktask:

$$\begin{aligned} \text{prepareBlock}(b) = (& \\ &\text{flattenBlock} \\ &\circ \text{setIgnoreError} \\ &\circ \text{drawArrowsInBlock} \\ &\circ \text{propagateBAS}_\epsilon \\ &\circ \text{addGoalkeeper} \\ &)(b) \end{aligned}$$

In our rewrite rule, we map this onto every task, atomic or block, so we write a function `prepareIfBlock` that ignores atomic tasks, and applies `prepareBlock` if it is a blocktask. We always convert it to a list, so that in the end we can concat all the lists together to form one big flattened list of atomic tasks.

$$\text{prepareIfBlock}(t) = \begin{cases} [t] & \text{if } t \text{ is Atomic} \\ \text{prepareBlock}(t) & \text{if } t \text{ is Blocktask} \end{cases}$$

7.5.4.2 Rewrite Rule

$$\frac{\begin{array}{l} \exists t \in \bar{t} : t \text{ is Blocktask} \vee \exists h \in \bar{h} : h \text{ is Blocktask} \\ \bar{t}' = \text{concatMap}(\text{prepareIfBlock}, \bar{t}) \\ \bar{h}' = \text{concatMap}(\text{prepareIfBlock}, \bar{h}) \end{array}}{\text{Play } \{\text{tasks} : \bar{t}, \text{handlers} : \bar{h}\} \rightarrow \text{Play } \{\text{tasks} : \bar{t}', \text{handlers} : \bar{h}'\}}$$

7.5.5 Desugaring Loops

Let us examine loops in Ansible.

```
- name: Attempt to delete a file 3 times.
  ansible.builtin.file:
    path: /home/targetmachine/my_file.py
    state: absent
  loop:
  - 1
  - 2
  - 3
```

Code Block 27

This will call the `ansible.builtin.file` module 3 times with the same parameters. Luckily for us, because this module was written to be idempotent, it does not error out. It will check if the file `/home/targetmachine/my_file.py` exists, and if so, will delete it; else, it will do nothing. The exact elements of the loop in this case are not used and do not matter; they could just as easily have been:

```
- name: Attempt to delete a file 3 times.
  ansible.builtin.file:
    path: /home/targetmachine/my_file.py
    state: absent
  loop:
  - 1
  - "never gonna give you up"
  - true
```

Code Block 28

since the only way the loop is used here is to repeat the module call. In order to use the loop elements, Ansible uses built-in Jinja templating:

```
- name: Delete my_file1.py, my_file2.py and my_file3.py.
  ansible.builtin.file:
    path: /home/targetmachine/my_file{item}.py
    state: absent
  loop:
  - 1
  - 2
  - 3
```

Code Block 29

This task will attempt to delete 3 files:

- `/home/targetmachine/my_file1.py`
- `/home/targetmachine/my_file2.py`
- `/home/targetmachine/my_file3.py`

The variable used to signal a template to the Jinja subsystem for a loop is `item` by default, but can be customized by stating a `loop_var` attribute. Ansible puts such loop-related attributes in a bigger attribute called `loop_control`:

```
- name: Delete my_file1.py, my_file2.py and my_file3.py.
  ansible.builtin.file:
    path: /home/targetmachine/my_file{my_file_number}.py
    state: absent
  loop:
  - 1
  - 2
  - 3
  loop_control:
    loop_var: my_file_number
```

Code Block 30

This task will run exactly like before.

`index_var` is another attribute under `loop_control`, which provides the user with a way to access the (zero-indexed) iteration number.

```
- name: Delete my_file1.py, my_file2.py and my_file3.py.
  ansible.builtin.file:
    path: /home/targetmachine/my_file{my_file_number}.py
    state: absent
  loop:
  - "blah"
  - "blah"
  - "blah"
  loop_control:
    index_var: my_file_number
```

Code Block 31

This task will also run like before.

Note that an Ansible task cannot have nested loops.

Since we do not consider dynamic variables in Ansible, loops are simply unrolled. The main complicating factor is that in Ansible, if any tasks within a loop fail, the loop as a whole is considered to have failed.

The loop still attempts to run all iterations; if the a task has 5 iterations, and the 2nd one fails, it will still try to run the remaining iterations. However, after doing so, the next task after the looptask will not run since the looptask as a whole has failed.

For instance, the following:

```
- debug:
  msg: "Line {idx}: Never gonna {item}"
  loop:
  - "give you up"
  - "let you down"
  - "run around and desert you"
  loop_control:
    loop_var: item
    index_var: idx
```

Code Block 32

Is unrolled into:

```

- debug:
  msg: "Line 0: Never gonna give you up"
  ignore_errors: true
  register: reg1
- debug:
  msg: "Line 1: Never gonna let you down"
  ignore_errors: true
  register: reg2
- debug:
  msg: "Line 2: Never gonna run around and desert you"
  ignore_errors: true
  register: reg3
- _gk: # goalkeeper
  t1: reg1
  t2: reg2
  t3: reg3
  ignore_errors: false
  register: reg4

```

Code Block 33

This is very similar to the block concept; we therefore take a similar approach. We:

- Unroll the loop into multiple non-loop tasks
- Set `ignore_errors: true` on all unrolled tasks
- Create a goalkeeper module at the end.

Unlike block goalkeepers, which only need to deal with B, BR, BA and BRA blocks, the parameters on a loop module are not finitely enumerable. Later in Section 8.1, we will learn that given the limitations of BBFS, we must in fact generate unique modules for each length of loop encountered in the Ansible script. Luckily, this dynamic generation, though verbose, is not very complex. We will cover this in Section 8.1.

When unrolling, we resolve loop and index variables the same way we do any static variables. In fact, we can reuse `resolveTask` from Section 7.5.3.2.

As a minor note, although index variables are numbers in Ansible, in this thesis we will treat index variables as strings (i.e. “1”, “2”, “3” instead of 1,2,3) because $\mathcal{A}^H$ does not support numbers.

Lastly, there might have been tasks in the play that were previously pointing at the looptask. It is not a complex operation to correct those tasks to now point at the goalkeeper, but this is relatively tedious. Instead, if we simply make our UID system (which is not heavily covered in the formalization) such that the goalkeeper retains the register UID of the original looptask, and the subtasks change UID, then we do not have to do anything.

7.5.5.1 Rewrite Rule

$$\begin{aligned}
 & t.\text{attSet}.\text{loop} \neq \varepsilon \\
 \overline{v_i} &= \{ \langle t.\text{loop}.\text{loopVar} \rightarrow t.\text{loop}.\text{loopList}[i] \rangle, \langle t.\text{loop}.\text{indexVar} \rightarrow \text{toString}(i) \rangle \} \quad \forall i \in \text{length}(t.\text{loop}.\text{loopList}) \\
 t'_i &= \text{resolveTask}_{\overline{v_i}}(t) \\
 t_{\text{gk}} &= \text{createLoopGK}([t'_1, t'_2, \dots]) \\
 \hline
 t &\rightarrow [t'_1, t'_2, \dots, t'_{\text{gk}}]
 \end{aligned}$$

7.5.6 Desugaring Handlers

Recall that handlers are tasks that listen to one or more topics. When any of those topics are notified by a task, the handler will run. For instance,

```

- - -
- tasks:

  - debug:
    msg: "first task"
    notify: topic 2

- handlers:

  - debug:
    msg: "first handler"
    listen: topic 1

  - debug:
    msg: "second handler"
    listen: topic 2

  - debug:
    msg: "third handler"
    listen: topic 2

```

Code Block 34

Here the first task notifies the second and third handlers.

However, the handlers do not run immediately, but only when execution reaches them. Execution in Ansible runs downwards through the list of all tasks in a play, and only then proceeds to the list of handlers.

Furthermore, handlers do not run in the order they are notified. Depending on the origin of the handler, Ansible prescribes an ordering of categories of handlers:

1. Handlers from the `roles` section of a play
2. Handlers from the `handlers` section of a play
3. Handlers from calls to `import_role`

Within each category, handlers run in the order they are defined. So, if we have:

```

- - -
- tasks:

  - debug:
    msg: "first task"
    notify: topic 1

  - debug:
    msg: "second task"
    notify: topic 2

  - debug:
    msg: "third task"
    notify: topic 2

- handlers:

  - debug:
    msg: "first handler"
    listen: topic 2

  - debug:
    msg: "second handler"
    listen: topic 1

  - debug:
    msg: "third handler"
    listen: topic 1

```

Code Block 35

all three handlers are notified — some multiple times — but they will run in the order: First handler, then second, then third; each handler runs just once. In this way, we can see that the handler construct can be expressed using when, and can be merged with tasks as long as the handlers are placed after all tasks.

With regards to the ordering of categories of handlers, at this point, we already have the handlers in the correct order due to how we perform desugaring for inline roles and imports (see Section 7.5.1.3 and Section 7.5.2.2). As such, all we need to do is to express the notify-listen construct using when, and move handlers to the tasks section of plays.

For instance:

```
---  
- tasks:  
  
  - debug:  
    msg: "first task"  
    notify: topic 1  
  
  - debug:  
    msg: "second task"  
    notify: topic 2  
  
  - debug:  
    msg: "third task"  
    notify: topic 2  
  
- handlers:  
  
  - debug:  
    msg: "first handler"  
    listen: topic 2  
  
  - debug:  
    msg: "second handler"  
    listen: topic 1  
  
  - debug:  
    msg: "third handler"  
    listen: topic 1
```

Code Block 36

```

- - -
- tasks:

  - debug:
    msg: "first task"
    register: reg1

  - debug:
    msg: "second task"
    register: reg2

  - debug:
    msg: "third task"
    register: reg3

  - debug:
    msg: "first handler"
    when: reg2 is defined or reg 3 is defined

  - debug:
    msg: "second handler"
    when: reg1 is defined

  - debug:
    msg: "third handler"
    when: reg1 is defined

```

Code Block 37

7.5.6.1 Rewrite Rule

$$\begin{aligned}
\bar{h} &= [h_1, h_2, \dots, h_n], \bar{t} = [t_1, t_2, \dots, t_m] \\
s_{h_i} &= h_i.\text{attSet}.\text{listen} \quad \forall h_i \in \bar{h} \\
s_{t_j} &= t_j.\text{attSet}.\text{notify} \quad \forall t_j \in \bar{t} \\
w_i &= \bigvee_{t_j} (t_j \text{ is defined}) \forall h_i \in \bar{h}, \quad \forall t_j : \exists s \in s_{h_i} : s \in s_{t_j} \\
t_{h_i} &= h_i \oplus \{\text{attSet} : h.\text{attSet} \oplus \{\text{when} : (h_i.\text{when} \wedge w_i)\}\} \forall h_i \in \bar{h} \\
t' &= [t_1, t_2, \dots, t_m, t_{h_1}, t_{h_2}, \dots, t_{h_n}]
\end{aligned}$$

$$\text{Play}\{\text{tasks} : t, \text{handlers} : h\} \rightarrow \text{Play}\{\text{tasks} : t', \text{handlers} : []\}$$

7.5.7 Desugaring Plays and Playbooks

A playbook is nothing more than a YAML file containing a list of *plays*. A play consists of:

- A roles section, containing a list of *names* of roles
- A tasks section, containing a list of tasks
- A handlers section, containing a list of handlers

All three sections are optional; technically, the only compulsory part of a play is an attribute called *hosts*.

What is *hosts*? The chief raison d'être of the play construct in Ansible is that each play targets a single set of remote machines. This set is denoted by *hosts*. For example,

```

---
- hosts: myServers
  tasks:
  - name: create file
    create_file:
      file: /home/remotemachines/myfile.txt

```

Code Block 38

The above is a playbook consisting of one play, which targets a group of remote machines called myServers. myServers will be defined in an inventory file somewhere. In this thesis, we will not deal with hosts nor the inventory structure; this is just for context. Indeed, although in real Ansible the hosts keyword is compulsory for a play, we will not use it in this thesis. A play can be distinguished by its use of the roles, tasks and/or handlers section.

This is a single play:

```

- roles:
  - role1
  tasks:
  - name: first task
    debug:
      msg: "hi!"
  - name: second task
    debug:
      msg: "bye!"

```

Code Block 39

This is a list of two plays:

```

- roles:
  - role1
- roles:
  - role1

```

Code Block 40

This is also a list of two plays:

```

- roles:
  - role1
- tasks:                                # the extra dash at `tasks` indicates a new
  - name: first task                    # list item in YAML, meaning this is a different
  debug:                               # play
    msg: "hi!"
  - name: second task
  debug:
    msg: "bye!"

```

Code Block 41

Recall that for our purposes, a play is mostly just a container where more interesting constructs like tasks, handlers and blocks find themselves due to the mandated structure of Ansible.

The desugaring of plays and playbooks is simply to remove all the tasks from them after all the other desugaring steps have been done, and arrange them in one long list.

7.5.7.1 Rewrite Rule

$$\begin{array}{c}
\forall p_i \in \bar{p} (p_i = \text{Play}\{\text{roles} : [], \text{tasks} : \bar{t}_i, \text{handlers} : []\}) \\
\forall t \in \bar{t}_i (t.\text{attSet}.\text{loop} \neq \varepsilon) \\
\forall t \in \bar{t}_i : t \text{ is Blocktask} \\
\exists \text{VAR at some level within } p_i \\
\hline
\bar{p} \rightarrow \bar{t}_1 \# \bar{t}_2 \# \dots \# \bar{t}_n
\end{array}$$

7.6 Summary of Rewrite Rules

As mentioned previously, this should be taken as a translational definition of $\mathcal{A}^{\min}$ from $\mathcal{A}^H$.

DS_INLINE_RULE

$$\frac{\bar{t}' = \text{map}(\text{toImport}, \bar{r}) \# \bar{t}}{\text{Play} \{ \text{roleNames} : \bar{r}, \text{tasks} : \bar{t} \} \rightarrow \text{Play} \{ \text{roleNames} : [], \text{tasks} : \bar{t}' \}}$$

DS_IMPORT_ROLE_TASKS

$$\begin{array}{c}
\text{rd}_1 = \text{RootDir} \{ \text{playbook} : \bar{p} \}, \\
\exists p \in \bar{p} : \exists t \in p.\text{tasks} : t.\text{modDecl of type ImportRole} \vee t.\text{modDecl of type ImportTasks} \\
\bar{p}' = \text{map}(\text{desugarImportsInPlay}_{\text{rd}}, p) \\
\text{rd}_2 = \text{rd}_1 \oplus \{ \text{playbook} : \bar{p}' \} \\
\hline
\text{rd}_1 \rightarrow \text{rd}_2
\end{array}$$

DS_VAR

$$\frac{\exists \text{VAR at some level within } p}{p \rightarrow \text{resolvePlay}(p)}$$

DS_BLOCK

$$\begin{array}{c}
\exists t \in \bar{t} : t \text{ is Blocktask} \vee \exists h \in \bar{h} : h \text{ is Blocktask} \\
\bar{t}' = \text{concatMap}(\text{prepareIfBlock}, \bar{t}) \\
\bar{h}' = \text{concatMap}(\text{prepareIfBlock}, \bar{h}) \\
\hline
\text{Play} \{ \text{tasks} : \bar{t}, \text{handlers} : \bar{h} \} \rightarrow \text{Play} \{ \text{tasks} : \bar{t}', \text{handlers} : \bar{h}' \}
\end{array}$$

DS_LOOP

$$\begin{array}{c}
t.\text{attSet}.\text{loop} \neq \varepsilon \\
\bar{v}_i = \{ \langle t.\text{loop}.\text{loopVar} \rightarrow t.\text{loop}.\text{loopList}[i], \langle t.\text{loop}.\text{indexVar} \rightarrow \text{toString}(i) \rangle \} \quad \forall i \in \text{length}(t.\text{loop}.\text{loopList}) \\
t'_i = \text{resolveTask}_{\bar{v}_i}(t) \\
t_{\text{gk}} = \text{createLoopGK}([t'_1, t'_2, \dots]) \\
\hline
t \rightarrow [t'_1, t'_2, \dots, t_{\text{gk}}]
\end{array}$$

DS_HANDLER

$$\begin{array}{c}
\bar{h} = [h_1, h_2, \dots, h_n], \bar{t} = [t_1, t_2, \dots, t_m] \\
s_{h_i} = h_i.\text{attSet}.\text{listen} \quad \forall h_i \in \bar{h} \\
s_{t_j} = t_j.\text{attSet}.\text{notify} \quad \forall t_j \in \bar{t} \\
w_i = \bigvee_{t_j} (t_j \text{ is defined}) \forall h_i \in \bar{h}, \quad \forall t_j : \exists s \in s_{h_i} : s \in s_{t_j} \\
t_{h_i} = h_i \oplus \{\text{attSet} : h.\text{attSet} \oplus \{\text{when} : (h_i.\text{when} \wedge w_i)\}\} \forall h_i \in \bar{h} \\
t' = [t_1, t_2, \dots, t_m, t_{h_1}, t_{h_2}, \dots, t_{h_n}] \\
\hline
\text{Play}\{\text{tasks} : t, \text{handlers} : h\} \rightarrow \text{Play}\{\text{tasks} : t', \text{handlers} : []\}
\end{array}$$

DS_PLAY_PLAYBOOK

$$\begin{array}{c}
\forall p_i \in \bar{p} (p_i = \text{Play}\{\text{roles} : [], \text{tasks} : \bar{t}_i, \text{handlers} : []\}) \\
\forall t \in \bar{t}_i (t.\text{attSet}.\text{loop} \neq \varepsilon) \\
\forall t \in \bar{t}_i : t \text{ is Blocktask} \\
\exists \text{VAR at some level within } p_i \\
\forall t \in p_i.\text{tasks} : t.\text{modDecl of type ImportRole} \vee t.\text{modDecl of type ImportTasks} \\
\hline
\bar{p} \rightarrow \bar{t}_1 \# \bar{t}_2 \# \dots \# \bar{t}_n
\end{array}$$

8 From $\mathcal{A}^{\min}$ to BBFS

8.1 Translating Modules

As described previously in this thesis, when a module is written by a member of the Ansible community, we intend for them to write a BBFS annotation as part of the documentation.

One might notice that BBFS has no real mechanism to account for variables and parameters. For example, take the following Python code:

```
def create_or_delete_file(path: str, mode: str):
    match mode:
        case "create":
            if not Path(path).exists():
                Path(path).touch()
        case "delete":
            if Path(path).exists():
                shutil.rmtree(path)
```

Code Block 42

This corresponds to a module `create_or_delete_file` with two parameters: `path` and `mode`. We might use it in a task as such:

```
- create_or_delete_file:
    path: "/home/remotemachine/old_file.py"
    mode: "delete"
```

Code Block 43

This will idempotently delete `/home/remotemachine/old_file.py`; we know this because we can see the arguments passed to it in this particular Ansible script.

However, when the module author wishes to generically annotate the module, there is no way to represent this with just BBFS. We introduce TBBFS (Templated BBFS):

8.1.1 Syntax

Strings $s ::= \dots$

Paths $p ::= s \mid s/p \mid \text{TSub}_p(s)$

Booleans $b ::= \text{true} \mid \text{false} \mid \text{TSub}_b(s)$

Logical File System $\hat{\sigma} ::= \{p_1 \rightarrow b_1, p_2 \rightarrow b_2, \dots\}$

Expression $e ::= \text{err} \mid \text{if ask}(\hat{\sigma}) \text{ then } e_1 \text{ else } e_2 \mid e_1; e_2 \mid \text{unsafeTransform}(\hat{\sigma}) \mid \text{TChoice}(s) \{ \overline{\langle s \rightarrow e \rangle} \}$

8.1.2 Semantics

Let Δ be a mapping of strings to strings that comes from the parameters of the module. For instance, Δ from Code Block 43 will be:

$$\{\text{path} : \text{/home/remotemachine/old_file.py}, \text{mode} : \text{delete}\}$$

$$\text{TChoice}(s_0) \{s_1 \rightarrow e_1, s_2 \rightarrow e_2, \dots\}(\Delta) \stackrel{\text{def}}{=} e_k \text{ if } s_0 \rightarrow s_k \in \Delta$$

$$\text{TSub}_p(s_0)(\Delta) \stackrel{\text{def}}{=} p_0 \text{ if } s_0 \rightarrow s_p \in \Delta : \text{toPath}(s_p) = p_0$$

$$\text{TSub}_b(s_0)(\Delta) \stackrel{\text{def}}{=} b_0 \text{ if } s_0 \rightarrow s_b \in \Delta : \text{toBool}(s_b) = b_0$$

Let us now see a simple example. To annotate the above `create_or_delete_file` module, the author might write a program in TBBFS:

```

TChoice("mode") {
  "create": [
    | unsafeTransform({TSub("path"): true})
  ],
  "delete": [
    | unsafeTransform({TSub("path"): false})
  ]
}

```

Figure 16

TChoice (Template Choice) and TSub (Template Substitution) are constructs which are evaluated statically based on the parameters supplied. This template resolution phase is completed before the idempotence analysis is run, and yields plain BBFS.

For example, in Code Block 43, we know that `create` is assigned value `"/home/remotemachine/old_file.py"`, and `mode` is assigned value `"delete"`.

Given this, we first resolve TChoice by choosing the option indexed by the value of `mode`, which yields a TBBFS expression:

```
unsafeTransform({TSub("path"): false})
```

Figure 17

We then resolve TSub by substituting in the value of `path` to yield simply:

```
unsafeTransform({/home/remotemachine/old_file.py: false})
```

Figure 18

At this point, we have plain BBFS with variables that appear hardcoded, and only now do we perform our idempotence analysis.

Our method will of course disallow dynamic variable substitution, which remains fully beyond scope for now.

8.1.3 Goalkeepers

Now, it is possible for us to write the goalkeeper modules. First, we write the block goalkeeper:

```

TChoice("mode") {
  "BRA": [
    if ask({TSub("a"): true}) then
      if ask({TSub("b"): true}) then
        | nop
      else
        if ask({TSub("r"): true}) then
          | nop
        else
          | err
      else
        | err
    ]
  "BR": [
    if ask({TSub("b"): true}) then
      | nop
    else
      if ask({TSub("r"): true}) then
        | nop
      else
        | err
    ]
  "BA": [
    if ask({TSub("b"): true}) then
      if ask({TSub("a"): true}) then
        | nop
      else
        | err
    else
      | err
  ]
  "B": [
    if ask({TSub("b"): true}) then
      | nop
    else
      | err
  ]
}

```

Figure 19

The loop goalkeeper is more complex, and in fact steps out of the bounds of TBBFS. We must have a special function that dynamically generates a module for each length of loop that we encounter in the Ansible script.

Our loop goalkeeper needs to do two jobs:

1. Mark itself as having run successfully if all loop subtasks are successful
2. Error out itself if any loop subtasks fail

For job 1, we need to generate an *and* expression for all abstract success paths of the subtasks. It will take the form:

```

1  if ask({TSub:("s1"): true}) then
2      if ask({TSub:("s2"): true}) then
3          if ask({TSub:("s3"): true}) then
4              ...
5              if ask({TSub:("sn"): true}) then
6                  | mS
7              else
8                  | err
9              ...
10         else
11             | err
12     else
13         | err
14 else
15     | err

```

Figure 20

In this thesis we do not allow the user to manually specify the when attribute. This means that a loop's subtasks and the goalkeeper always share the same when attribute, which in turn means we would not encounter any situation where the goalkeeper runs but one or more of the subtasks do not run. Therefore we do not need to first check if these success paths are defined in Figure 20.

Furthermore, since all subtasks must have run for the goalkeeper to run, if success is not found for a task, it implies that it must have failed (this is not true in general for other tasks, because a task not succeeding could have been because it never ran in the first place). Thus, we can directly error out the way we do in Figure 20, taking care of job 2.

$$\text{writeLoopGK}(n) = \begin{cases} \text{if ask}(\{\text{TSub} : \text{"successPath}(n)": \text{true}\}) \text{ then writeLoopGK}(n - 1) \text{ else err} & \text{if } n > 0 \\ \text{mS} & \text{otherwise} \end{cases}$$

8.2 Expanding Translation from Modules to Tasks

$\mathcal{A}^{\min}$ is essentially a single long list of tasks, where execution always flows down this list. Control flow is done through predicated execution; when the execution reaches a task, the task decides whether or not it should run based on whether certain earlier tasks were run, and if so, whether they succeeded or failed. For an example, we can refer to Code Block 25, where a try-catch control flow was implemented in this manner.

As such, when these tasks are translated to BBFS, it is important for us to be able to keep track of:

- If a task was run
- If a task succeeded
- If a task failed

At first glance, this seems impossible to do in BBFS, since we only keep track of the filesystem. However, we will simply appropriate a family of paths to store these variables; these paths are assumed to be unreachable by programs (in a real implementation this would be done with a normal dictionary store, though it can technically be done with paths by having the root of all such paths be a long random hash). We also assume that all these paths are mapped to `false` at the beginning of execution of the program.

We will have one root for tracking if a task was run, one for tracking tasks which succeeded, and one for tracking tasks which failed.

For example, for tracking whether a task was run, we can consult the boolean assigned to the abstract path:

`"run"/"task1"`

which in more convenient notation we will write as `#R/task1`. Note that this path does not even include the conventional root directory.

In summary, for tracking whether a task (with some UID, say `task1`) was:

- run:
 - Formal notation: `"run"/"task1"`
 - Informal notation: `%R/task1`
- successful:
 - Formal notation: `"succeeded"/"task1"`
 - Informal notation: `%S/task1`
- successful:
 - Formal notation: `"failed"/"task1"`
 - Informal notation: `%F/task1`

For instance, a task `task1` can mark itself as having run by including:

`unsafeTransform( $\hat{\sigma}_r$ )`

where

$\hat{\sigma}_r = \{\%R/task1 : \text{true}\}$

To see this in action, let us say we have an Ansible module represented by the below Python program:

```
def nonidemp_create():
    os.makedirs("/home/remotemachine/myfolder")
```

Code Block 44

This code will create `/home/remotemachine/myfolder` if it does not yet exist. If it already exists, it will return a `FileExistsError`. Thus, this module can be represented by the BBFS:

if ask($\hat{\sigma}_Q$) then unsafeTransform($\hat{\sigma}_T$) else err (11)

where:

$\hat{\sigma}_Q = \{\text{/home/removemachine/myfolder} : \text{false}\}$
 $\hat{\sigma}_T = \{\text{/home/removemachine/myfolder} : \text{true}\}$

if we have an $\mathcal{A}^{\min}$ program:

```

- nonidemp_create:
  when: true
  ignore_errors: true
  register: reg1

- nonidemp_create:
  when: reg1 is defined and reg1 is succeeded
  ignore_errors: true
  register: reg2

```

Code Block 45

We might be tempted to simply represent the first task with Equation 11. However, if we do so, there is no way for the second task will to know whether it should run or not.

In our translation, Equation 11 is only the main body of the task; i.e. the business logic. In addition to the main body, each task will be modified according to the following steps:

- If the task's `when` attribute is not `true`, then add a check for the `when` condition.
 - If met:
 - Mark as having been run
 - Perform the main body of the task
 - Else, perform `nop`
- If `when` is `true`, then simply add `mR` before the main body.

We will call this marking operation:

$$mR_{uid} \stackrel{\text{def}}{=} \text{unsafeTransform}(\{\%R/uid\})$$

We will assume that some simple subsystem is in place to assign UIDs to all tasks. When the meaning is obvious, we might even forgo explicitly specifying the UID and write `mR`.

We also need to mark if the task succeeded. At the very end of the main body of the program, we mark the program as having succeeded. This is:

$$mS_{uid} \stackrel{\text{def}}{=} \text{unsafeTransform}(\{\%S/uid\})$$

or simply `mS` when convenient.

Marking if a task has failed is much more complex. Firstly, our notation will depend on whether the task has `ignore_errors` set to `true` or `false`.

A task can only fail due to its module failing. Module code is translated manually to BBFS by the module writer, and any point within the code that causes a failure should be translated using the BBFS `err` construct. The converse is also true: all occurrences of `err` within a module's BBFS translation should correspond to real-life code that causes failure.

If `ignore_errors` is set to `false`, then we can just leave this be with no modification. We do not need to mark failure explicitly in the state store for later tasks to use, since execution is supposed to instantly stop anyway and no other task will be run. This is what `err` already does in our BBFS semantics.

On the other hand, if `ignore_errors` is set to `true`, then we should find all occurrences of `err`, and change them to `mF`. This will continue execution, but mark the task as having failed for later control flow.

However, consider the following simple BBFS module code:


```
err; unsafeTransform({/home: false})
```

Figure 21

Clearly, Figure 21 should always immediately error out, and never run the `unsafeTransform`. We want this to stay the same even if the task that contains this module has `ignore_errors: true`. However, if we only change `err` to `mF`:

```
mF; unsafeTransform({/home: false})
```

Figure 22

We successfully mark that the task has failed, but because `mF` is just an `unsafeTransform` and does not halt BBFS execution, we perform the next `unsafeTransform`, which we do not want to do.

Of course, in this situation, we can simply institute a rule that any expressions sequenced after an `mF` are deleted. However, consider Equation 11, which we write out below using indents for structure:

```
if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_T$ )
else
  | err
```

Figure 23

We also know the following things about the task that contains the module:

- `when` is set to `true`, so we need to add `mR` before the main body.
- We want to be able to mark this task as successful if it completes without errors, so we place `mS` after the main body.
- We also want to be able to mark this task as failed if it hits an error, but `ignore_errors` is set to `true`, so we do not use `err`, but rather `mF`.

If we naively modify Figure 23 accordingly:

```
mR;
if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_T$ )
else ;
  | mF
mS
```

Figure 24

If `ask( $\hat{\sigma}_Q$ )` returns `false`, then we will execute `mF`, but as before this does not halt execution. This means we continue on to execute `mS` as well — and now our program has both failed and succeeded. This is obviously incorrect behavior.

Because `mF` does not stop execution, what we need to do is to ensure that every path taken after `mF` is executed has a guard to see if the task has already failed. If it has, it should not do anything. If it has not failed, then continue with the rest of the body of the program.

When BBFS execution hits `mF`, there are only two cases for the next instruction it executes after `mF`:

1. An instruction on the same nesting level as `mF`, but sequenced after `mF`

2. An instruction sequenced after execution jumps out of the nesting level of mF .

We have seen case 1 in Figure 22, and the latter case in Figure 24.

Unlike case 1, we cannot simply delete instructions that occur after mF . Rather, after the occurrence of mF , on the first jump out to a nesting level higher than mF , we must immediately insert an if-else:

```
if ⟨check if already failed⟩  
  | nop  
else ⟨continue rest of nesting level⟩
```

Figure 25

To check if we have already failed, we can check if the current task's failure path exists. We shall abbreviate this as cF :

$$cF_{uid} \stackrel{\text{def}}{=} \text{ask}(\{\%F/uid: \text{true}\})$$

For convenience, to specify that the UID being checked is that of the current task (as it is in this case), we can optionally notate it as cFs .

nop can simply be done thusly:

$$\text{nop} \stackrel{\text{def}}{=} \text{unsafeTransform}(\{\})$$

Modifying Figure 24 in this manner gives us:

```
mR;  
if ask( $\hat{\sigma}_Q$ ) then  
  | unsafeTransform( $\hat{\sigma}_T$ )  
else ;  
  | mF  
if cFs then  
  | nop  
else  
  | mS
```

Figure 26

And this is our final translation of the first task in Code Block 45.

Now we will see some BBFS expressions to solidify our understanding of cases 1 and 2 mentioned earlier. Note that the following:

```

mR;
if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_1$ )
else ;
  | mF ;
  | if ask( $\hat{\sigma}_Q$ ) then
    | unsafeTransform( $\hat{\sigma}_2$ )
  | else
    | unsafeTransform( $\hat{\sigma}_3$ )
mS

```

Figure 27

still counts as case 1, since the next instruction that is evaluated after mF is the nested if-else itself (which is on the same nesting level as mF), not the contents of the nested if-else.

Similarly,

```

mR;
if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_1$ )
else ;
  | mF
if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_2$ )
else
  | if ask( $\hat{\sigma}_Q$ ) then
    | unsafeTransform( $\hat{\sigma}_3$ )
  | else
    | unsafeTransform( $\hat{\sigma}_4$ )
mS

```

Figure 28

still counts as case 2.

Note that case 2 only applies if there is an expression on the same level after the jump-out. In the below BBFS, we do not need to do anything else other than convert err to mF, because every time we jump out of one level of nesting, there is no next expression encountered before we jump out another level of nesting.

```

if ask( $\hat{\sigma}_Q$ ) then
  if ask( $\hat{\sigma}_Q$ ) then
    if ask( $\hat{\sigma}_Q$ ) then
      | err
    else
      | unsafeTransform( $\hat{\sigma}_T$ )
  else
    | unsafeTransform( $\hat{\sigma}_T$ )
else
  | unsafeTransform( $\hat{\sigma}_T$ )

```

Figure 29

However, add one thing (say, mS) *after* a jump-out, and now we do need to apply case 2:

```

if ask( $\hat{\sigma}_Q$ ) then
  if ask( $\hat{\sigma}_Q$ ) then
    if ask( $\hat{\sigma}_Q$ ) then
      | err
    else
      | unsafeTransform( $\hat{\sigma}_T$ )
  else
    | unsafeTransform( $\hat{\sigma}_T$ )
else ;
  | unsafeTransform( $\hat{\sigma}_T$ )
mS // we need to insert an if-else here now

```

Figure 30

Let us now move on to the second task:

- It houses the same module as the first task, so the main body BBFS translation is still Equation 11.
- `when` is set to `false`, so we need to check if the previous task ran and succeeded before the main body.
- `ignore_errors` is set to `true`, so we need to convert `err` to `mF`.

Let us proceed step-by-step. First, we recall the BBFS translation of the module:

```

if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_T$ )
else
  | err

```

Figure 31

We add mS at the very end of this main body:

```

if ask( $\hat{\sigma}_Q$ ) then
  | unsafeTransform( $\hat{\sigma}_T$ )
else;
  | err
mS

```

Figure 32

We then add the if-else check for whether task 1 has been run and succeeded. BBFS has no primitive *and*, so we express this by nesting if-elses.

```

if cRtask1 then
  | if cStask1 then
    | if ask( $\hat{\sigma}_Q$ ) then
      | unsafeTransform( $\hat{\sigma}_T$ )
    else;
      | err
    mS
  else
    | nop
else
  | nop

```

Figure 33

Finally, we perform the err to mF conversion, inserting if-else guards when necessary:

```

if cRtask1 then
  | if cStask1 then
    | if ask( $\hat{\sigma}_Q$ ) then
      | unsafeTransform( $\hat{\sigma}_T$ )
    else;
      | mF
    if cF then
      | nop
    else
      | mS
  else
    | nop
else
  | nop

```

Figure 34

Although this is verbose and involves large amounts of if-else expressions, the execution time during the idempotence check should not be greatly affected, since there is no ambiguity around the state of these checks.

- With real filepaths, we begin not knowing if they exist or not, and thus must often explore both possibilities. If we have many such if-elses, we encounter path explosion.
- However, with these abstract paths representing task-state, we begin knowing their states completely: None of them exist (i.e. are all mapped to `false`) and so we never have to explore both possibilities. Either they are still `false`, or they have been updated to `true` and we will know that by the time we get to the if-else.

Let us formalize the process of converting a module into a task. The steps are:

1. Fetch module definition in BBFS
2. Prepend `mR` to front and append `mS` to back
3. Add `when` check if needed
4. Perform `ignore_errors` alternation if needed

8.2.1 when Conversion

Recall that `when` is of type `JBE_EXP`. This is essentially a boolean expression with:

- `and`
- `or`
- `nor`
- parentheses
- `true`
- `false`
- variables (tests such as x is defined)

For example, the `JBE_EXP`:

$$\begin{aligned} & (r_1 \text{ is defined} \wedge r_1 \text{ is succeeded}) \wedge \\ & ((r_2 \text{ is defined} \wedge r_2 \text{ is succeeded}) \vee (r_3 \text{ is defined} \wedge r_3 \text{ is succeeded})) \end{aligned} \quad (12)$$

corresponds to the `when` condition of one of the modules in Figure 12. This expression contains 6 tests:

- r_1 is defined
- r_2 is defined
- r_3 is defined
- r_1 is succeeded
- r_2 is succeeded
- r_3 is succeeded

Each test yields a value when evaluated at runtime. When execution reaches the module with this `when` condition, all tests are replaced with their respective values. If Equation 12 evaluates to `true`, the module should run. Otherwise, the module should be skipped. This is simply the mechanism of `when` which we have seen many times by now.

However, instead of `JBE_EXP`, this expression must be written in BBFS because our idempotence analysis is done in BBFS. Furthermore, when the expression evaluates to `true`, instead of literally returning `true`, we want our module code to run (since this means the `when` check has passed). If the expression evaluates to `false`, we want to run `nop`, meaning the module is skipped.

We can use a Shannon expansion to convert the `JBE_EXP` into a tree of if-else BBFS constructs.

Let:

- τ indicate a `JBE_EXP` test such as `t is successful` for some task t .
- Λ be an ordered list of tests τ .

- Φ be a mapping of tests to booleans.
- $\text{eval}(j, \Phi)$ be a function that takes a JBE_EXP j , substitutes each test with its boolean value according to Φ , and evaluates it according to conventional boolean algebra rules.
- m be the BBFS translation of the module itself (with mR and mS already added).

8.2.1.1 Formalization

$$\text{shExpand}(\Lambda, \Phi, m, j) = \begin{cases} m & \text{if } \Lambda = [] \wedge \text{eval}(j, \Phi) \\ \text{nop} & \text{if } \Lambda = [] \wedge \neg \text{eval}(j, \Phi) \\ \text{if } \text{ask}(\text{toQuery}(\Lambda[0])) & \\ \quad \text{then } \text{shExpand}(\Lambda[1:], \Phi \# \langle \Lambda[0] \rightarrow \text{true} \rangle, m, j) & \\ \quad \text{else } \text{shExpand}(\Lambda[1:], \Phi \# \langle \Lambda[0] \rightarrow \text{false} \rangle, m, j) & \text{otherwise} \end{cases}$$

$$\text{toQuery}(\tau) = \begin{cases} \{\%R/t : \text{true}\} & \text{if } \tau = t \text{ is defined} \\ \{\%S/t : \text{true}\} & \text{if } \tau = t \text{ is succeeded} \\ \{\%F/t : \text{true}\} & \text{if } \tau = t \text{ is failed} \end{cases}$$

8.2.2 ignore_errors Alteration

We would like to implement a function that takes in BBFS and outputs a modified function. We will presume some code infrastructure that allows us to treat BBFS as data. More specifically, we will treat a sequence of BBFS as a list. For example, the following program:

$\text{mR}; \text{err}; \text{if } \text{ask}(\hat{\sigma}_1) \text{ then } B_1 \text{ else } B_2; \text{unsafeTransform}(\hat{\sigma}_2); \text{unsafeTransform}(\hat{\sigma}_3); \text{mS}$

would be treated like a list:

$[\text{mR}, \text{err}, \text{if } \text{ask}(\hat{\sigma}_1) \text{ then } B_1 \text{ else } B_2, \text{unsafeTransform}(\hat{\sigma}_2), \text{unsafeTransform}(\hat{\sigma}_3), \text{mS}]$

B_1 and B_2 are the bodies of the if-else expression, and are therefore also sequences/lists. Since if-elses can be nested arbitrarily, we essentially have an irregularly nested list.

$[x, x, \text{if } a \text{ then } [x, x, x] \text{ else } [x, \text{if } a \text{ then } [x] \text{ else } [x, x], x], x]$

If-elses are similar to 2-tuples; we can represent the above as:

$[x, x, ([x, x, x], [x, ([x], [x, x]), x]), x]$

When we write the function below, we will use functions `getIfSeq` and `getElseSeq`; depending on the code infrastructure used, these can be as simple as `fst` and `snd`. We also presume the existence of functions that help us check the type of expression: `isIfElse`, `isErr`, and `isOtherElem`.

This function is relatively hard to read with the notation that we have been using thus far: I will present first a Python implementation, and only thereafter the formalism in the usual format.

```

def trav(seq, acc, idx, seenError):
    if idx < len(seq):
        if isinstance(seq[idx], BBFS_IfElse):
            ifSeq, ifSE = trav(getIfSeq(seq[idx]), [], 0, False)
            elseSeq, elseSE = trav(getElseSeq(seq[idx]), [], 0, False)
            if (ifSE or elseSE) and idx < len(seq)-1:
                restSeq, restSE = trav(seq[idx+1:], [], 0, False)
                return (
                    acc + [(ifSeq, elseSeq)] + [[], restSeq],
                    ifSE or elseSE or restSE or seenError
                )
            else:
                return trav(
                    seq,
                    acc + [(ifSeq, elseSeq)],
                    idx + 1,
                    ifSE or elseSE or seenError
                )
        elif isinstance(seq[idx], BBFS_Err):
            return acc + BBFS_mF, True
        else:
            return trav(
                seq,
                acc + [seq[idx]],
                idx + 1,
                seenError
            )
    else:
        return acc, seenError

```

Code Block 46

8.2.2.1 Formalization

$$\text{traverseSeq}(\text{seq}, \text{acc}, \text{idx}, \text{seenError}) = \begin{cases} \langle \text{case1Seq}, \text{case1SeenError} \rangle & \text{if Case 1} \\ \text{case2Cont} & \text{if Case 2} \\ \langle \text{acc} \# \text{mF}, \text{true} \rangle & \text{if Case 3} \\ \text{traverseSeq}(\text{seq}, \text{acc} \# \text{seq}[\text{idx}], \text{idx} + 1, \text{seenError}) & \text{if Case 4} \\ \langle \text{acc}, \text{seenError} \rangle & \text{if Case 5} \end{cases}$$

where:

$$\text{Case 1} = \text{idx} < \text{length}(\text{seq}) \wedge \text{isIfElse}(\text{seq}[\text{idx}]) \wedge (\text{ifSE} \vee \text{elseSE}) \wedge \text{idx} < \text{length}(\text{seq}) - 1$$

$$\text{Case 2} = \text{idx} < \text{length}(\text{seq}) \wedge \text{isIfElse}(\text{seq}[\text{idx}]) \wedge \neg((\text{ifSE} \vee \text{elseSE}) \wedge \text{idx} < \text{length}(\text{seq}) - 1)$$

$$\text{Case 3} = \text{idx} < \text{length}(\text{seq}) \wedge \text{isErr}(\text{seq}[\text{idx}])$$

$$\text{Case 4} = \text{idx} < \text{length}(\text{seq}) \wedge \text{isOtherElem}(\text{seq}[\text{idx}])$$

$$\text{Case 5} = \text{idx} \geq \text{length}(\text{seq})$$

$$\text{case1Seq} = \text{acc} \# [\text{if } a \text{ then ifSeq else elseSeq}] \# [\text{if cFs then nop else restSeq}]$$

$$\text{case1SeenError} = \text{ifSE} \vee \text{elseSE} \vee \text{restSE} \vee \text{seenError}$$

$$\text{case2Cont} = \text{traverseSeq}(\text{seq}, \text{acc} \# [\text{if } a \text{ then ifSeq else elseSeq}], \text{idx} + 1, \text{ifSE} \vee \text{elseSE} \vee \text{seenError})$$

$$\langle \text{ifSeq}, \text{ifSE} \rangle = \text{traverseSeq}(\text{getIfSeq}(\text{seq}[\text{idx}]), [], 0, \text{false})$$

$$\langle \text{elseSeq}, \text{elseSE} \rangle = \text{traverseSeq}(\text{getElseSeq}(\text{seq}[\text{idx}]), [], 0, \text{false})$$

$$\langle \text{restSeq}, \text{restSE} \rangle = \text{traverseSeq}(\text{seq}[\text{idx} + 1 :], [], 0, \text{false})$$

9 Discussion

9.1 Effect Map, Invariant Checking and Stronger Properties

When we end our symbolic execution, we will have calculated $\hat{\sigma}_E$. From Equation 10, we know that the filesystem at the end is the filesystem at the start, overridden with $\hat{\sigma}_E$. This means that given that the Ansible program has been correctly translated into BBFS, the user who runs the analysis can predict the exact result of the execution (since he will know his own computer's filesystem state), which has obvious benefits in terms of safety and ease of programming.

Similarly, easy analyses of invariants (such as always guaranteeing that a certain file will exist after the module, etc.) fall out of this result.

Another thing that can be done with the current analysis is a check for stronger properties; for instance, that for any input filesystem, the Ansible code will only ever produce a single possible end state.

9.2 Meaning of BBFS

Of course, this raises the question of whether in such a case BBFS should just serve as the implementation language of the modules themselves, if it always perfectly predicts the exact result of the filesystem operation.

Currently, although BBFS can predict exact results, the domain over which it has prediction power is limited. Namely, BBFS only covers the presence or absence of files/folders, not their content, and has no mechanism for, say, expressing complicated logic regarding interpretation of file contents to match those of general-purpose programming languages.

We think that it might be an open question, to what limit we can add expressivity to BBFS to allow for it to cover more domains, while avoiding trying to essentially solve the problem of creating a fully formalized, general-purpose industrial language, which is a wholly enormous endeavor. To this end, more research would have to be done to prioritize which domains exactly Ansible developer/IaC developers need expressivity in.

9.3 Practicality of BBFS

The minimality of BBFS makes it easier to prove properties about its programs, as was done in this thesis, and serves as a nice exercise in pondering what a minimal calculus for our analysis would be.

However, because translation of individual modules to BBFS is still done manually, we should consider the practicality of writing in BBFS. Currently, BBFS's expressivity is obviously limited by its minimality; jumps / loops, numbers and numerical operations, lists and list operations etc are all absent from BBFS. TBBFS serves to imitate some functionality of having functions with parameters, but is certainly unable to do things like recursion.

Technically, some of this functionality can be recreated through workarounds; for instance, lists of booleans could be implemented if a certain infinite set of paths, say `/mylist/0`, `/mylist/1`, etc were agreed to correspond to a global list. However, this is generally not practical.

In order for this thesis' method of analysis to be used, two options seem to suggest themselves:

- Augment BBFS with practical language features
- Use BBFS as a core or compile target for a more expressive language

We have already seen a glimpse of what the latter looks like; in this thesis, we have essentially compiled $\mathcal{A}^H$ into BBFS, whereupon we had to introduce TBBFS to make up for a lack of functions, dynamically generate goalkeepers for loops, and $\mathcal{A}^H$ does not have unbounded loops either. At the very least, it seems we would have to add jumps to BBFS, at which point BBFS becomes reminiscent

of some kind of assembly. From there, it might be interesting to explore the possibility of adding features like numbers purely by using abstract path spaces, where calculations are done on a binary level, by using paths set to `true` and `false`. However, for practical and efficiency purposes, a more conventional implementation of numbers will likely be necessary.

9.4 Time Complexity

We have not yet measured how much time the translated code takes to run. So far we do not have a construct where the BBFS translation has a different running complexity than the Ansible code it originated from. However, depending on what we might want to add in Section 9.3, we should take care that this remains the case.

It would also be ideal to test the time taken for the idempotence analysis to run on real Ansible scripts; if the time taken is too long, we could conceivably adopt a fuzzing approach instead of a full proof, similar to Waldemar et al.

9.5 Manual Translation and Translation Correction

Currently, assuming that translation of modules has to be done manually, the presumption is that a good way to disperse the burden of writing translations is by having each module’s writer write the translation as a form of documentation. This is of course still subject to the practical limits of individual programmers, where module writers might make mistakes, or might opt to ignore certain error cases, meaning the translation is incomplete.

On the other hand, it might be interesting to think of how one might automate correction of such cases of “incomplete/incorrect BBFS coverage” of a module. Tortoise is a tool that helps system administrators repair configurations by analyzing the manual repair effort of the system administrator, and comparing that to the constraints of the current configuration. Perhaps a similar approach can be taken where error reports from the community about the model not fitting the module wholly can be analyzed and compared to the current BBFS model of a module, and the module code itself.

9.6 Implementation

An implementation of the desugaring from $\mathcal{A}^H$ to $\mathcal{A}^{\min}$ was done, which takes in an Ansible project directory, lexes and parses it, and produces $\mathcal{A}^{\min}$. Due to time constraints, it was not possible to systematically test a large amount of real Ansible modules. Instead, internal test cases were written, which run successfully and serve as proof of concept of the desugaring.

Also due to time constraints, the implementation is slightly out of date from the current desugaring rules.

- Goalkeepers are not yet implemented; the limitation of not being able to nest blocks was simply accepted to be a limitation of the analysis by the author at that point.
- Imported handlers are dealt with using a different simplification of the original where if multiple handlers with the same name were called, only one would run.
- Inline roles are desugared using a different rule to `import_role`; these were found to be equivalent and thus combined in this thesis.

10 Future Work

10.1 Dynamic Variables

The lack of support for dynamic variables is by far one of the largest omissions of $\mathcal{A}^H$ compared to real Ansible. In real Ansible, `register` can not only be used to check module success/failure, but also returns a result object detailing the return value of that module. This result can be used in later tasks.

```
- name: add numbers
  adder:
    x: 2
    y: 3
  register: reg1

- name: print result
  debug:
    msg: reg1.result
```

Implementing this should be possible by including a mapping from variables to values, as done by Voogd et al., also in the context of symbolic execution.

10.2 Support for include

`include` is a version of `import` (for both roles and task files) that is done dynamically instead of at compile time. This is the only way (to the knowledge of the author) to form an infinite loop in Ansible, by having a role recursively including itself (or multiple roles including each other).

We think `include` can be implemented even without dynamic variables, by using:

- `blockMain` with one `import`
- `rescue` with another `import`
- a module that fails in order to choose between `blockMain` and `rescue`

10.3 Automating Module Translation

If we were able to develop a subset of Python (similar to Skylark) that can automatically be translated into BBFS, this would be valuable in helping adoption of BBFS, since Python is the default language for Ansible module writers. If a module writer writes his module in that subset of Python, he would not have to manually write BBFS. In some way, this idea is a subset of “Use BBFS as a core or compile target for a more expressive language” mentioned in Section 9.3.

10.4 Regexes and Groups of Paths

Instead of specifying single paths, it would be convenient to be able to specify a group of paths using regexes, so that a set of files that fit a certain pattern, such as semantically versioned files like `myfile3.10`, `myfile3.11`, etc; this would require implementation of set or list types.

10.5 Implementation

Of course, updating the implementation to be up to date would be ideal. The implementation in its current state can be seen at <https://anonymous.4open.science/r/taming-ansible-05F0>.

10.5.1 Expression Interpretation

$$\llbracket \langle e \rangle_L \rrbracket_L \in \{ \langle \hat{\sigma}, \hat{v} \rangle + \text{err} \} \rightarrow \{ \langle \hat{\sigma}, \hat{v} \rangle + \text{err} \}$$

$$\llbracket \langle e \rangle_L \rrbracket_L R \stackrel{\text{def}}{=} \begin{cases} \{ \text{err} \} & \text{if } R = \{ \text{err} \} \\ \text{concatMap}(\llbracket e \rrbracket_L, R) & \text{if } \text{err} \notin R \\ \text{concatMap}(\llbracket e \rrbracket_L, R) \cup \{ \text{err} \} & \text{otherwise} \end{cases}$$

$$\llbracket \langle e_1 \rangle_L; \langle e_2 \rangle_L \rrbracket_L \in \{ \langle \hat{\sigma}, \hat{v} \rangle + \text{err} \} \rightarrow \{ \langle \hat{\sigma}, \hat{v} \rangle + \text{err} \}$$

$$\llbracket \langle e_1 \rangle_L; \langle e_2 \rangle_L \rrbracket_L R \stackrel{\text{def}}{=} \llbracket \langle e_2 \rangle_L \rrbracket_L \llbracket \langle e_1 \rangle_L \rrbracket_L R$$

=====

$$\llbracket e \rrbracket_L \in \langle \hat{\sigma}, \hat{v} \rangle \rightarrow \{ \langle \hat{\sigma}', \hat{v}' \rangle + \text{err} \}$$

$$\llbracket \text{err} \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \{ \text{err} \}$$

$$\llbracket \text{if } a \text{ then } e_1 \text{ else } e_2 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e_1 \rrbracket_L \langle \langle \hat{\sigma}_K \cup \hat{\sigma}'_K, \hat{\sigma}_E \rangle, \hat{v} \rangle & \text{if } a = \text{ask}(\hat{\sigma}') \wedge \llbracket a \rrbracket_L \hat{\sigma} = \text{true} \\ \llbracket e_1 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{if } a \neq \text{ask}(\hat{\sigma}') \wedge \llbracket a \rrbracket_L \hat{\sigma} = \text{true} \\ \llbracket e_2 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{if } \llbracket a \rrbracket_L \hat{\sigma} = \text{false} \\ \llbracket e_1 \rrbracket_L \langle \langle \hat{\sigma}_K \cup \hat{\sigma}'_K, \hat{\sigma}_E \rangle, \hat{v} \rangle \cup \llbracket e_2 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{if } a = \text{ask}(\hat{\sigma}') \wedge \llbracket a \rrbracket_L \hat{\sigma} = \text{amb} \\ \llbracket e_1 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \cup \llbracket e_2 \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{if } a \neq \text{ask}(\hat{\sigma}') \wedge \llbracket a \rrbracket_L \hat{\sigma} = \text{amb} \end{cases}$$

where :

$$\hat{\sigma} = \langle \hat{\sigma}_K, \hat{\sigma}_E \rangle$$

$$\hat{\sigma}' = \langle \hat{\sigma}'_K, \hat{\sigma}'_E \rangle$$

$$\llbracket \text{setv}(r) \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \{ \langle \hat{\sigma}, r \rangle \}$$

$$\llbracket \text{trans}(\hat{\sigma}_1, \hat{\sigma}_2) \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \llbracket \text{if } \text{ask}(\hat{\sigma}_1) \text{ then } \text{unsafeTransform}(\hat{\sigma}_2) \text{ else err} \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle$$

$$\llbracket \text{unsafeTransform}(\hat{\sigma}_1) \rrbracket_L \langle \langle \hat{\sigma}_K, \hat{\sigma}_E \rangle, \hat{v} \rangle \stackrel{\text{def}}{=} \langle \langle \hat{\sigma}'_K, \hat{\sigma}'_E \rangle, v \rangle \text{ where :}$$

$$\hat{\sigma}'_K = \hat{\sigma}_K \setminus^{\times} \hat{\sigma}_1 \cup \hat{\sigma}_1$$

$$\hat{\sigma}'_E = \hat{\sigma}_E \setminus^{\times} \hat{\sigma}_1 \cup \hat{\sigma}_1$$

$$\llbracket \text{for } x \text{ in } l \text{ do } e \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \llbracket (\lambda x. e)l[1]; (\lambda x. e)l[2]; \dots \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \quad \forall l[i] \in l$$

$$\llbracket (\lambda x. e)r \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \llbracket e[r/x]_l \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle$$

$$\llbracket (\lambda s_1. e) \langle s_2, r \rangle \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \begin{cases} \llbracket e[\langle s_2, r \rangle]_n \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{if } s_1 = s_2 \\ \llbracket \lambda s_1. e[\langle s_2, r \rangle]_n \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle & \text{otherwise} \end{cases}$$

$$\llbracket \lambda s_1. e \rrbracket_L \langle \hat{\sigma}, \hat{v} \rangle \stackrel{\text{def}}{=} \{ \text{err} \}$$

Bibliography

- [1] R. Shambaugh, A. Weiss, and A. Guha, “Rehearsal: a configuration verification tool for puppet,” *SIGPLAN Not.*, vol. 51, no. 6, pp. 416–430, Jun. 2016, doi: 10.1145/2980983.2908083.
- [2] E. Dolstra, “The Purely Functional Software Deployment Model.” 2006.
- [3] “Chef Documentation — docs.chef.io.”
- [4] A. Community, “Ansible documentation.” [Online]. Available: <https://docs.ansible.com/>
- [5] T. Sharma, M. Fragkoulis, and D. Spinellis, “Does your configuration code smell?” in *Proceedings of the 13th International Conference on Mining Software Repositories*, in MSR '16. Austin, Texas: Association for Computing Machinery, 2016, pp. 189–200. doi: 10.1145/2901739.2901761.
- [6] T. Dai, A. Karve, G. Koper, and S. Zeng, “Automatically detecting risky scripts in infrastructure code,” in *Proceedings of the 11th ACM Symposium on Cloud Computing*, in SoCC '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 358–371. doi: 10.1145/3419111.3421303.
- [7] N. Saavedra and J. F. Ferreira, “GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, in ASE '22. Rochester, MI, USA: Association for Computing Machinery, 2023. doi: 10.1145/3551349.3556945.
- [8] A. Brogi, A. Di-Tommaso, and J. Soldani, “Sommelier: A Tool for Validating TOSCA Application Topologies,” in *Model-Driven Engineering and Software Development*, L. F. Pires, S. Hammoudi, and B. Selic, Eds., Springer International Publishing, 2018, pp. 1–22.
- [9] C. Krieger, U. Breitenbücher, K. Képes, and F. Leymann, “An Approach to Automatically Check the Compliance of Declarative Deployment Models,” [Online]. Available: <https://api.semanticscholar.org/CorpusID:53993079>
- [10] R. Opdebeeck, “Static analysis for quality assurance of Ansible infrastructure-as-code artefacts,” Crazy Copy Center Productions, Belgium, 2024.
- [11] J. Lepiller, R. Piskac, M. Schäfer, and M. Santolucito, “Analyzing Infrastructure as Code to Prevent Intra-update Sniping Vulnerabilities,” in *Tools and Algorithms for the Construction and Analysis of Systems*, J. F. Groote and K. G. Larsen, Eds., Cham: Springer International Publishing, 2021, pp. 105–123.
- [12] W. Fu, R. Perera, P. Anderson, and J. Cheney, “muPuppet: A Declarative Subset of the Puppet Configuration Language,” 2017, p. . doi: 10.4230/LIPIcs.ECOOP.2017.12.
- [13] A. Weiss, A. Guha, and Y. Brun, “Tortoise: Interactive system configuration repair,” in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2017, pp. 625–636. doi: 10.1109/ASE.2017.8115673.
- [14] O. Hanappi, W. Hummer, and S. Dustdar, “Asserting reliable convergence for configuration management scripts,” *SIGPLAN Not.*, vol. 51, no. 10, pp. 328–343, Oct. 2016, doi: 10.1145/3022671.2984000.
- [15] W. Hummer, F. Rosenberg, F. Oliveira, and T. Eilam, “Testing Idempotence for Infrastructure as Code,” in *Middleware 2013*, D. Eysers and K. Schwan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 368–388.

- [16] F. S. de Boer and M. Bonsangue, “Symbolic execution formally explained,” *Formal Aspects of Computing*, vol. 33, no. 4, pp. 617–636, Aug. 2021, doi: 10.1007/s00165-020-00527-y.
- [17] “Galaxy User Guide — Ansible Community Documentation — docs.ansible.com.”